



中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

学 校 武汉大学

参赛队号 25104860052

	1. 巫小珍
队员姓名	2. 王艳茹
	3. 王睿

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

题 目：面向高速列车轴承的无监督迁移诊断与可解释性分析研究

摘 要：

针对高速列车轴承在真实运营场景下故障数据稀缺、工况可变导致诊断模型泛化能力不足的重大挑战，本文以轴承试验台架振动数据为源域，以实际运营环境下高速列车的轴承振动数据为目标域，提出一种融合深度迁移学习与多维可解释性的智能故障诊断框架。

问题一要求面向最终的迁移任务构建源域数据的特征集。首先筛选出驱动端传感器采集的驱动端轴承信号数据，接着通过引入带通滤波、将源域数据重采样至与目标域一致的32kHz、以样本均衡为目标进行信号切片、实施 Z-score 标准化等方法对筛选出的数据执行标准化的预处理流程。对处理后的数据，从时域特征、频域谱矩特征、功率谱密度特征、关键频带能量特征等方面提取出 28 个一维特征，并通过连续小波变换（CWT）将一维振动信号转换为二维时频图像特征，形成“一维+二维”的特征体系。

问题二要求构建源域的轴承故障诊断模型。针对一维特征，广泛评测了 10 种典型的机器学习模型，最终发现多层感知机（MLP）的故障诊断效果最佳（加权 F1 分数=99.17%）。针对二维时频图像特征，构建了自定义的卷积神经网络（CNN）模型和残差网络（ResNet-18）模型进行对比分析，发现 ResNet-18 的分类效果优于自定义 CNN。而 ResNet-18（F1 分数>99.9%）的分类准确率又优于 MLP（F1 分数=99.8%），因此最终通过微调预训练的 ResNet-18 网络，构建了精度达 99.9%以上的源域诊断基准模型。

问题三要求设计一种有效的、无监督的迁移学习策略，实现从实验室台架（源域）到实际运营（目标域）的知识迁移。本文采取双轨并行的对比探索策略，同时实现并评估了基于显式统计度量的 MMD 方法和基于隐式对抗训练的 DANN 方法，并创新性地引入高置信度伪标签与全局熵最小化策略进行双重自监督优化，有效解决了跨域诊断中的分布对齐与语义对齐难题。从分类性能、领域自适应效果和特征分布三个维度评估迁移学习模型的效果，选定 DANN 为最终模型。

最后，本文构建了贯穿“事前设计—迁移过程—事后归因”的可解释性分析体系，利用 t-SNE 可视化领域特征的对齐过程，并借助梯度加权类激活图（Grad-CAM）逐层揭示模型决策依据与故障物理机理的内在关联。

实验结果表明，所提方法成功实现了领域知识的有效迁移，精准完成了对无标签目标域数据的故障标定，且模型的层次化诊断逻辑在跨域应用中表现出高度一致性。本研究不仅为解决工业领域的跨域诊断问题提供了一套高性能、高可信的解决方案，也为深度学习模型在关键安全部件中的应用落地提供了重要的理论与实践参考。

关键词： 迁移学习；轴承故障诊断；ResNet 模型；领域对抗网络（DANN）模型

目录

一、问题重述	4
1.1 问题背景	4
1.2 问题重述	4
1.2.1 数据分析与故障特征提取	4
1.2.2 源域故障诊断	5
1.2.3 迁移诊断	5
1.2.4 迁移诊断的可解释性	5
二、模型假设	5
三、问题分析	5
3.1 问题一分析	5
3.2 问题二分析	6
3.3 问题三分析	7
3.4 问题四分析	7
四、符号说明	8
五、模型的建立与求解	9
5.1 问题一的模型建立与求解	9
5.1.1 数据预处理	10
5.1.2 多维特征提取	11
5.1.3 特征提取有效性验证	17
5.2 问题二的模型建立与求解	19
5.2.1 基于一维量化特征的机器学习模型	20
5.2.2 基于二维时频图像的深度学习模型	25
5.2.3 结果分析	28
5.3 问题三的模型建立与求解	31
5.3.1 模型建立	32
5.3.2 模型的求解与结果分析	35
5.3.3 结果分析	36
5.4 问题四的模型建立与求解	40
5.4.1 建模思路	41
5.4.2 模型建立	41
5.4.3 模型的求解与结果分析	45
六、主要结论与模型评价	64
6.1 主要结论	64
6.1.1 问题一的主要结论	64
6.1.2 问题二的主要结论	64
6.1.3 问题三的主要结论	64
6.1.4 问题四的主要结论	64
6.2 模型评价	64
6.2.1 模型优点分析	64
6.2.2 模型缺点分析	65
附录	67
源程序 1：问题 1 中对 28 个一维指标的计算方法	67

源程序 2: 问题 1 中对代表性样本的可视化分析	69
源程序 3: 问题 2 中卷积神经网络训练	71
源程序 4: 问题 2 中机器学习模型训练代码	77
源程序 5: 问题 3 中 MMD LOSS 的计算方式	85
源程序 6: 问题 3 中 DANN 和迁移学习模型	85
源程序 7: 问题 3 中的迁移学习过程	87
源程序 8: 问题 4 中的 Grad-cam 图生成	90

一、问题重述

1.1 问题背景

高速列车作为我国客运交运体系的核心载体，凭借安全高效、便捷舒适和绿色低碳的显著优势，已成为支撑区域经济协同发展与民生出行的关键基础设施。轴承作为高速列车走行系统的核心旋转部件，直接承担列车车体重量和运行冲击载荷，长期服役于高转速、交变载荷、温度波动及振动干扰并存的复杂工况中。该部件的结构特性与服役环境决定了其具有较高的故障发生率，进而带来或轻或重的故障影响。在故障早期阶段，轴承微小缺陷若未及时识别，可能随运行逐步演化为准故障状态，导致列车临时降速或晚点；若发展为突发性失效（如滚动体碎裂、内外圈剥落），则可能引发车轮对卡滞乃至脱轨等安全事故，对乘客生命安全和路网运营秩序造成严重威胁。因此，实现轴承故障的精准、实时诊断具有重要的工程和现实意义。

当前高速列车轴承状态监测与诊断技术正从依赖专家经验和传统信号处理的模式向数据驱动的智能化方法演进。以深度学习为代表的人工智能技术凭借其强大的非线性特征表征与模式识别能力，在处理海量检测数据方面展现出巨大潜力，为实现高精度的智能故障诊断提供了新的范式。然而，在深度学习模型向实际工程应用转化的过程中，存在两大关键瓶颈。其一，信号特征的弱化与淹没问题。在实际运营环境中，传感器采集的振动信号常受到轮轨激励、线路不平顺、环境噪声等多重复杂干扰源的污染，导致早期微弱的故障特征信号往往被强背景噪声所淹没，信噪比极低，这极大地削弱了故障特征的显著性，直接制约了诊断模型的识别准确率。其二，真实故障样本的极度稀缺性与训练数据的分布失衡问题。出于最高级别的安全保障要求，任何处于萌芽阶段的轴承异常状态都会被立即检修，这使得在实际运营中能够采集到的真实故障数据非常稀少。这种“富余”的正常数据与“稀缺”的故障数据构成了典型的类别不平衡（Class Imbalance）问题，严重限制了需要大量均衡数据进行训练的深度学习模型的性能，导致模型泛化能力不足，易对多数类的正常样本产生误判或对少数类的故障样本产生漏判。

为突破上述困境，迁移学习（Transfer Learning）提供了一条极具前景的技术路径。该方法的核心思想是将在一个相关但不同的“源域”（Source Domain）中学到的知识迁移至“目标域”（Target Domain）。具体到本问题，可以将实验室台架环境下采集的、数量充足且标签完备的轴承振动数据作为源域，将在轨运行列车采集的、标签未知且故障样本稀缺的数据作为目标域。由于两者的故障机理具有内在的相似性，通过构建源域的轴承故障诊断模型，提取可迁移的、具有代表性的故障特征知识，并将其适配到目标域数据上，有望在目标域数据极其有限的情况下，显著提升诊断模型的准确性和鲁棒性。

在此背景下，本问题聚焦于高速列车轴承智能故障诊断这一核心挑战，旨在探索一种融合故障机理分析与数据驱动模型的解决方案。通过对实验室台架数据进行深度分析与特征提取，构建高性能的源域诊断模型，并借助迁移学习技术，将诊断知识有效迁移至列车实际运营数据，以解决真实复杂工况下因信噪比低和故障样本稀缺所导致的诊断难题。此外，深入探讨迁移诊断模型的可解释性，对于增强用户信任、推动智能诊断技术在关键安全领域的工程化应用同样具有深远意义。

1.2 问题重述

本赛题围绕高速列车轴承智能故障诊断的全流程展开，基于提供的源域和目标域数据集，需完成特征提取、源域建模、跨域迁移及可解释性分析四项子任务。

1.2.1 数据分析与故障特征提取

问题一要求面向最终的迁移任务，对源域数据进行筛选与分析。需要结合滚动轴承的故障物理机理，运用时域、频域或时频域分析等信号处理技术，结合二维图像分析，从原始振动信号中提取能够表征不同故障类别（内圈故障、外圈故障、滚动体故障及正常状态）且具有较强鲁棒性和可迁移性的特征。此任务的核心是构建一个高质量、具有代表性的特征集，为后续模型训练与跨域知识迁移奠定坚实基础。

1.2.2 源域故障诊断

问题二要求基于第一步提取的特征数据集，设计并训练一个诊断模型，以实现在源域数据上的高精度故障分类。该任务需要合理划分源域训练集与测试集，选择或构建合适的机器学习或深度学习模型结构，并对模型的诊断性能进行全面的量化评估。此模型的性能将作为后续迁移学习效果的基准，其本身也构成了待迁移的“知识源”。

1.2.3 迁移诊断

问题三要求在源域模型的基础上，深入分析并应对源域与目标域之间的数据分布差异。需要设计一种有效的、无监督的迁移学习策略，将从源域学到的诊断知识迁移并适配到目标域。最终目标是利用该迁移模型，对全部 16 个目标域数据文件进行状态标定，明确指出每个文件对应的轴承工作状态（正常、内圈故障、外圈故障或滚动体故障），并提供可视化的分析来佐证迁移过程的有效性。

1.2.4 迁移诊断的可解释性

为增强所构建的智能诊断模型在安全关键领域应用中的可信度，问题四要求对迁移诊断过程的“黑箱”特性进行剖析，从事前、迁移过程和事后中选择至少一个维度，结合轴承故障的物理特征与机理，对任务三中模型的行为进行解释。此任务的核心是揭示模型做出特定诊断决策的依据，阐明其如何克服域间差异实现知识迁移，从而提升用户对模型诊断结果的理解与信任。

二、模型假设

本文作出如下假设：

1. 假设提供的源域（实验室台架数据）和目标域（实际运营列车数据）均为真实、有效的数据，且数据采集过程符合标准规范，无人为篡改或系统性误差。
2. 假设从振动信号中提取的时域、频域及时频域特征（如 RMS、峭度、CWT 时频图等）能够充分反映轴承的健康状态，且这些特征与故障机理（如 BPFO/BPFI 故障频率）具有物理关联性。
3. 假设通过域适应技术（如 MMD、DANN）能够有效对齐两域的特征分布，缩小因采样频率、噪声水平等差异导致的域偏移。
4. 假设在源域上训练的高性能模型（如 ResNet-18）通过迁移学习后，能够泛化至目标域，且其诊断结果在无标签目标数据上具有可信度。
5. 假设通过 SHAP 值、t-SNE 可视化等技术能够解释模型的决策依据，且这些解释与轴承故障的物理机理一致，从而增强工程人员的信任度。
6. 假设源域数据中的轻微噪声（如台架试验干扰）可通过滤波和特征选择消除，且缺失数据比例不超过 30%，不影响模型构建。

三、问题分析

3.1 问题一分析

针对问题一，核心任务是基于给定的轴承试验台架振动数据（源域），构建一个能够有效表征故障状态并服务于后续迁移学习任务的特征体系。鉴于源域（试验台架环境）与目标域（列车实际运营环境）之间存在显著的工况和噪声差异，本问题的解决质量直接决定了后续诊断模型的精度、泛化能力及迁移学习的成败。为此，将该任务分解为三个逻辑递进的子任务：

（1）面向迁移的数据预处理。首先采取目标导向的数据筛选策略，从 161 个源域文件中精心挑选出覆盖所有工作状态（正常、外圈故障、内圈故障、滚动体故障）、载荷工况（0、1、2、3 马力）及故障尺寸（0.007、0.014、0.021、0.028 英寸）的代表性样本。这一决策的理论依据在于，通过迫使模型在多样化的工况下学习故障的不变性特征（Invariant Features），可以显著提升其鲁棒性，从而提取出更具普适性、可迁移的诊断知识。同时，由于距离故障轴承较近的采集位置振动传递路径短，故障信号的幅值更明显，因此选择驱动端（DE）传感器采集的驱动端轴承信号作为主要分析对象，以最清晰地捕捉故障产生的瞬态冲击。随后，对筛选出的数据执行标准化的预处理流程，包括引入带通滤波滤除噪声、将源域信号重采样至与目标域一致的 32kHz、以样本均衡为目标进行信号切片、对每个样本实施 Z-score 标准化等。

（2）多维度信息互补的特征提取。轴承故障信号是典型的非平稳信号，其特征信息分布在时域、频域和时频域中，单一维度的特征提取方法存在信息损失的风险。因此，本文设计并实施了一套“一维特征向量+二维时频图像”并行的多维度特征提取方案。其中一维统计特征向量提取了涵盖时域统计特征、频域谱矩特征、功率谱密度特征、关键频带能量特征以及基于小波变换的能量统计特征的 28 个经典时域与频域统计指标，接着运用连续小波变换（Continuous Wavelet Transform, CWT）算法将一维振动信号转化为二维图像，将复杂的信号分类问题范式转换为成熟的图像分类问题，使得后续能够直接利用卷积神经网络（CNN）进行深度学习。

（3）基于物理机理的特征有效性验证。为确保所构建的特征体系并非简单的数学变换，而是真实反映了轴承的故障机理，本文增加了一个验证环节，选取典型的故障样本，通过可视化分析，将信号的时域波形、快速傅里叶变换（FFT）频谱图和 CWT 时频图进行综合展示。在 FFT 频谱图上，根据轴承参数和转速计算并标注了内圈故障特征频率（BPFI）及其 2 倍、3 倍谐波。通过观察频谱峰值与理论频率的吻合程度及时频图上的能量集中模式，可以直观地验证所提取特征的有效性。

本题的重点和难点在于源域和目标域数据处理的规范化和对齐，以及在众多时频分析方法中如何选择并论证连续小波变换（CWT）的优越性。

3.2 问题二分析

问题二要求在问题一构建的故障特征基础上，划分源域训练集和测试集，设计并遴选出一个合适的源域故障诊断模型。所设计的模型需仅利用试验台架数据，对正常、滚动体故障、内圈故障及外圈故障四种工作状态进行高精度分类。最后需要选取恰当的指标对分类结果加以评估，进而遴选出一个较优的故障诊断模型。

从模型建立方面看，基于问题一构建的“一维特征向量+二维时频图像”特征体系和特征-模型匹配原则，可以针对不同模态的特征分别建立适宜的模型，然后对比择优。一维特征向量是对原始信号在统计学、频域物理学等维度上的高度凝练与抽象，其优点在于维度较低、计算开销小且具有明确的物理可解释性，是传统机器学习算法的优势领域。通过 CWT 生成的二维时频图像保留了信号能量在时间-频率二维平面上的完整分布信息，这种数据形态蕴含着丰富的局部结构、纹理及上下文信息，其高维度和复杂性超越了传统机器学习模型的处理能力，但天然契合了卷积神经网络（CNN）的核心机制——通过卷积核学习空间层级特征。因此，采用双轨并行的建模策略是科学且必要的，它能够为各自的特征

形态匹配最适宜的算法范式，从而最大限度地发挥每种特征的潜力。

在具体的模型选择上，对于基于一维特征的机器学习模型，本文首先利用随机森林分类器选取出重要性排名前 15 的特征构成最终的特征子集。然后通过广泛的横向评测系统训练并评估了十种主流的机器学习分类器，包括逻辑回归、决策树、随机森林、支持向量机（SVC）、K 近邻（KNN）、梯度提升机（GBM）、AdaBoost、XGBoost、多层感知机（MLP）以及 HistGradientBoosting，最终发现 MLP 分类器表现最为出色。基于二维图像的深度学习模型的选择则是采取“基准—深化”的递进策略。首先构建了一个自定义的轻量级 CNN 作为性能基准，以评估从零开始训练的可行性。接着引入预训练 ResNet-18 模型并对其进行微调，这样做的好处主要有两方面：一是 ResNet 在海量通用图像库上学习到的强大基础特征提取能力是高度可复用的，将其迁移至时频图谱识别任务可以极大地加速模型收敛并提升性能，有效克服自身数据集规模相对有限的问题；二是 ResNet 的残差连接设计从根本上解决了深度网络训练中的梯度消失问题，使得构建更深、表达能力更强的模型成为可能，能够确保对复杂时频模式的深度挖掘。

在模型评估层面，所有模型均在划分出的测试集上进行评估，并以加权 F1 分数（Weighted F1-Score）作为核心评判指标。该指标能综合考量模型的精确率（Precision）和召回率（Recall），对于多分类问题，比单一的准确率更能稳健地反映模型的综合分类性能。

3.3 问题三分析

本任务的核心在于解决一个典型的无监督领域自适应（Unsupervised Domain Adaptation, UDA）问题。问题二中构建的源域诊断模型，尽管在实验室台架数据（源域）上表现出卓越的性能，但其直接应用于实际运营列车数据（目标域）时，性能预期将出现显著下降，出现这一现象的根本原因在于领域偏移。对此，本文的核心策略是学习领域不变性特征（Domain-Invariant Features），其核心思想是：尽管源域和目标域的原始信号在表观上存在巨大差异，但控制其状态的物理机理是共通的。因此，本文通过问题二优选的 ResNet 模型，将原始数据映射到一个共享的、高维的潜在特征空间。在这个空间中，由领域特异性导致的数据差异被最大程度地消除，而由故障类型决定的内在判别性信息则被保留和强化。

领域自适应领域存在多种主流技术路径，为避免单一技术路线可能存在的局限性，本文设计了一种双轨并行的对比探索策略，同时实现并评估基于显式统计度量的 MMD 方法和基于隐式对抗训练的 DANN 方法这两种最具代表性的方法。MMD 代表了显式对齐的思路，即直接定义一个“距离”，然后命令模型去缩小该距离；DANN 则代表了隐式混淆的思路，即不直接告诉模型如何对齐，而是创造一个“对手”，迫使模型在博弈中学会如何隐藏自己的领域身份。通过对比这两种不同哲学的方法，能够更全面地评估哪种对齐策略更适合本任务的特征空间特性。

仅仅实现特征分布的对齐并不足以保证最终的成功，还有一个潜在的风险是语义错配：目标域的数据可能形成了与源域类别对应的簇，但分类器却将错误的标签指派给了这些簇。为解决这一关键挑战，本文设计了一套“高置信度伪标签+全局熵最小化”的双重自监督演化机制，其优越性在于能够利用目标域自身的结构信息来引导和校准分类器。

3.4 问题四分析

问题四要求在任务三构建的迁移诊断模型基础上，深入探讨其可解释性。核心目标是打破深度学习模型的“黑箱”困境，从事前（Ante-hoc）、迁移过程（In-process）和事后（Post-hoc）三个维度，结合轴承故障机理，对模型的结构设计、知识迁移路径及最终决策依据进行全面、透明的分析，从而增强使用者对模型诊断结果的理解与信任。

从模型建立方面看，本题的可解释性分析并非一个独立的建模任务，而是依附于任务三的迁移模型，为其建立一套“透明化”的分析与验证体系。此体系的构建依赖于一系列先进的可视化与分析技术。采用 t-SNE 降维算法，将模型在高维特征空间的学习结果映射至二维平面，用以直观揭示迁移学习前后领域特征分布的变化。同时，为探究模型做出具体诊断的“视觉得分点”，将引入梯度加权类激活图（Grad-CAM）技术，生成热力图，高亮显示输入图像中对最终分类决策贡献最大的区域，从而将模型的抽象判断与时频图中的具体物理特征联系起来。

从模型求解方面看，对于模型可解释性的分析将是一个贯穿事前、迁移过程、事后的多层次论证过程。在事前可解释性分析中，本文将回顾并论证“CWT 时频图+ResNet-18”这一技术路线与轴承故障物理机理的内在一致性。迁移过程中，利用 t-SNE 对迁移模型（DANN）的全连接层前一层的输出特征进行降维，通过对比分析“迁移前源域模型对目标域数据的表征”与“迁移后模型对目标域数据的表征”，直观展示模型如何从“领域混淆”走向“领域自适应”。事后，将开展关键的案例分析，选取每个预测类别中置信度最高的样本，利用 Grad-CAM 生成其决策热力图，并与源域中同类故障样本的热力图进行对比，以可视化的方式证明模型学到的诊断知识是普适且有效的，并且这种知识成功地从源域迁移到了目标域。

本题的重点和难点在于，如何将抽象的数学模型内部状态（如特征向量、梯度）与具体的工程物理知识（如 BPFO、调制现象）建立起强关联。重点在于构建一个从“模型设计初衷”到“中间过程验证”再到“最终结果归因”的完整解释链条。难点则在于：一是如何深刻理解并清晰阐述 Grad-CAM 热力图高亮区域所对应的物理意义；二是如何通过对比源域与目标域的激活图，有力地论证知识迁移的一致性与有效性，即证明模型在两个不同领域关注的是同一种本质的故障模式。

四、符号说明

表 4-1 符号说明

符号	意义	符号	意义
$\mathcal{D}_s$	带标签的源域数据集	$\mathcal{L}_{cls}$	源域上的监督分类损失（通常为交叉熵损失）
$\mathcal{D}_t$	无标签的目标域数据集	$\mathcal{L}_{MMD}$	最大均值差异损失，用于度量域间分布差异
$(\mathbf{x}^s, y^s)$	源域中的一个样本，其中 $\mathbf{x}^s$ 为数据， y^s 为标签	$\mathcal{L}_d$	领域判别损失，用于训练 DANN 中的 G_d
$\mathbf{x}^t$	目标域中的一个无标签样本	$\mathcal{L}_{pl}$	高置信度伪标签损失
$P(\mathbf{X}_s), P(\mathbf{X}_t)$	源域和目标域数据的边缘概率分布	$\mathcal{L}_{ent}$	目标域预测的熵最小化损失
$W_x(a, b)$	信号 $x(t)$ 在尺度 a 和平移 b 处的连续小波变换（CWT）系数	$\lambda_{adv}, \gamma, \delta$	分别用于平衡领域对抗、伪标签和熵最小化损失的超参数

$\psi(t)$	用于 CWT 的母小波函数	$KL(P\ Q)$	P 分布与 Q 分布之间的 KL 散度, t-SNE 的优化目标
G_f	特征提取器网络	y^c	模型对类别 c 的预测分数 (Softmax 层前)
G_y	标签分类器网络	A^k	CNN 最后一个卷积层的第 k 个特征图
G_d	领域判别器网络, 用于 DANN 模型	α_k^c	对于类别 c, 第 k 个特征图的重要性权重 (在 Grad-CAM 中计算)
$\theta_f, \theta_y, \theta_d$	分别对应网络 G_f, G_y, G_d 的可学习参数	$L_{\text{Grad-CAM}}^c$	最终生成的针对类别 c 的梯度加权类激活图 (热力图)

五、模型的建立与求解

5.1 问题一的模型建立与求解

问题一要求面向最终的迁移任务，对源域数据进行筛选与分析，并构建一个高质量、具有代表性的特征集，为后续模型训练与跨域知识迁移奠定坚实基础。对此，按照“数据处理与分析—特征提取—有效性验证”的思路加以解决。问题一的研究思路如图 5-1 所示。

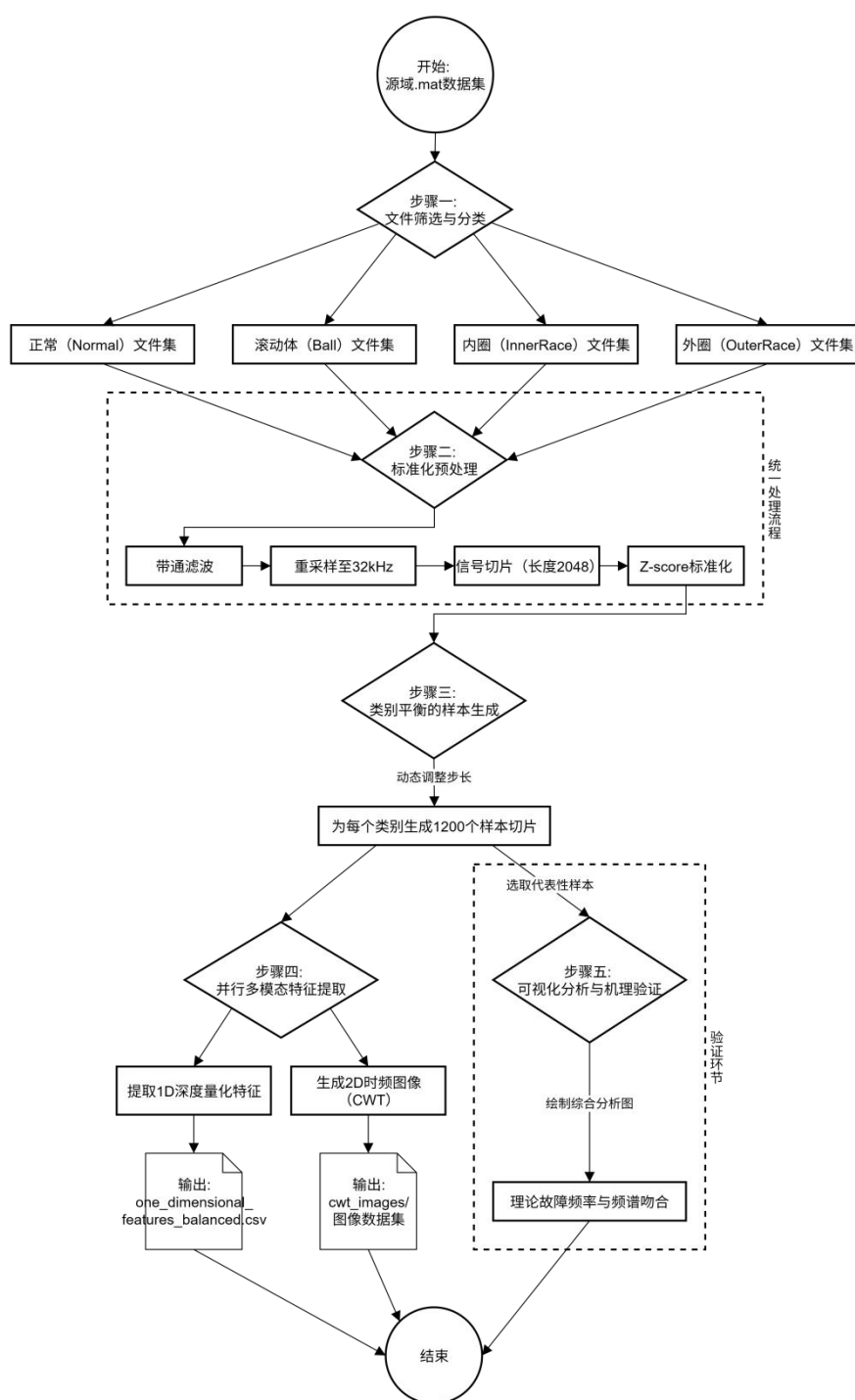


图 5-1 问题一流程图

5.1.1 数据预处理

题目中共给出两组数据集，两组数据集中轴承都包含外圈故障（OR）、内圈故障（IR）、滚动体故障（B）以及正常（N）4种工作状态。一是161个具有代表性的轴承试验台架振动数据文件，作为源域数据集，驱动端轴承采样频率为12KHz和48KHz，风扇端轴承采样频率为12KHz，各数据所属工作状态已知；二是16个实际的列车轴承故障文件，作为目标域数据集，采样频率为32KHz，各数据所属工作状态未知。首先对源域数据集中的全部161个.mat格式文件进行自动化扫描与识别，依据文件命名规则中包含的故障类型标识（N、B、IR、OR），将所有文件精确地归类至正常、滚动体故障、内圈故障和外圈故障

四个类别中，为后续数据处理奠定结构化基础。

针对每个类别中的数据文件，由于距离故障轴承最近的采集位置振动传递路径短，故障信号的幅值更明显，本文挑选出驱动端传感器采集到的驱动端轴承信号数据作为分析对象，共 116 个文件，并对筛选后的样本按类别进行了统计，确保全面覆盖外圈（OR）、内圈（IR）、滚动体（B）故障和正常（N）四种工作状态，涵盖 0 至 3 马力四种不同的载荷情况及 0.007、0.014、0.021、0.028 英寸四种故障尺寸，以保证各类样本数量的均衡性，为后续模型训练提供良好的数据基础。

完成数据筛选后，需对其执行一套预处理流程，包括滤波、重采样、信号切片和 Z-score 标准化。为凸显由故障产生的周期性冲击信号，并抑制工频和谐波干扰以及高频随机噪声，以增强信噪比，首先对原始信号施加一个四阶巴特沃斯带通滤波器，将分析的频带限制在 100 赫兹至 10000 赫兹之间，使之聚焦于故障信息富集的频带。由于源域信号采样频率（12kHz 或 48kHz）与目标域采样频率（32kHz）不一致，为消除源域和目标域在数据采集层面的差异，采用基于傅里叶方法的 `scipy.signal.resample` 重采样函数，将所有滤波后的信号统一重采样到与目标域一致的 32kHz，消除跨域采样频率不一致导致的信号周期、幅值分布偏差。接着将重采样后的长信号切分为多个独立的短信号段。鉴于源域数据中各类故障文件数量差异较大，若直接对信号进行切片，将导致各类训练样本数量严重不平衡。在模型训练中，模型的优化目标（最小化总损失）会使其倾向于预测样本量大的类别，而忽视样本量小的类别，这会导致模型对“多数类”过拟合，而对“少数类”识别能力低下。为解决该问题，本文设计了一种目标数量驱动的动态样本生成机制。具体而言，首先为每个故障类别设定一个统一的目标样本数量（1200 个），然后采用动态滑动窗口切片技术实现动态切片。该技术以 2048 个数据点为标准窗口长度，根据每个类别下可用信号的总长度和 1200 个样本的目标，自动反向计算出实现该目标所需的最优滑动步长。通过动态调整切片的滑动步长，从有限的原始文件中生成足够且数量均衡的样本，使 OR、IR、B、N 四种工作状态对应的样本量分布均等，如图 5-2 所示。采用以样本均衡为目标的切片法不仅有助于规避模型偏见，提升故障识别能力，还通过样本的高度重叠增加了数据的多样性，增强模型的泛化性能与稳定性。

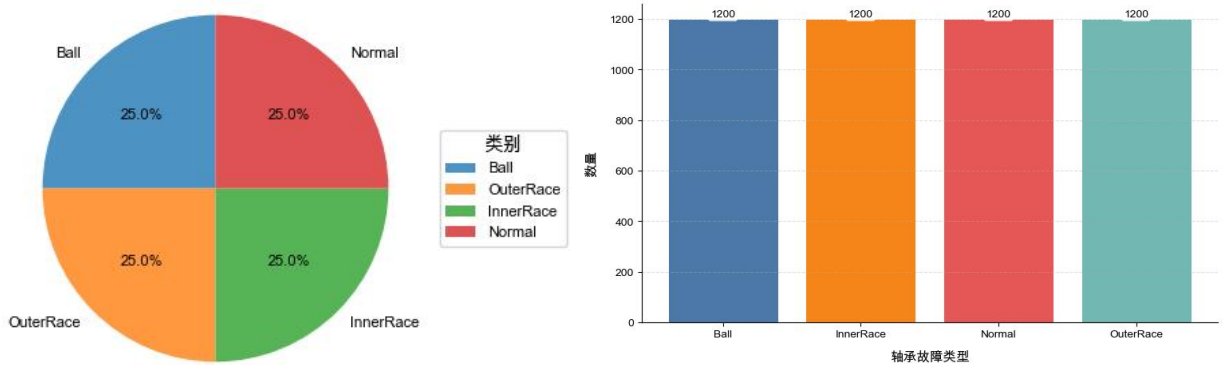


图 5-2 故障类型分布饼图和直方图

最后，对每一个信号段应用下式进行 Z-score 标准化处理，使其均值为 0，标准差为 1，消除幅值差异带来的影响。标准化后的信号数据分布更集中，可减少极端幅值对特征计算的干扰，为后续深度学习模型的权重更新提供更稳定的输入，避免因原始数据幅值波动导致模型训练震荡。

$$x_i'[n] = \frac{x_i[n] - \mu_i}{\sigma_i}, \quad n=1, 2, \dots$$

其中， μ_i 和 σ_i 分别是信号段 x_i 的均值和标准差。

5.1.2 多维特征提取

对预处理后的标准样本 x_i' ，并行构建两个特征提取模型，分别生成一维统计特征向量和二维时频图像。

1. 一维特征向量提取模型

本文共选取 28 个具有明确物理意义且广泛应用于故障诊断的特征，涵盖了时域统计特征、频域谱矩特征、功率谱密度特征、关键频带能量特征以及基于小波变换的能量统计特征，各特征定义如下：

(1) 时域特征

i. 均值 (mean)：信号所有数据点的算术平均值，计算公式为

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i。$$

ii. 均方根值 (rms)：信号平方后的均值的平方根，反映信号的有效功率或能量，计算公式为

$$\text{rms} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}。$$

iii. 标准差 (std)：衡量信号数据点相对于其均值的离散程度，计算公式为

$$\sigma = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2}$$

iv. 信号绝对最大值 (peak)：信号中所有数据点绝对值的最大值，反映信号的最大振幅，计算公式为

$$\text{peak} = \max_{i=1 \dots N} (|x_i|)。$$

v. 峭度 (kurtosis)：衡量信号波形尖峭或平坦程度的四阶统计量，计算公式为

$$\text{kurtosis} = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^4}{\sigma^4}。$$

vi. 偏度 (skew)：衡量信号概率分布对称性的三阶统计量，计算公式为

$$\text{skew} = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^3}{\sigma^3}。$$

vii. 脉冲因子 (impulse_factor)：信号峰值与其绝对值均值的比率，用于衡量信号的冲击性，计算公式为

$$\text{impulse_factor} = \frac{\text{peak}}{\frac{1}{N} \sum_{i=1}^N |x_i|}。$$

viii. 峰值因子 (crest_factor)：信号峰值与其均方根值的比率，同样用于衡量信号的冲击性，计算公式为

$$\text{crest_factor} = \frac{\text{peak}}{\text{rms}}。$$

ix. 过零率 (zero_crossings)：信号波形穿过零值（或均值）的次数，通常是计算相邻采样点符号改变的次数，公式为

$$\text{zero_crossings} = \sum_{i=2}^N \mathbb{I}(x_i \cdot x_{i-1} < 0)$$

其中 $\mathbb{I}(\cdot)$ 是指示函数，当条件为真时为 1，否则为 0。

(2) 功率谱特征

i. 功率谱总能量 (psd_energy)：功率谱密度 (PSD) 在所有频率点上的总和，根据帕萨瓦尔定理，它与时域信号的总能量成正比，计算公式为

$$\text{psd_energy} = \sum_{k=1}^K P_k。$$

ii. 功率谱均值 (psd_mean)：功率谱密度的算术平均值，计算公式为

$$\text{psd_mean} = \frac{1}{K} \sum_{k=1}^K P_k = \frac{\text{psd_energy}}{K}。$$

iii. 功率谱峰值对应频率 (psd_peak_freq)：功率谱密度达到最大值时所对应的频率，计算公式为

$$\text{psd_peak_freq} = \arg \max_{f_k} (P_k)。$$

(3) FFT 频谱特征

i. 频谱加权平均频率 (spectral_mean)：又称频谱质心，表示频谱能量的中心位置，计算公式为

$$\text{spectral_mean} = \frac{\sum_{k=1}^K f_k \cdot M_k}{\sum_{k=1}^K M_k}。$$

ii. 频谱标准差 (spectral_std)：衡量频谱能量围绕其均值频率的分布宽度，计算公式为

$$\text{spectral_std} = \sqrt{\frac{\sum_{k=1}^K M_k \cdot (f_k - \text{spectral_mean})^2}{\sum_{k=1}^K M_k}}。$$

iii. 频谱熵 (spectral_entropy)：衡量频谱的复杂度或不确定性。首先将频谱归一化为概率分布 p_k ，然后计算其信息熵，计算公式为

$$\text{spectral_entropy} = - \sum_{k=1}^K p_k \log_2(p_k), \quad p_k = \frac{M_k}{\sum_{j=1}^K M_j}$$

iv. 主频分量 (topN_freq, topN_mag)：在整个频谱中，幅值 M_k 排名最高的 N 个点所对应的频率 f_k 和幅值 M_k 。

(4) 频带能量特征

δ 、 θ 、 α 、 β 、 γ 波段能量：计算在特定频率范围 $[f_{\text{start}}, f_{\text{end}}]$ 内的能量总和，通常通过对该范围内的功率谱密度 P_k 求和得到，计算公式为

$$\text{Band Energy} = \sum_{k \text{ s.t. } f_{\text{start}} \leq f_k \leq f_{\text{end}}} P_k$$

对于 $[f_{\text{start}}, f_{\text{end}}]$ ， δ 对应 0.1-4Hz， θ 对应 4-8Hz， α 对应 8-13Hz， β 对应 13-30Hz， γ 对应 30-100Hz。

(5) 小波特征

i. 小波能量均值 (wavelet_mean)：对信号进行连续小波变换后，计算各尺度下小波系数的能量值，再对这些能量值求均值。计算公式为

$$\text{wavelet_mean} = \frac{1}{N} \sum_{i=1}^N E_i$$

其中， E_i 为第 i 个尺度下的小波能量， N 为尺度总数。

ii. 小波能量标准差 (wavelet_std)：在上述小波能量的基础上，计算能量值的标准差。

计算公式为

$$wavelet_std = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (E_i - wavelet_mean)^2}.$$

对于任意一个标准化信号段 x_i' ，定义一个 28 维的特征向量 $F_{1D,i}$ ：

$$F_{1D,i} = [f_1, f_2, \dots, f_{28}]^T$$

其中每个分量 f_k 代表一个具体的统计特征。对每一个预处理后的信号段 x_i' ，依次计算上述 28 个特征值，构成其对应的一维特征向量 $F_{1D,i}$ 。将所有样本的特征向量与其故障类别标签、来源文件等元信息整合，最终保存为一个名为 `one_dimensional_features.csv` 的表格文件。

在构建一维特征提取模型后，得到一个包含 4800 个样本、28 个特征的综合特征集。为深入评估这些特征的质量、理解其内在分布规律、探究特征间的相互关系，并最终验证其对故障诊断任务的潜在价值，本文实施了系统的探索性数据分析（EDA）。

单变量特征分布分析。首先，对整个特征数据集进行完整性检验。如图 5-3 所示，所有特征列（包括元数据 `label`, `slice_index` 等）均显示了 4800 个非空值，完整度为 100%，数据集不存在任何缺失值。这表明数据预处理和特征提取流程是稳健、可靠的，生成了一个高质量、可以直接用于机器学习建模的“干净”数据集。这免去了复杂的数据插补步骤，保证了后续模型训练的稳定性和结果的可靠性。

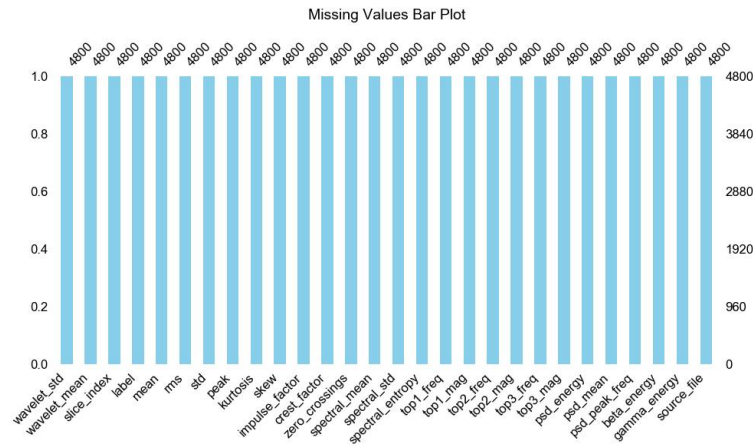


图 5-3 数据集特征列

接着，通过箱线图和直方图对每个特征的分布进行可视化分析，以揭示其统计特性。从图中可以看出，冲击性指标（K、IF、CF）表现出显著的区别潜力。这三个特征的直方图均呈现出明显的右偏态分布，箱线图也显示其绝大部分数据点集中在较低的数值区，但存在大量数值极高的异常值。这种分布形态与轴承故障的物理机理高度吻合，正常状态或早期故障的振动信号较为平稳，其冲击性指标值较低，构成了分布的主体。而当轴承发生局部故障时，滚动体每次碾过损伤点都会产生剧烈的瞬时冲击，导致信号的峭度、脉冲因子和峰值因子急剧增大，这些高价值点即是箱线图上的“异常值”。其次，多个频域相关特征（Smean, Sstd, T1freq 等）的直方图展示出多模态特性，即数据分布存在多个峰值，例如 Smean 和 T1freq 的分布图均有两到三个明显的峰值。这暗示了数据内部存在异构的子群体，这些子群体可能对应着不同的轴承工作状态。不同状态下，信号的能量会集中在不同频带：正常状态能量主要在低频的转频附近；而不同类型的故障会激发不同的系统共振频率，或在其独特的故障特征频率及其谐波处产生能量集中。这些不同的能量分布模式，最终体现在频谱统计特征的多个众数上。此外，小波能量特征（Wmean, Wstd）提供了独

特的时频视角。 $Wmean$ 的分布也呈现出多模态的趋势，而 $Wstd$ 则展现了较好的数据区分度。 $Wstd$ 衡量的是信号能量在时频平面上的不均匀性，平稳的正常信号其能量分布相对均匀， $Wstd$ 值较小；而故障信号的冲击能量在时频图上表现为孤立的亮点，导致能量分布极不均匀，从而产生较大的 $Wstd$ 值。

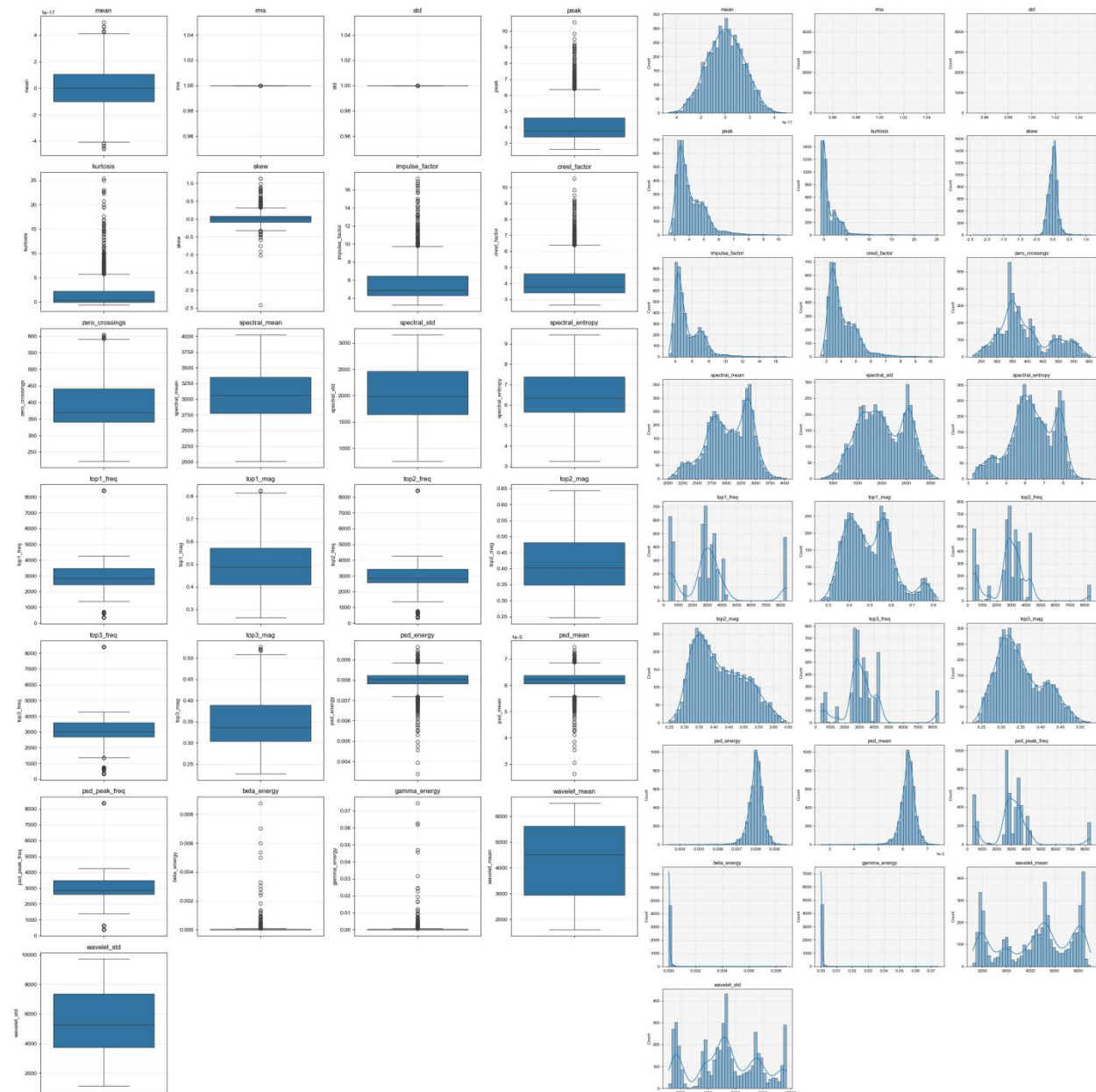


图 5-4 特征箱线图和直方图

多变量特征相关性分析。利用相关性热力图和成对关系图探索特征之间的线性关系。观察相关性热力图可以发现， P 、 K 、 IF 、 CF 四个核心冲击指标之间的相关系数均超过 0.9，表明它们之间存在极强的正相关性，证明了这些特征在衡量信号“冲击性”方面的高度一致性。当轴承出现局部故障时，会产生瞬时的高能冲击，直接导致信号峰值的剧增。这种剧增的峰值，相对于信号的均方根值（反映平均能量）和绝对值均值，必然会使峰值因子和脉冲因子同步飙升。在统计分布上，这种“尖峰”信号形态表现为高峭度。其次，两个分析图都表明 β 波段能量（5-10kHz）和 γ 波段能量（10-16kHz）之间存在显著的正相关性。这表明在本数据集的工况下，激发高频振动的物理机制（通常是故障冲击）具有宽带特性。故障冲击产生的能量并非局限于某个狭窄的频带，而是同时覆盖了 β 和 γ 两个相邻

的高频区域。spectral_entropy, wavelet_mean, wavelet_std 之间的相关性均大于 0.7，这建立了一个关键的联系：信号能量的增加和冲击性的增强（时频域指标）与频谱的复杂化（频域指标）是同步发生的。S_entropy、Wmean、Wstd 之间的相关性均大于 0.7，在一定程度上表明信号能量的增加和冲击性的增强（时频域指标）与频谱的复杂化（频域指标）是同步发生的。

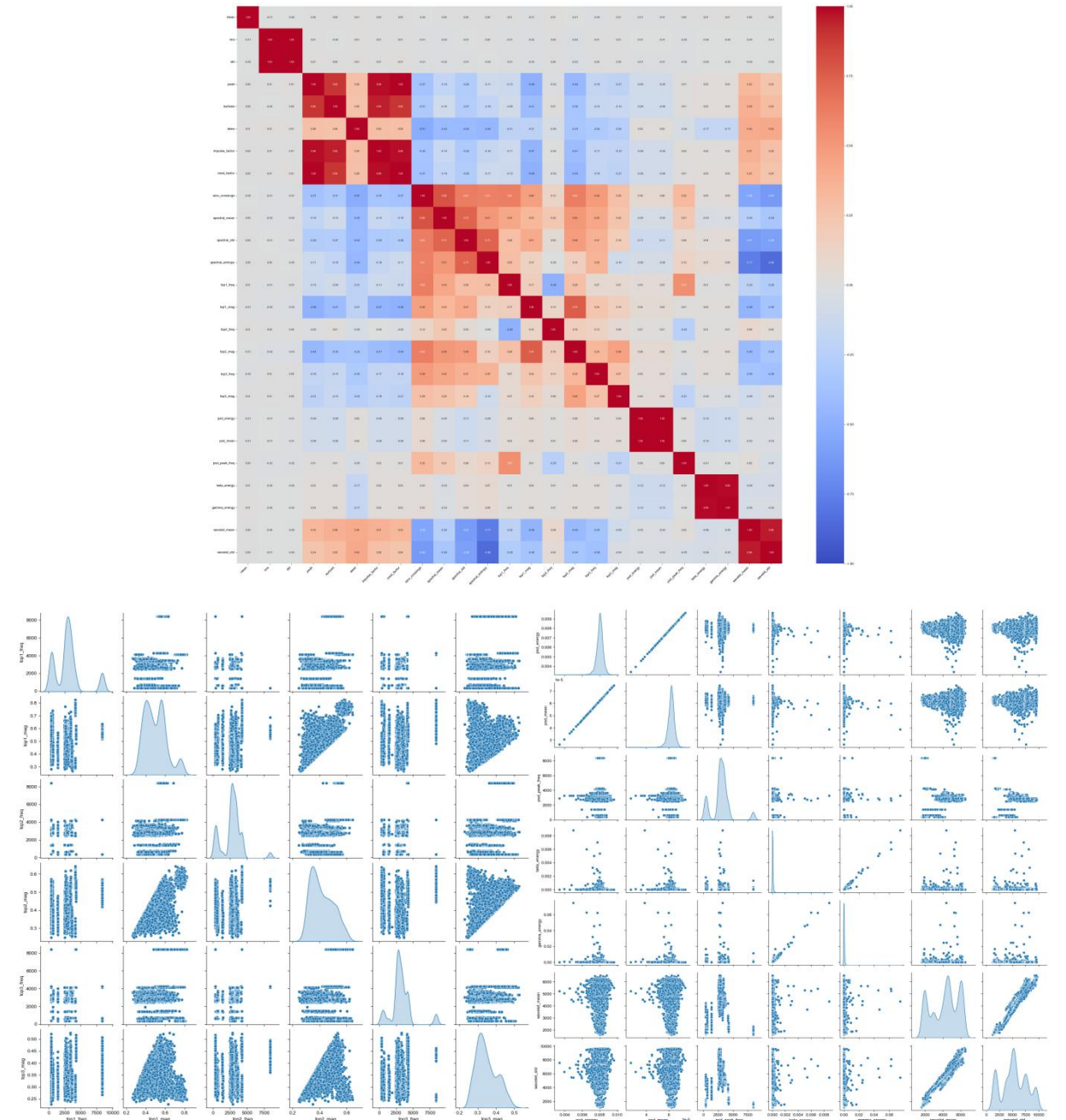


图 5-5 相关性热力图和成对关系图

2. 二维时频图像生成模型

轴承故障信号是典型的非平稳信号，其特征在时间上是局部化的，在频率上是宽带的。传统的傅里叶变换（FT）只能提供全局的频率信息而完全丢失时间定位；短时傅里叶变换（SFFT）虽引入时间窗，但其时频分辨率受限于海森堡不确定性原理，无法对高频和低频成分同时达到最优解析度。本文选择连续小波变换（CWT）作为核心转换模型，相比较而言,CWT 的优越性在于其多分辨率分析能力使其能够同时精细捕捉故障信号中的高频瞬态

冲击和低频调制现象。它通过对一个母小波函数进行伸缩和平移来分析信号，在分析高频成分时，CWT 自动使用压缩的、时间分辨率高的小波（对应小尺度 a ），能够精确捕捉故障冲击的发函发生时刻；在分析低频成分时，CWT 自动使用拉伸的、频率分辨率高的小波（对应大尺度 a ），能够精确分辨信号的周期性成分。这种自适应的“时频变焦”特性，使 CWT 成为提取非平稳信号瞬态特征的理想方法。对于一个有限能量信号 $x(t) \in L^2(\mathbb{R})$ ，其连续小波变换定义为：

$$W_x(a, b) = \int_{-\infty}^{\infty} x(t) \psi_{a,b}(t) dt = \frac{1}{\sqrt{a}} \int_{-\infty}^{\infty} x(t) \psi\left(\frac{t-b}{a}\right) dt$$

式中 $\psi_{a,b}(t)$ 是由母小波经过伸缩与平移得到的： $\psi_{a,b}(t) = \frac{1}{\sqrt{a}} \psi\left(\frac{t-b}{a}\right)$, $a, b \in \mathbb{R}, a > 0$ 。

其中 a 为尺度因子，表示与频率相关的伸缩； b 为平移因子； $\psi(t)$ 为小波基函数。当 a 取值较大时，有利于提取信号中的低频特征，取值较小时，则有利于提取信号中的高频特征。

将 CWT 应用于离散信号段 x_i' 的求解过程如下：

Step1: 母小波选择

CWT 的关键是母小波的选取，选择的母小波波形应与信号的故障特征相似。本文选择 Complex Morlet 小波（cmor1.5-1.0）作为母小波，该小波是一个由高斯函数调制的复正弦函数，其形态与故障冲击的衰减振荡特性相似，能有效匹配并提取故障特征。

Step2: 尺度参数离散化

设定一个离散化的尺度序列 $\{a_j\}$ ，该序列的选择需使其对应的伪频率范围能够覆盖轴承故障的特征频率带。

Step3: CWT 计算

对于每个标准化信号段 x_i' ，遍历所有时间和离散尺度，计算小波系数矩阵 $W_i[j, k] = W_{x_i'}(a_j, b_k)$ 。

Step4: 生成时频图像

取小波系数矩阵的模值 $|W_i|$ ，得到一个二维的时频能量分布矩阵。将该矩阵的数值通过色彩映射转换为像素值，生成一张二维时频图像，该图像能够精细刻画信号能量在时间与频率上的联合分布。

Step5: 图像数据集构建

将生成的每张图像保存为 PNG 文件，并根据其故障类别标签，分别存放在 Normal/、Ball/、InnerRace/、OuterRace/ 等子文件夹中，形成一个可以直接用于训练卷积神经网络（CNN）的图像数据集。

5.1.3 特征提取有效性验证

为证明上述模型提取的特征确实反映了故障的物理本质，本文建立了一个验证流程。当轴承特定元件（内圈、外圈、滚动体）出现局部损伤时，滚动体每次碾过该损伤点都会产生一次冲击脉冲，这些脉冲以特定的频率周期性出现，即故障特征频率。其理论计算公式如表 5-1 所示：

表 5-1 轴承故障特征频率

名称	特征频率
外圈故障特征频率（BPFO）	$f_r \cdot \frac{N_d}{2} \cdot \left(1 - \frac{d}{D}\right)$

内圈故障特征频率（BPFI）

$$f_r \cdot \frac{N_d}{2} \cdot \left(1 + \frac{d}{D}\right)$$

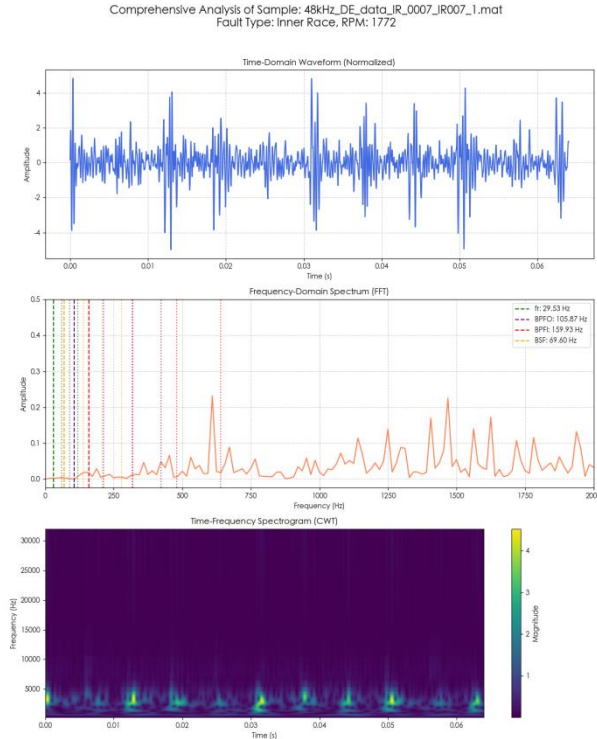
滚动体故障特征频率（BSF）

$$f_r \cdot \frac{D}{d} \left[1 - \left(\frac{d}{D}\right)^2\right]$$

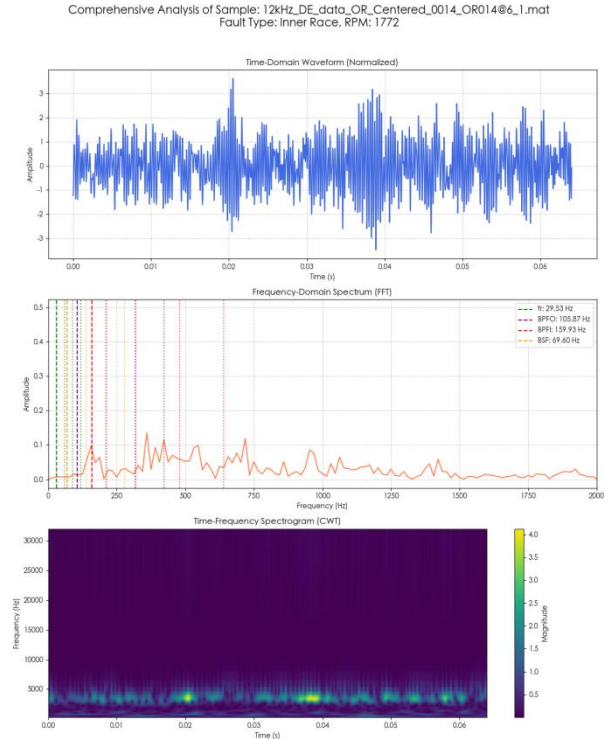
其中， N_d 为滚动体数量， f_r 为轴承转频， d 为滚动体直径， D 为轴承节径（通常可由 $\frac{\text{内径}+\text{外径}}{2}$ 估算）。

在 IR、OR、B、N 四种工作状态下分别选取一个典型的故障样本（此处选择 IR007_1.mat、OR014@6_1.mat、B014_1.mat、N_0.mat）进行求解与验证：

- 参数提取：从文件名或实验记录中获取该样本的电机转速，换算为轴转速 f_r 。
- 理论计算：将 f_r 代入对应公式，计算出理论的故障特征频率及其二倍、三倍谐波频率。
- 特征提取与可视化：分别对四个样本信号进行可视化生成综合分析图，包含时域波形、快速傅里叶变换（FFT）频谱图和 CWT 时频图三个子图。时域波形图清晰地展示了故障信号中存在的周期性冲击脉冲；FFT 频谱图展示了信号在频域的能量分布，在此图上用不同颜色的虚线精确标注根据该样本转速计算出的故障特征频率及其 2 倍、3 倍谐波；CWT 时频图直观地显示了冲击能量在时间和频率上的二维分布。
- 对比验证：可以观察到 FFT 频谱图上的能量峰值与理论标注线高度吻合，可以直观地验证数据处理和特征提取方法能够有效捕捉到与故障机理相符的关键信息。同时，CWT 时频图存在与故障特征频率周期相对应的周期性能量冲击带，进一步确认了故障特征的存在。



a.IR007_1



b.OR014@6_1

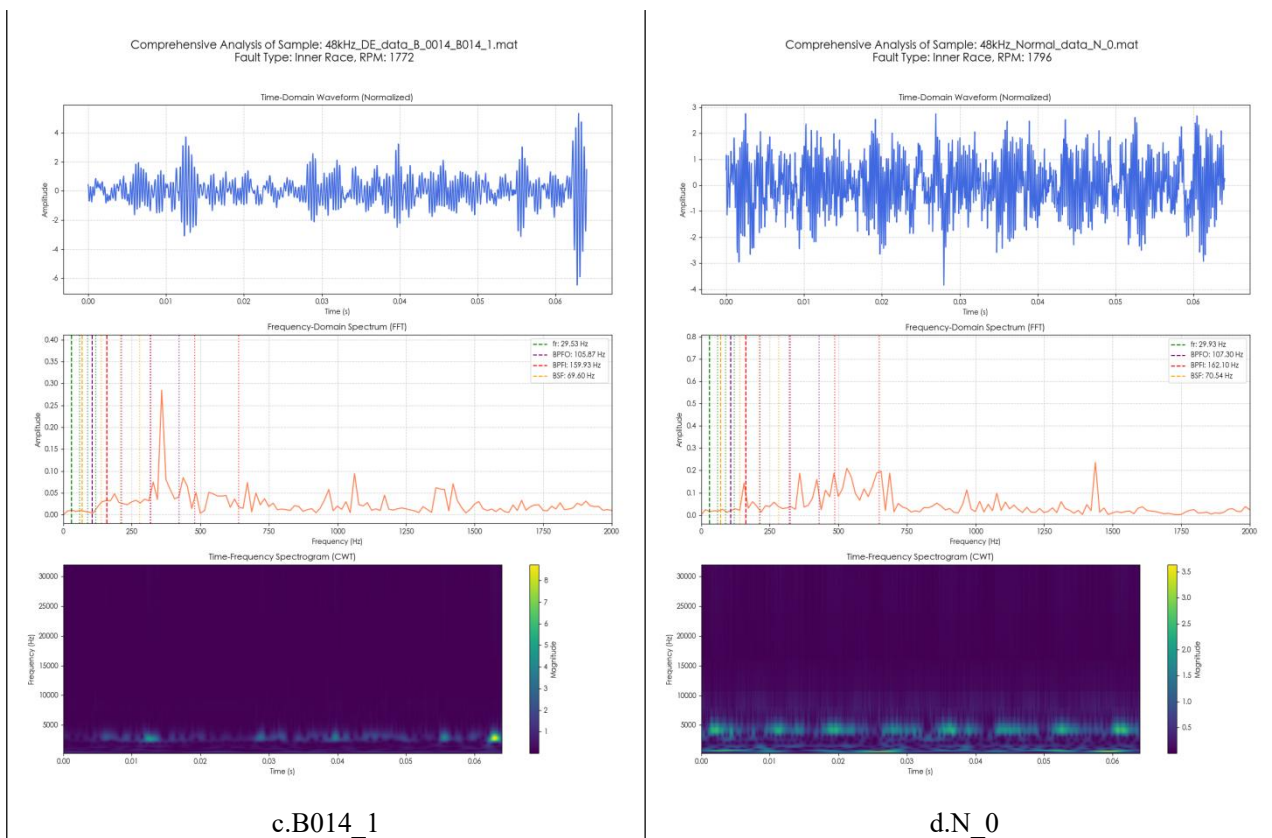


图 5-6 综合分析图

5.2 问题二的模型建立与求解

本问题的核心任务是构建并评估一个高性能的源域故障诊断模型，对此，本文针对问题一提取出的一维特征向量和二维时频图像设计并实施了两个并行的实验流程，以确定最优的源域故障诊断模型。问题二的研究思路如图 5-7 所示。

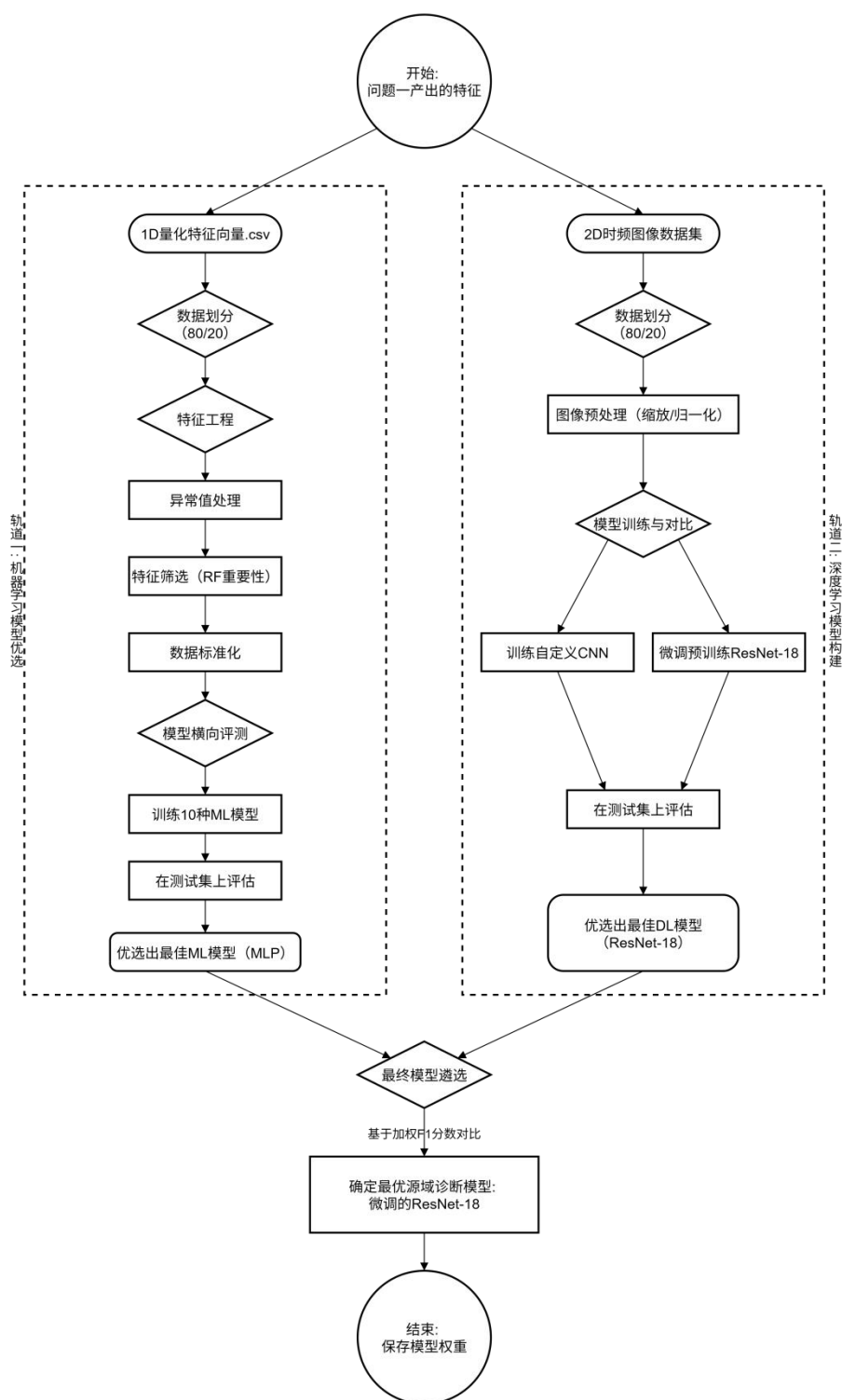


图 5-7 问题二流程图

5.2.1 基于一维量化特征的机器学习模型

1. 典型机器学习分类模型的选择

(1) 多层感知机 (Multilayer Perceptron, MLP)

多层感知机 (MLP)，也是人工神经网络 (Artificial Neural Network, ANN)，是一种全连接 (全连接: MLP 由多个神经元按照层次结构组成，每个神经元都与上一层的所有神经元相连) 的前馈神经网络模型。MLP 由至少三个层次的神经元组成: 一个输入层、一

个或多个隐藏层以及一个输出层。其算法流程见图 5-8。

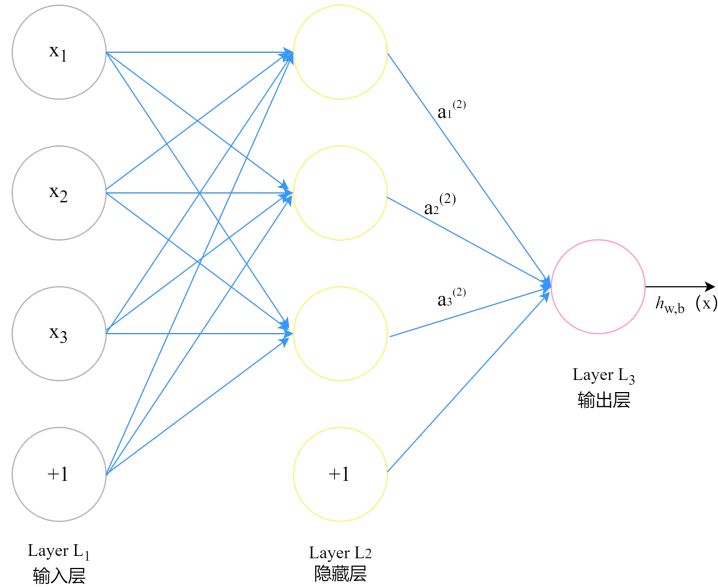


图 5-8 多层感知机算法流程图

数据从输入层进入，每个神经元接收来自上一层神经元的加权输入，加上一个偏置项，然后通过一个非线性激活函数处理后，将输出传递给下一层。这个过程可以数学化地表示为 $\mathbf{a}^{(l)} = g(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$ ，其中， $\mathbf{a}^{(l)}$ 是第 l 层的激活输出向量， $\mathbf{W}^{(l)}$ 和 $\mathbf{b}^{(l)}$ 分别是第 l 层的权重矩阵和偏置向量， $g(\cdot)$ 是激活函数。通过堆叠多个隐藏层和使用非线性激活函数，MLP 能够学习输入特征与输出标签之间极其复杂的非线性关系。在本题中，将经过筛选和标准化的 28 个一维特征向量作为 MLP 的输入，模型的隐藏层负责将这些特征组合、变换，学习高阶的交互信息。输出层则包含 4 个神经元，并通过 Softmax 激活函数输出样本属于四种故障类别的概率。

(2) 逻辑回归 (Logistic Regression, LR)

逻辑回归是一种广义线性模型，常用于解决二分类问题，并可通过“一对多”或“多项式”策略扩展至多分类任务。其核心思想是，将输入特征的线性组合通过一个非线性函数（Sigmoid 或 Softmax）映射到 $(0, 1)$ 区间，从而得到样本属于某一类别的概率。对于 K 分类问题，模型为每个类别 $k \in \{1, \dots, K\}$ 计算一个分数 $z_k = \mathbf{w}_k^T \mathbf{x} + b_k$ 。然后，通过 softmax 函数将这些分数转换为概率： $P(y = j|\mathbf{x}) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$ ，其中 $P(y = j|\mathbf{x})$ 是样本 $\mathbf{x}$ 属于类别 j 的预测概率， $\mathbf{w}_k$ 和 b_k 是类别 k 对应的权重向量和偏置项。模型通过最小化交叉熵损失函数来学习这些参数。

(3) 支持向量机 (Support Vector Classifier, SVC)

支持向量机的基本目标是在特征空间中寻找一个能将不同类别样本最大化分开的“最优超平面”。这个超平面由距离它最近的几个训练样本点（即支持向量）唯一确定，使得它与这些支持向量之间的间隔最大化。对于线性不可分的数据，SVC 通过核技巧 (Kernel Trick) 将数据映射到更高维的特征空间，在新的空间中实现线性可分。SVC 的优化目标是 minimize 超平面的法向量 $\mathbf{w}$ 的 L2 范数，同时满足分类约束。其原始优化问题可表示为：

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t. } y_i(\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1, \forall i. \quad \text{其中, } \phi(\mathbf{x}) \text{ 是将输入 } \mathbf{x} \text{ 映射到高维空间的函数。}$$

在实践中，无需显式定义 ϕ ，而是通过核函数 $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ 来直接计算高维空间的内积。

空间中的内积，常用核函数包括线性核、多项式核及高斯径向基函数核。SVC 在处理高维数据和非线性问题上表现出色，且对过拟合不敏感，选择它是为了评估基于“最大间隔”思想的分类器在本任务上的有效性。

(4) 决策树 (Decision Tree, DT)

决策树是一种树形结构的非参数监督学习模型。它通过从根节点开始，递归地选择最优特征对数据集进行划分，每个内部节点代表一个特征测试，每个分支代表一个测试输出，每个叶节点代表一个类别标签。决策的目标是使得划分后各子集的“纯度”最高。特征选择的依据是信息增益、信息增益率或基尼不纯度 (Gini Impurity)，基尼不纯度衡量从数据集中随机抽取两个样本，其类别标签不一致的概率。对于一个节点，其基尼不纯度计算公式为： $Gini(D) = 1 - \sum_{k=1}^K p_k^2$ 。其中， p_k 是该节点中属于类别 k 的样本比例。决策树在分裂时会选择使得基尼不纯度下降最大的特征和阈值。之所以选择 DT 作为候选模型之一，是因为决策树模型具有极高的可解释性，并且是构建更强大的集成模型（如随机森林和梯度提升树）的基础单元。

(5) 随机森林 (Random Forest, RF)

随机森林是一种基于 Bagging (Bootstrap Aggregating) 策略的集成学习模型，它由多个决策树构成。其核心思想是通过构建并结合多棵“弱而不同”的决策树来形成一个“强而稳”的模型。其随机性主要体现在两个方面：行抽样，即每棵树使用自助采样法得到不同的训练子集，和列抽样（每棵树在分裂节点时仅从一个随机的特征子集中选择最优特征）。最终的分类结果由所有树的投票决定。对于一个分类任务，随机森林的最终预测 $\hat{y}_{RF}$ 是所有 B 棵决策树预测结果的众数： $\hat{y}_{RF}(\mathbf{x}) = \operatorname{argmax}_k \sum_{b=1}^B I(\hat{y}_b(\mathbf{x}) = k)$ 。其中， $\hat{y}_b(\mathbf{x})$ 是第 b 棵树对样本 $\mathbf{x}$ 的预测， $I(\cdot)$ 是指示函数。随机森林通常具有很高的预测精度，对噪声和异常值有很好的鲁棒性，并且能有效防止过拟合，因此将其选为传统机器学习分类模型的候选模型之一。

(6) 梯度提升机 (Gradient Boosting Machine, GBM)

梯度提升机是一种典型的 Boosting 算法，其通过为初始模型的伪残差建立基学习器的方式实现在梯度方向上减少残差，并且基于权重将不同的基学习器进行线性组合，削弱表现不好的基学习器，增强表现较好的基学习器，反复迭代上述过程直至找到使损失函数期望达到最小的模型。GBM 训练步骤为：

- 1) 定义训练集 $T = \{(x_1, y_1), \dots, (x_N, y_N)\}$ 的损失函数为 $L(y, f(x))$ 。
- 2) 初始化弱学习器。 $f_0(x) = \operatorname{argmin}_\beta \sum_{i=1}^N L(y_i, r)$ ，式中： $L(y_i, r)$ 为目标值 y_i 的损失函数； r 为学习率。
- 3) 对 $m=1, 2, \dots, M$ (M 为基学习器的个数)，循环执行①、②、③、④四个步骤。
 - ① 计算负梯度 y'_i 。 $y'_i = -\frac{\partial L(y_i, f_{m-1}(x_i))}{\partial f_{m-1}(x_i)}$ $i = 1, \dots, N$
 - ② 使用一个基学习器 $h_m(x)$ 拟合 y'_i 。 $\alpha_m = \operatorname{argmin}_\alpha \sum_{i=1}^N (y'_i - \beta h(x_i, \alpha_m))^2$
 - ③ 利用线性寻优方式确定学习率 r_m 。 $r_m = \operatorname{argmin}_r \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + r h(x_i, \alpha_m))$
 - ④ 更新模型。 $f_m(x) = f_{m-1}(x) + r_m h(x_1, \alpha_m)$
- 4) 输出强学习器，即 $f_M(x) = \sum_{m=1}^M f_m(x)$

(7) XGBoost (Extreme Gradient Boosting)

XGBoost 是梯度提升算法的一种高效、灵活且可扩展的实现，属于 Boosting 集成学习的范畴。与随机森林并行构建树不同，梯度提升是串行地构建决策树，每一棵新树都是为了拟合和修正前面所有树的预测结果与真实值之间的残差（或梯度）。XGBoost 在传统梯度提升的基础上进行了多项优化，包括对损失函数进行二阶泰勒展开，并加入了正则化项来控制模型复杂度。其优化目标函数在第 t 轮迭代时可以表示为：

$$\text{Obj}^{(t)} = \sum_{i=1}^n [l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i))] + \Omega(f_t)$$

其中， l 是可微的损失函数， $\hat{y}_i^{(t-1)}$ 是前 $t-1$ 棵树的累积预测， f_t 是当前正在学习的第 t 棵树， $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2$ 是正则化项，用以惩罚树的复杂性（ T 为叶节点数量， $\mathbf{w}$ 为叶节点权重）。XGBoost 在各类结构化数据竞赛和实际应用中长期处于领先地位，以其卓越的性能和高效率著称，将其纳入评测是为了探索 Boosting 算法家族在本任务中的性能上限。

(8) 梯度提升决策树 (Gradient Boosting Decision Tree, GBDT)

梯度提升树 (GBDT) 是一种迭代的决策树算法，由多棵决策树组成，综合所有树的预测作为最终预测。该算法的核心在于每棵树学习之前所有树的残差；而为了消除残差，模型在残差减少的梯度方向上建立一个新的模型，即采用梯度下降法来求解最优模型。因此在 GBDT 中，每个新树的建立是为了使得之前模型的残差沿梯度方向减少。此外，决策树进行分支时以最小化平方误差为标准，对每一个特征，每一个阈值进行穷举以寻求最优的分割点。对于训练集 $\text{Train} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ ，构建梯度提升树 $f_M(\mathbf{x})$ 的算法流程如下：

- ① 初始化 $f_0(\mathbf{x}) = 0$
- ② 对于 $m = 1, 2, \dots, M$ ，计算其预测残差：
$$r_{mi} = y_i - f_{m-1}(\mathbf{x}_i) \quad i = 1, 2, \dots, N$$
- ③ 对于 N 个残差学习得到一个回归树 $T(\mathbf{x}; \phi_m)$
- ④ 更新 $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \text{Train}(\mathbf{x}; \phi_m)$
- ⑤ 最终得到梯度提升树： $f_M(\mathbf{x}) = \sum_{m=1}^M T(\mathbf{x}; \phi_m)$

GBDT 是一种加法模型，在较少的调参时间情况下能够获得相对较高的预测准确率。同时由于使用的损失函数相对稳健，GBDT 算法对异常值的鲁棒性非常强，算法的主要参数包含：① 每个弱学习器的权重缩减系数，即学习率 (δ)；② 弱学习器的最大迭代次数 (N)；③ 决策树最大深度 (maxdep)。

(9) K-近邻算法 (K-Nearest Neighbors, KNN)

K-近邻算法是一种非参数、基于实例的“懒惰学习”算法。其核心假设是，相似的样本在特征空间中彼此邻近。对于一个待分类的新样本，KNN 会计算它与训练集中所有样本的距离，并找出距离最近的 K 个邻居。然后，根据这 K 个邻居的类别进行多数票表决，将得票最多的类别作为新样本的预测类别。给定一个测试样本 $\mathbf{x}$ ，其预测类别 $\hat{y}$ 由其 K 个最近邻居的类别决定。设 $N_K(\mathbf{x})$ 为 $\mathbf{x}$ 的 K 个最近邻居的集合，则预测公式为： $\hat{y} = \text{argmax} \sum_{\mathbf{x}_i \in N_K(\mathbf{x})} I(y_i = c)$ ，其中， C 是所有类别的集合， y_i 是邻居样本 $\mathbf{x}_i$ 的真实类别， $I(\cdot)$

为指示函数（当条件成立时为 1，否则为 0）。样本间的距离通常使用欧氏距离来度量：

$$\sqrt{\sum_{j=1}^d (x_{aj} - x_{bj})^2}。$$

(10) AdaBoost (Adaptive Boosting)

AdaBoost 是 Boosting 集成学习家族中最具代表性的算法之一。它通过串行方式训练一系列的弱分类器（通常是决策树桩，即单层决策树），并迭代地调整样本权重。在每一轮中，AdaBoost 会提高被前一个弱分类器错误分类的样本的权重，同时降低被正确分类的样本的权重。这样，后续的弱分类器就会更加关注那些“难啃的”样本。最终的强分类器是所有弱分类器的加权组合，性能更好的弱分类器会获得更高的权重。最终的强分类器 $G(\mathbf{X})$

时 M 个弱分类器 $G_m(\mathbf{x})$ 的加权和： $G(\mathbf{x}) = \text{sign}(\sum_{m=1}^M \alpha_m G_m(\mathbf{x}))$ ，其中第 m 个弱分类器的

权重 α_m 由其在加权样本上的错误率 ϵ_m 决定： $\alpha_m = \frac{1}{2} \ln \left(\frac{1-\epsilon_m}{\epsilon_m} \right)$ 。作为 Boosting 算法的开山之作，AdaBoost 的引入可以评估一种更侧重于样本权重自适应调整的集成策略对本任务的有效性，并与后续更复杂的梯度提升算法形成对比。

2. 基于一维量化特征的机器学习模型构建与优选

针对问题一提取的 28 个一维特征向量，选取 10 种不同的经典算法构建机器学习模型，对分类结果进行评估并从中选择最优模型。首先明确该流程中的数据和变量定义，如表 5-2 所示。

表 5-2 数据与变量定义

数据/变量	描述
输入数据	问题一生成的 one_dimensional_features_balanced.csv 文件
特征变量	令样本集为 $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ ，其中 $N=4800$ 为总样本数，每个样本的特征向量 $\mathbf{x}_i \in \mathbb{R}^{28}$ 是一个包含 28 个时域、频域、小波域等指标的实数向量
目标变量	$y_i \in \{0,1,2,3\}$ 为类别标签，分别对应正常、滚动体故障、内圈故障和外圈故障。

Step1: 数据准备与特征工程

首先，将数据集 D 按照 80% : 20% 的比例进行分层随机抽样，划分为训练集 D_{train} 和测试集 D_{test} 。分层抽样确保了训练集和测试集中各故障类别的比例与原始数据集保持一致。

对训练集 D_{train} 执行如下特征工程流程：

- 异常值处理：为增强模型鲁棒性，对数值型特征采用 IQR (Interquartile Range) 方法进行处理。定义特征 j 的上下界为 $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$ ，将超出此范围的极端值约束在边界上。
- 特征筛选：为降低模型复杂度并剔除冗余信息，利用随机森林分类器对全部 28 个特征进行训练，并基于基尼不纯度下降量 (Gini Impurity Decrease) 计算各特征的重要性。选取重要性排名前 15 的特征构成最终的特征子集 $\mathbf{x}_i' \in \mathbb{R}^{15}$ 。
- 数据标准化：对筛选出的 15 维特征子集进行 Z-score 标准化，使其服从均值为 0，

标准差为 1 的分布。标准化公式为： $\mathbf{x}_{std} = \frac{\mathbf{x}' - \mu_{train}}{\sigma_{train}}$ ，其中 μ_{train} 和 σ_{train} 是在训练集上

计算得到的均值和标准差向量。这些参数将被保存，并同样应用于测试集 D_{test} 。

Step2: 模型横向评测与求解

在经过工程化处理的训练集上，系统性地训练十种主流机器学习分类器，包括逻辑回归、决策树、随机森林、支持向量机 (SVC)、K 近邻 (KNN)、梯度提升机 (Gradient Boosting)、AdaBoost、XGBoost、多层感知机 (MLP) 以及 HistGradientBoosting。所有模型均采用网

格搜索与交叉验证优化关键超参数。

Step3: 性能评估与模型确定

让所有训练好的模型在独立的测试集 D_{test} 上进行性能评估，计算每个模型的加权 F1 分数，其定义为： $F1_{weighted} = \sum_{c=1}^C w_c \cdot F1_c = \sum_{c=1}^C w_c \cdot \frac{2P_c R_c}{P_c + R_c}$ ，其中 C 是类别数， w_c 是类别 c 的样本占比， P_c 和 R_c 分别是类别 c 的精确率和召回率。

5.2.2 基于二维时频图像的深度学习模型

(1) 卷积神经网络 (Convolutional Neural Networks, CNN)

卷积神经网络 (CNN) 是一类包含卷积计算且具有深度结构的前馈神经网络 (Feedforward Neural Networks)，是深度学习的代表算法之一。CNN 是为处理具有网格结构数据（如图像）而专门设计的深度学习模型。其核心优势在于通过局部感受野、参数共享和空间下采样来高效地学习空间层级特征。CNN 的基本结构由输入层、卷积层、池化层、全连接层及输出层构成。卷积层是 CNN 的核心，它使用一组可学习的滤波器在输入图像上滑动，每个滤波器专门负责检测一种特定的局部特征。通过卷积操作，生成代表这些特征存在的特征图。池化层通常接在卷积层之后，用于对特征图进行下采样。它能逐步减小数据的空间维度，从而减少计算量和参数数量，同时赋予模型一定程度的平移不变性。卷积层和池化层一般会取若干个，采用卷积层和池化层交替设置。卷积层中输出特征面的每个神经元与其输入进行局部连接，并通过对应的连接权值与局部输入进行加权求和再加上偏置值，得到该神经元输入值，该过程等同于卷积过程。在经过多层卷积和池化后，最终的特征图被展平为一维向量，并输入到全连接层中。全连接层的作用与 MLP 类似，它整合所有局部特征，进行最终的分类决策。

本文构建了一个包含三个卷积—池化模块和两个全连接层的轻量级 CNN 作为性能基准，模型架构如图 5-9 所示。该自定义 CNN 模型以 3 通道 128×128 的输入图像为起点，通过 3 个依次包含 Conv2d、BatchNorm2d、ReLU 和 MaxPool2d 的卷积块提取特征，经自适应平均池化与展平后，由含 ReLU、Dropout 的线性层分类器输出 4 类结果。通过在这样一个相对简单的架构上进行训练，可以验证将故障诊断问题转化为图像分类问题的可行性，并为评估更复杂模型所带来的性能提升提供一个明确的参照点。

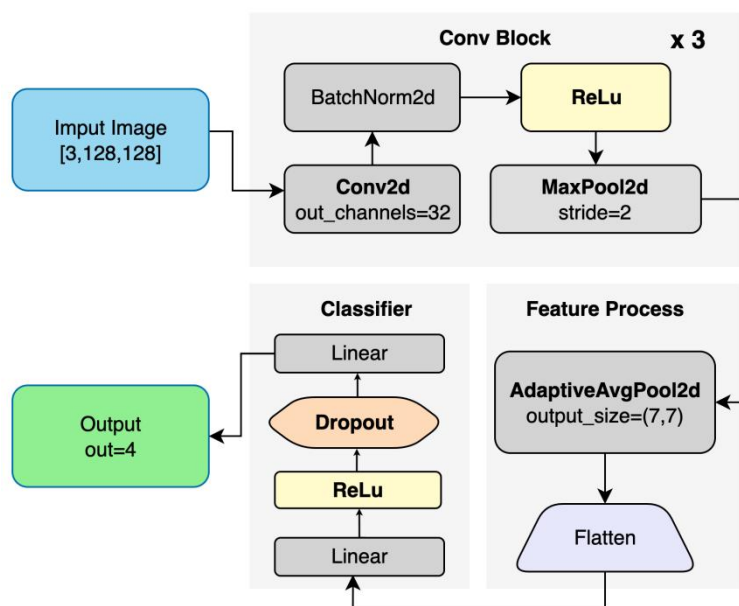


图 5-9 自定义 CNN 模型架构图

(2) 残差网络模型 ResNet-18

在 CNN 的应用过程中，研究人员发现，随着网络增加到一定层数，模型的分类效果会变差。其本质原因主要有两点：深层网络反向传播时，梯度经过多轮矩阵乘法易出现梯度消失或梯度爆炸，导致浅层参数难以更新；同时，过度非线性变换（如 ReLU 激活）使网络难以拟合恒等映射（即输入输出一致的简单映射），而浅层网络的最优特征无法传递至深层。为解决训练极深网络时的梯度消失和模型退化问题，残差网络这一模型被提出。ResNet 的核心思想是通过残差连接（shortcut connection）将输入信号直接传递到后面的层，使得网络可以学习到残差而不是全局特征。ResNet 的结构如图 5-10 所示。

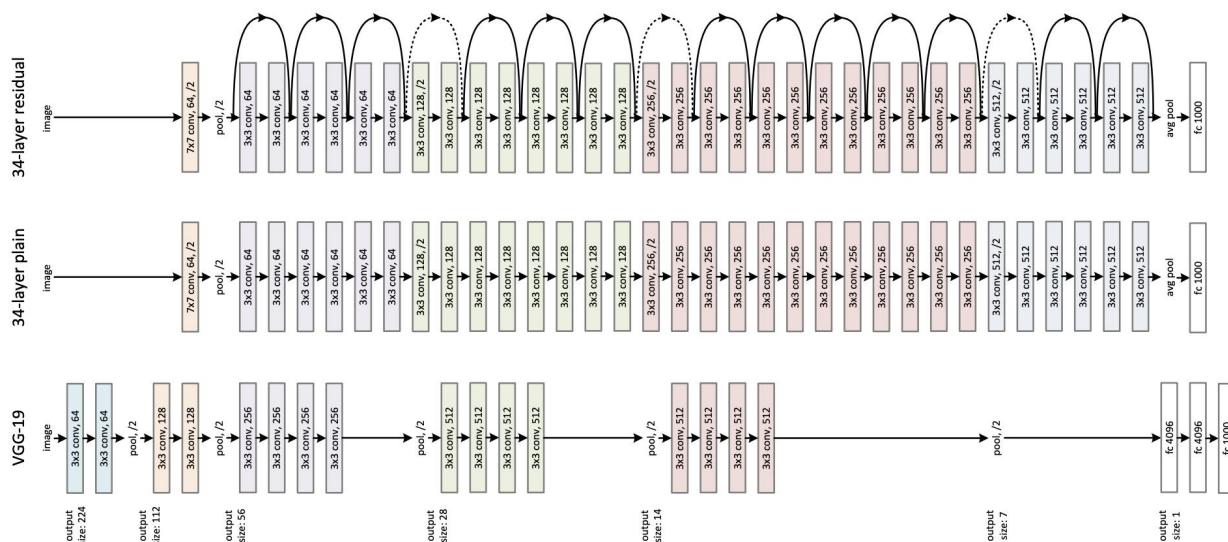


图 5-10 ResNet 结构图

ResNet 的整体架构以残差块（Residual Block）为基本单元，不同深度的 ResNet（如 ResNet-18/34/50/101/152）仅在残差块数量与卷积通道数上存在差异，核心设计逻辑一致。ResNet 残差块结构如图 5-11 所示，具体来说，ResNet 残差块使用了一种名为“残差连接”的连接方式，将输入 x 与经过堆叠的权重层得到的 $F(x)$ 进行跨层连接，得到输出 $H(x)=F(x)+x$ 。此时 $F(x)=H(x)-x$ 即为残差，公式为 $F = W_2\sigma(W_1x)$ ，式中 F 为残差函数， σ 为 ReLU 非线性激活函数， W_1 和 W_2 为权重。这种设计使得网络优化变得极为容易。在极端情况下，若理想映射为恒等映射（即 $H(x)=x$ ），则只需将 $F(x)$ 的权重学习为零即可，而无需通过复杂的非线性层去拟合一个恒等变换，极大降低了学习难度。这使得信息和梯度可以在网络中更顺畅地流动，从而能够成功训练比以往深得多的网络。

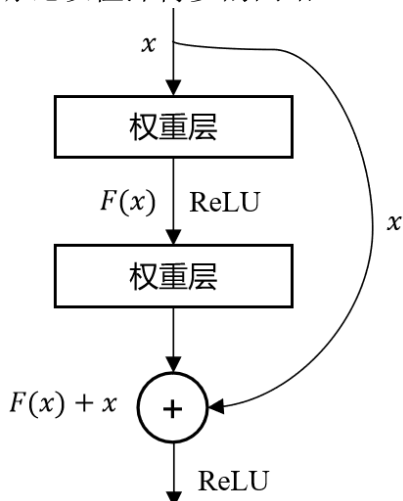


图 5-11 自定义 CNN 模型架构图

本文采用的是其 18 层的版本——ResNet-18。ResNet-18 是工业界最常用的轻量级版本，“18”指网络中含权重的层（卷积层+全连接层）总数为 18 层，不含池化层与 BatchNorm 层。其整体结构分为 6 个模块，输入为 $3 \times 224 \times 224$ 的 RGB 图像，具体流程如下：

表 5-3 ResNet-18 各模块信息

模块	层结构	输出特征图尺寸	通道数	权重层数量
初始层 (输入层)	7×7 Conv (stride=2, padding=3) → BatchNorm2d → ReLU → 3×3 MaxPool (stride=2)	56×56	64	1 (仅卷积层)
Layer1	2 个恒等映射残差块(每个含 2 个 3×3Conv)	56×56	64	4
Layer2	1 个维度调整残差块 (1×1 Conv+2 个 3×3 Conv) +1 个恒等映射残差块	28×28	128	4
Layer3	1 个维度调整残差块+1 个恒等映射残差块	14×14	256	4
Layer4	1 个维度调整残差块+1 个恒等映射残差块	7×7	512	4
分类层 (输出层)	AdaptiveAvgPool2d (输出 1×1) → Flatten → 全连接层 (输出类别数)	1×1	512	1 (全连接层)

(2) 基于二维时频图像的深度学习模型构建与深化

针对问题一生成的二维时频图像，构建 CNN 基准模型和 ResNet-18 深化模型。首先明确该流程中的数据和变量定义，如表 5-4 所示。

表 5-4 数据与变量定义

数据/变量	描述
输入数据	问题一生成的二维时频图像数据集，每个类别 1200 张
特征变量	输入数据为图像 $I \in \mathbb{R}^{H \times W \times C}$ ，其中 H 、 W 为图像高宽， C 为通道数
目标变量	与流程一相同， $y_i \in \{0,1,2,3\}$

Step1: 数据集构建与预处理

同样按 80% : 20%比例对图像数据集进行分层抽样，构建训练集和测试集。对所有输入图像应用以下变换：

- 尺寸归一化：将图像尺寸统一调整为 224×224 像素，以满足预训练 ResNet 模型的输入要求。
- 张量转换与归一化：将图像转换为 PyTorch 张量，并根据 ImageNet 数据集的均值 [0.485, 0.456, 0.406]和标准差[0.229, 0.224, 0.225]进行归一化，确保输入数据分布与预训练模型所见数据的一致性。

Step2: 基准模型与深化模型建立

- 基准模型(Custom CNN): 构建一个包含三个卷积层和两个全连接层的自定义 CNN, 作为性能基准。
- 深化模型(Fine-tuned ResNet-18): 引入在 ImageNet 数据集上预训练好的 ResNet-18 模型, 并对其实施微调策略, 冻结模型大部分卷积层的权重, 因为它们学习到的通用视觉特征是可迁移的。然后替换原有的 1000 类输出的全连接层为一个新的、随机初始化的全连接层, 其输出维度为 4 (对应四个故障类别)。

Step3: 模型训练与求解

两个模型均采用交叉熵损失函数(Cross-Entropy Loss)作为优化目标, 其公式为: $L_{CE} = -\sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$, 其中 $y_{i,c}$ 是样本 i 属于类别 c 的真实标签 (one-hot 编码), $\hat{y}_{i,c}$ 是模型预测概率。使用 Adam 优化器对模型参数进行端到端的训练, 在训练过程中, 监控训练集和验证集上的损失与准确率变化, 以判断模型是否收敛且未发生严重过拟合。

Step4: 性能评估

在独立的图像测试集上评估两个模型, 得到分类结果各类别的精确率、召回率和 F1-score 等指标, 通过对比分析决策出较优模型。

5.2.3 结果分析

(1) 基于一维特征向量的机器学习模型

首先利用随机森林分类器计算并排序各特征的重要性, 结果如图 5-12 所示。选取出重要性排名前 15 的特征构成最终的特征子集, 包括小波能量标准差、过零率、小波能量均值、第三主频分量频率、功率谱峰值对应频率、频谱加权平均频率、第一主频分量频率、频谱熵、频谱标准差、峭度、第二主频分量频率、偏度、第二主频分量幅值、脉冲因子、信号绝对最大值。

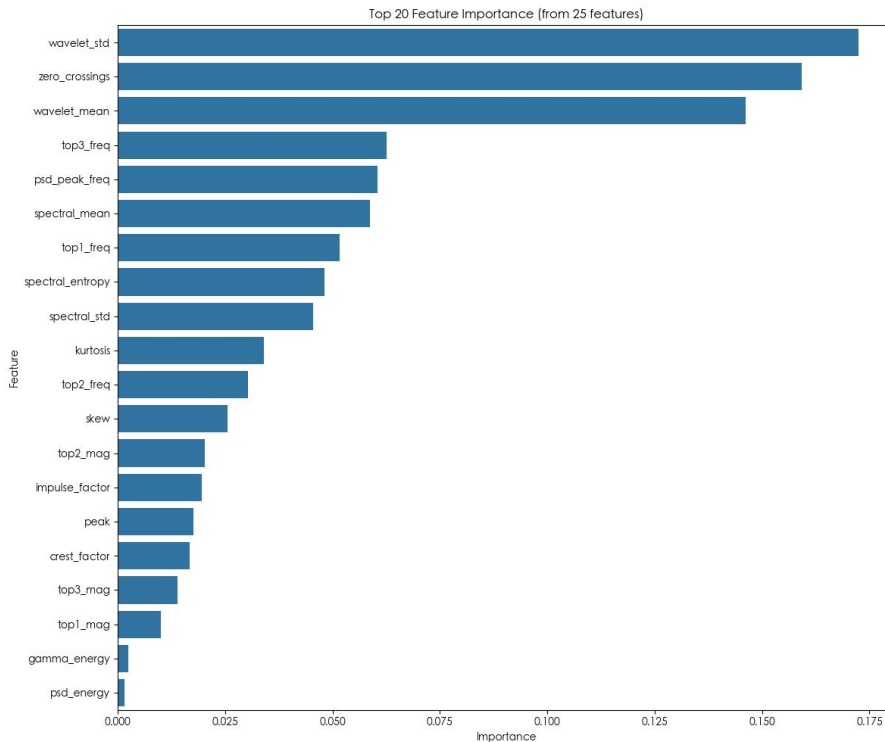


图 5-12 一维特征重要性排序

基于一维特征向量的机器学习各模型的性能评估结果如表 5-5 所示, 结果表明多层感知机 (MLP) 分类器在此任务中表现最佳, 其在测试集上的加权 F1 分数达到了 99.17%。

因此，MLP 被确定为此流程下的最优模型。使用混淆矩阵对 MLP 的预测数据进行有效性验证，见图 5-13。

表 5-5 轨道一候选模型评估结果

模型	准确率（Accuracy）	加权 F1 分数 （Weighted F1-Score）	备注
MLP	99.17%	99.17%	轨道一最优模型
HistGradientBoosting	99.06%	99.06%	性能优异
随机森林	99.06%	99.06%	性能优异
XGBoost	98.75%	98.75%	性能稳健
决策树	98.54%	98.54%	性能良好
梯度提升机	98.44%	98.44%	性能良好
SVC	98.33%	98.33%	性能良好
KNN	97.60%	97.60%	性能中等
逻辑回归	96.15%	96.15%	性能中等
AdaBoost	50.00%	37.50%	性能不佳

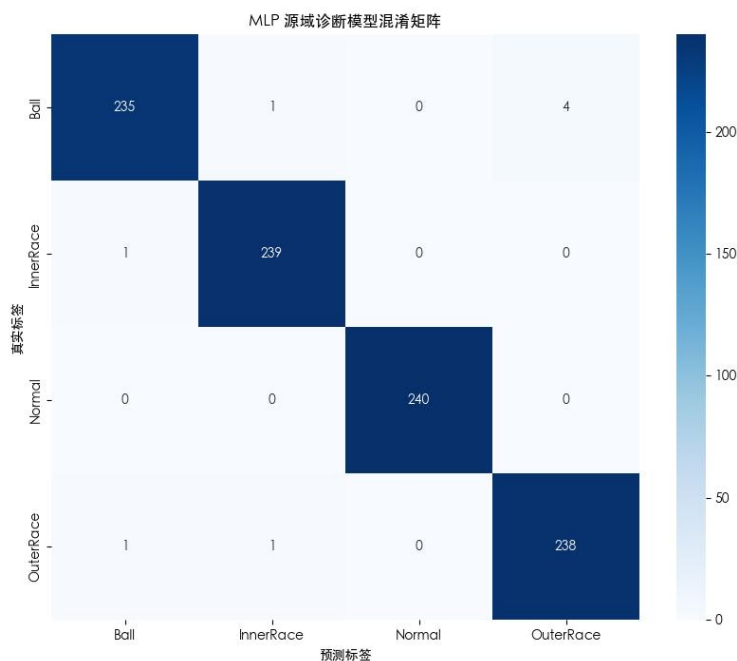


图 5-13 MLP 源域诊断模型混淆矩阵

从基于二维时频图像的深度学习模型分类结果（见表 5-6 和表 5-7）来看，自定义 CNN 模型在源域数据上表现优异，各类别（Ball、InnerRace、Normal、OuterRace）的精确率、召回率和 F1-score 均接近 1，整体准确率达 0.9990；而微调的 ResNet-18 模型在源域数据上的各类评估指标均等于 1，表现更为出色。这表明两种模型在源域轴承故障分类任务中都具有很强的性能，但微调的 ResNe-18 模型在源域数据上的分类效果略胜一筹，可能得益于其预训练的特征提取能力和微调后的适配性。

表 5-6 自定义 CNN 分类结果评估

	precision	recall	f1-score	support
Ball	0.9959	1	0.9979	240
InnerRace	1	1	1	240

Normal	1	1	1	240
OuterRace	1	0.9958	0.9979	240
accuracy	0.9990	0.9990	0.9990	0.9990
macro avg	0.9990	0.9990	0.9990	960
weighted avg	0.9990	0.9990	0.9990	960

表 5-7 微调的 ResNet-18 分类结果评估

	precision	recall	f1-score	support
Ball	1	1	1	240
InnerRace	1	1	1	240
Normal	1	1	1	240
OuterRace	1	1	1	240
accuracy	1	1	1	1
macro avg	1	1	1	960
weighted avg	1	1	1	960

使用混淆矩阵对二者的预测数据进行有效性验证，见图 5-14，自定义 CNN 出现 1 个错误预测的样本，而经过微调的 ResNet-18 模型全部预测正确。结果显示，经过微调的 ResNet-18 模型性能远超自定义 CNN，其在所有故障类别上的识别准确率和 F1 分数均接近完美，综合加权 F1 分数达到了 99.9%以上。

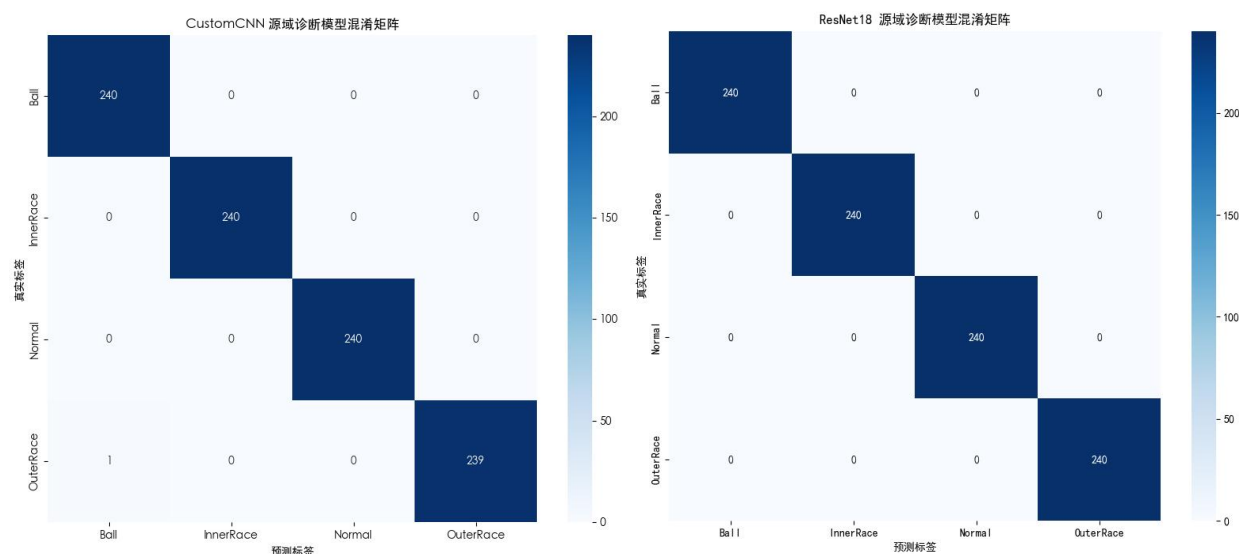


图 5-14 Custom CNN 源域诊断模型混淆矩阵和 ResNet-18 源域诊断模型混淆矩阵

分析自定义 CNN 和微调的 ResNet-18 两个模型的训练和预测损失曲线（见图 5-15），可以发现，自定义 CNN 的训练损失（蓝色）从初始近 0.5 快速下降，在约 5 个 epoch 后趋于稳定并维持在极低水平；测试损失（橙色）前期有小幅波动，后逐渐与训练损失趋近，整体呈现同步下降趋势，最终两者都稳定在接近 0 的区间。这表明 Custom CNN 模型在训练过程中收敛效果较好，且训练集与测试集的损失差异小，模型泛化能力较强，没有出现明显的过拟合现象。ResNet-18 的训练损失（蓝色）初始约 0.11，快速下降后保持平稳；测试损失（橙色）在训练过程中出现了明显的尖峰波动，但后续也逐渐下降并趋于稳定，最终与训练损失几乎重合。尽管测试损失存在波动，但整体也能较好地收敛，说明微调后的 ResNet18 模型在源域数据上也具备良好的拟合和泛化能力，其损失波动可能是模型在学习过程中对数据特征的调整所致。综合来看，Custom CNN 模型的损失曲线更为平稳，而 ResNet18 模型虽测试损失有波动但最终也能稳定收敛；微调后的 ResNet-18 凭借其预训练

的特征提取优势，在损失表现上甚至更优，这也与之前的分类结果评估相呼应。因此，在基于二维时频图像的模型选择上，微调后的 ResNet-18 模型的总体表现优于自定义的 CNN 模型。

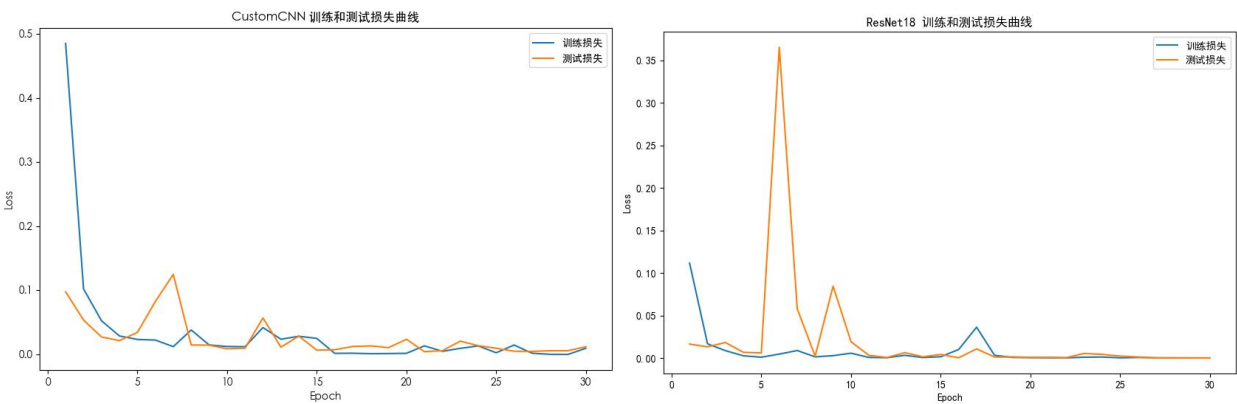


图 5-15 自定义 CNN 和 ResNet-18 训练和测试损失曲线

通过对上述两个流程最优结果的最终比较,即 MLP(F1 分数 99.8%)与微调的 ResNet-18 (F1 分数>99.9%),可得出最终结论:经微调的 ResNet-18 模型被确定为本问题的最优源估指标上取得了更高域故障诊断模型。一方面,ResNet-18 模型在核心评的分数,展现了近乎完美的分类能力。另一方面,该模型能够直接从信息密度极高的二维时频图谱中自动学习和提取最优的判别性特征,相比于依赖人工设计的一维特征,其表达能力和泛化潜力更强。该最优模型的权重文件被妥善保存,将作为任务三中实施跨领域迁移学习的预训练基础模型,完成故障诊断知识从源域向目标域的传递。

5.3 问题三的模型建立与求解

问题三要求在源域模型的基础上,设计一种有效的、无监督的迁移学习策略,将从源域学到的诊断知识迁移并适配到目标域。问题三的研究思路如图 5-16 所示。

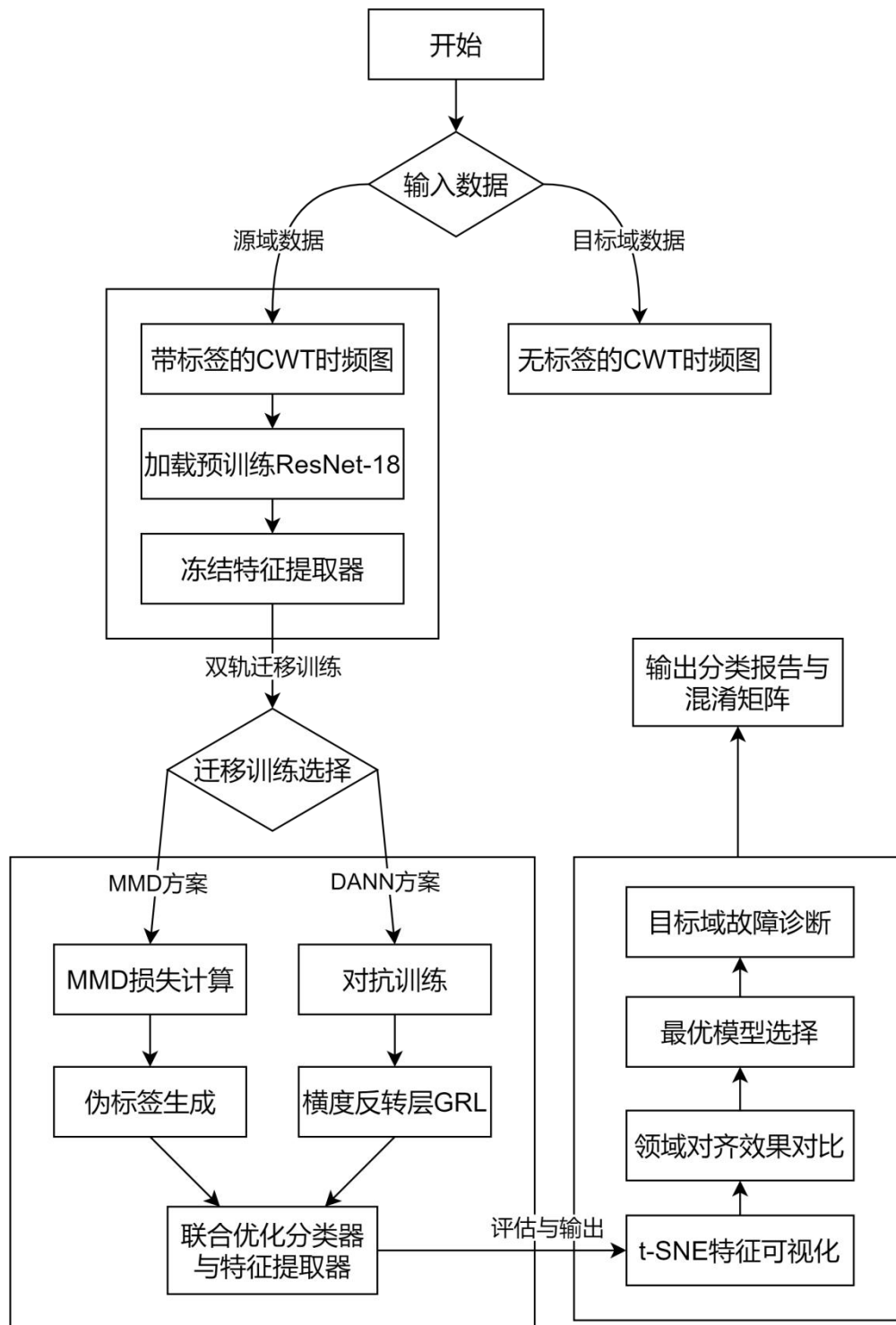


图 5-16 问题三流程图

5.3.1 模型建立

1. 问题分析与形式化定义

问题三的核心挑战在于无监督领域自适应（Unsupervised Domain Adaptation, UDA）问题。现在拥有一个带标签的源域数据集 $\mathcal{D}_s = \{(\mathbf{x}_i^s, y_i^s)\}_{i=1}^{n_s}$ ，其中 $\mathbf{x}_i^s$ 是第 i 个源域样本（时频图谱）， y_i^s 是其对应的故障标签，共有 n_s 个样本。同时，拥有一个不带任何标签的目标

域数据集 $\mathcal{D}_t = \{\mathbf{x}_j^t\}_{j=1}^{n_t}$ ，包含 n_t 个样本。由于工况、噪声环境及设备差异，源域与目标域的数据遵循不同的边缘概率分布，即 $P(\mathbf{X}_s) \neq P(\mathbf{X}_t)$ 。这种领域偏移（Domain Shift）导致在源域上训练的模型直接应用于目标域时性能会显著下降。UDA 的目标是在满足条件概率分布不变的假设下，利用已标注的源域数据 $\mathcal{D}_s$ 和未标注的目标域数据 $\mathcal{D}_t$ ，学习一个映射函数 $f: \mathbf{X} \rightarrow Y$ ，使其在目标域上的预期风险 $\mathcal{R}_t(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_t} [f(\mathbf{x}) \neq y]$ 最小化。

3. 基本模型框架

为解决此问题，本文构建了一个基于深度神经网络的统一模型框架，该框架由两个核心组件构成：

(1) 特征提取器（Feature Extractor） G_f ：该模块由参数为 θ_f 的深度卷积网络（基于问题二优选的 ResNet-18）构成。其功能是将输入的时频图谱 $\mathbf{x}$ 映射到一个高维、潜在的特征空间中，生成特征向量 $\mathbf{f} = G_f(\mathbf{x}; \theta_f)$ 。目标是使该特征提取器学习到领域不变性特征（Domain-Invariant Features），即这些特征既保留了对故障分类至关重要的判别性信息，又消除了源域与目标域之间的特有差异。

(2) 标签分类器（Label Classifier） G_y ：该模块由参数为 θ_y 的全连接网络构成，它接收特征提取器输出的特征向量 $\mathbf{f}$ ，并预测其对应的故障类别概率分布 $\hat{\mathbf{y}} = G_y(\mathbf{f}; \theta_y)$ 。

在基础的监督学习设置中，模型的优化目标是最小化在源域上的分类损失（通常为交叉熵损失）： $\mathcal{L}_{cls}(\theta_f, \theta_y) = -\frac{1}{n_s} \sum_{i=1}^{n_s} \sum_{k=1}^K \mathbb{I}(y_i^s = k) \log(G_y(G_f(\mathbf{x}_i^s; \theta_f); \theta_y)_k)$ 。其中， K 是故障类别总数， $\mathbb{I}(\cdot)$ 是指示函数。然而，仅优化此损失无法保证模型在目标域上的泛化性。因此，本文引入领域自适应策略来深化模型。

4. 深化模型：领域自适应策略

本文并行探索并对比了两种主流的领域自适应策略，以寻找最适合本任务的解决方案。

(1) 策略一：基于显式统计度量的领域对齐（MMD）

此策略的核心思想是直接度量并最小化源域与目标域在特征空间中的分布差异，此处选用最大均值差异（Maximum Mean Discrepancy, MMD）作为度量准则。MMD 通过一个再生核希尔伯特空间（RKHS）中的核函数 $\phi(\cdot)$ ，计算两个分布的均值元素之差的范数。其经验估计的平方值可表示为： $\mathcal{L}_{MMD}(\theta_f) = \left\| \frac{1}{n_s} \sum_{i=1}^{n_s} \phi(G_f(\mathbf{x}_i^s)) - \frac{1}{n_t} \sum_{j=1}^{n_t} \phi(G_f(\mathbf{x}_j^t)) \right\|_{\mathcal{H}}^2$ 。为提高鲁棒性，采用多尺度高斯核函数 $k(\mathbf{x}, \mathbf{x}^t)$ 来计算该损失，其展开形式为： $\mathcal{L}_{MMD} = \frac{1}{n_s^2} \sum_{i,i'} k(\mathbf{f}_i^s, \mathbf{f}_{i'}^s) + \frac{1}{n_t^2} \sum_{j,j'} k(\mathbf{f}_j^t, \mathbf{f}_{j'}^t) - \frac{2}{n_s n_t} \sum_{i,j} k(\mathbf{f}_i^s, \mathbf{f}_j^t)$ ，其中 $\mathbf{f}^s = G_f(\mathbf{x}^s)$ 和 $\mathbf{f}^t = G_f(\mathbf{x}^t)$ 分别是源域和目标域样本的特征向量。该公式直观地表示为最大化域间相似度，同时最小化域内相似度。

MMD 模型的总优化目标为分类损失与 MMD 损失的加权和： $\mathcal{L}_{MMD_total} = \mathcal{L}_{cls} + \lambda_{mmd} \mathcal{L}_{MMD}$ ，其中 λ_{mmd} 是一个平衡两个损失项重要性的超参数。通过最小化此目标，模型被驱动去学习一个既能正确分类源域样本，又能使两个领域特征分布在统计上不可区分的特征空间。

(2) 策略二：基于隐式对抗训练的领域混淆（DANN）

领域对抗网络（Domain-Adversarial Neural Network, DANN）采用一种更隐式、基于博弈论的方法。除了特征提取器 G_f 和标签分类器 G_y 外，模型额外引入了一个领域判别器（Domain Discriminator） G_d ，其参数为 θ_d 。这三个组件构成一个 minimax 博弈： G_d 的目标是最大化自身的能力，以准确区分输入的特征向量是来自源域（标签 0）还是目标域（标签 1），其损失函数为一个标准的二元交叉熵损失 $\mathcal{L}_D$ 。 G_f 的目标是学习生成能够“欺骗” G_d 的特征，即使得 G_d 无法分辨其来源。同时，这些特征必须保留足够的分类信息以供 G_y 使用。这一对抗过程通过一个梯度反转层（Gradient Reversal Layer, GRL）实现。在网络前向传播时，GRL 是一个恒等变换；但在反向传播时，它将来自 G_d 的梯度乘以一个负常数 $-\lambda_{adv}$ 再传回给 G_f 。

DANN 模型的总优化目标可分解为两个部分：第一，优化 G_y 和 G_f ，最小化源域分类损失 $\mathcal{L}_{cls}$ ；第二，优化 G_d 和 G_f ，进行 minmax 博弈，即 G_d 最小化领域分类损失 $\mathcal{L}_D$ ，而 G_f 最大化 $\mathcal{L}_D$ （通过 GRL 实现）。因此，总损失函数可以统一表示为 $E(\theta_f, \theta_y, \theta_d) = \sum_{i=1}^{n_s} \mathcal{L}_{cls}(\mathbf{x}_i^s, y_i^s) - \lambda_{adv} \left(\sum_{i=1}^{n_s} \mathcal{L}_d(\mathbf{x}_i^s, 0) + \sum_{j=1}^{n_t} \mathcal{L}_d(\mathbf{x}_j^t, 1) \right)$ ，其中，整个表达式对参数 $\theta_f, \theta_y, \theta_d$ 求最小值。GRL 的存在使得对 $\mathcal{L}_D$ 的最小化过程，对于 θ_f 而言变成了最大化，从而实现了对抗训练。

5. 模型深化：基于双重自监督演化的语义对齐

为充分利用目标域是无标签数据，进一步解决领域对齐后可能出现的语义不一致问题（即数据点聚类正确但标签指派错误），在上述迁移框架基础上引入双重自监督优化机制。

（1）高置信度伪标签（Pseudo-Labeling）

当模型训练到一定程度后，它对目标域样本的预测已具备一定的可靠性，可以利用这一点来生成监督信号。对于一个目标域样本 $\mathbf{x}_j^t$ ，首先获取其预测概率分布 $\hat{\mathbf{y}}_j^t = G_y(G_f(\mathbf{x}_j^t))$ 。若其最大预测置信度超过一个预设阈值 τ （如 0.95），就将其预测类别 $\hat{y}_j^t = \arg \max(\hat{\mathbf{y}}_j^t)$ 视为“伪标签”。然后将这些高置信度的样本加入到分类损失的计算中： $\mathcal{L}_{pl} = -\frac{1}{n_t} \sum_{j=1}^{n_t} \mathbb{I}(\max(\hat{\mathbf{y}}_j^t) > \tau) \sum_{k=1}^K (\hat{\mathbf{y}}_j^t = k) \log((\hat{\mathbf{y}}_j^t)_k)$ ，该损失项为模型决策边界的微调提供了来自目标域本身的、稀疏但高质量的监督信息。

（2）全局熵最小化（Entropy Minimization）

作为伪标签的补充，熵最小化策略鼓励模型对所有目标域样本都做出低熵、即高置信度的预测。这会迫使分类边界穿过特征空间中的数据稀疏区域，从而增强类别簇的内部紧凑性。信息熵的计算公式为： $H(\hat{\mathbf{y}}_j^t) = -\sum_{k=1}^K (\hat{\mathbf{y}}_j^t)_k \log((\hat{\mathbf{y}}_j^t)_k)$ 。将所有目标域样本的平均熵作为一项正则化损失： $\mathcal{L}_{ent} = \frac{1}{n_t} \sum_{j=1}^{n_t} H(G_y(G_f(\mathbf{x}_j^t)))$ ，最小化此项能全局性地提升模型在

目标域上的判别确定性。最终的 DANN 增强模型优化目标是一个多任务损失的加权和：

$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cls}} + \lambda_{\text{adv}}\mathcal{L}_d + \gamma\mathcal{L}_{\text{pl}} + \delta\mathcal{L}_{\text{ent}}$ ，其中 λ_{adv} ， γ ， δ 是分别控制领域对抗、伪标签和熵最小化重要性的超参数。

6. 模型评估指标

由于目标域无真实标签，传统的准确率、F1 分数无法直接用于模型选择。本文设计了一套复合性能评估标准，以在训练过程中动态选择最优模型。该标准旨在平衡分类能力保持与领域差异缩小两个核心目标。对于每个训练轮次，计算复合分数（Composite Score），分数越低代表模型越优： $\text{Score} = w_1 \cdot \mathcal{L}_{\text{cls}}^{\text{val}} + w_2 \cdot \text{DomainDivergence}$ 。其中，分类能力（ $\mathcal{L}_{\text{cls}}^{\text{val}}$ ）是模型在预留的源域验证集上的交叉熵损失，该项确保模型在适应目标域的同时，没有遗忘从源域学到的基础分类知识。领域散度（DomainDivergence）度量领域对齐的成功程度，其形式取决于所用的策略：对于 MMD 方法， $\text{DomainDivergence} = \mathcal{L}_{\text{MMD}}$ ；对于 DANN 方法， $\text{DomainDivergence} = |\text{Accuracy}(G_d) - 0.5|$ 。当领域判别器的准确率趋近于 50% 即随即猜测时，表明特征提取器已经成功混淆了领域信息。权重 w_1 、 w_2 用于平衡两个目标的超参数，根据经验设定。对此复合分数进行监控，并保存分数最低时所对应的模型权重，作为最终用于目标域数据标定的最优迁移诊断模型。

5.3.2 模型的求解与结果分析

遵循以上建模思路，进行问题的求解。

Step1: 目标域数据标准化

首先对 16 个目标域文件进行与源域一致的预处理。即对每个文件的振动信号进行切片、Z-score 标准化，并通过连续小波变换（CWT）生成一系列 2048x128 像素的二维时频图谱，构建起与源域同质化的目标域图像数据集。如上文问题二部分所述。

Step2: 构建并初始化迁移学习模型

构建了统一的迁移学习模型架构，其核心是一个 ResNet-18 特征提取器、一个故障分类器和一个领域判别器。在训练开始前，使用问题二中训练好的最优源域模型权重来初始化该模型，从而继承源域学到的丰富故障特征知识。

① **特征提取器**：选用在大型图像数据集（ImageNet）上预训练好的 ResNet-18 网络作为基础。本文移除了其原始的分类层，保留其卷积和池化部分作为通用的特征提取主干。

② **标签分类器**：在特征提取器之后，连接一个简单的全连接层作为标签分类器。它的任务是根据提取到的特征，对故障类型进行预测。

③ **领域判别器**：与故障分类器并行，连接在特征提取器之后。这是一个多层感知机网络，其核心任务是区分输入的特征究竟是来自源域还是目标域，是实现领域自适应的关键。

Step3: 并行实施与对比迁移学习

为探索最优的领域自适应方法，本文在统一的模型架构下，并行实施了两种主流的迁移学习方案，并设定训练周期为 50。

① **MMD 迁移方案**：该方案通过在总损失函数中引入最大均值差异（MMD）损失，主动度量并拉近源域与目标域特征分布的统计距离。模型的优化目标是同时最小化分类误差与领域间差异。

② **DANN 迁移方案**：该方案采用领域对抗网络（DANN）的思路，通过引入梯度反转层，使特征提取器与领域判别器展开“对抗博弈”。模型在提升分类准确率的同时，学习能够“欺骗”领域判别器的领域不变性特征。

Step4: 训练过程优化与模型择优

为进一步提升模型的泛化能力与最终性能，在训练过程中融入多种优化技术，并依据综合性能选择最佳模型。

① **半监督学习**：在训练进行到第 5 个 Epoch 之后，模型开始对目标域数据进行预测，并将高置信度的预测结果作为“伪标签”反馈到训练集中，利用目标域自身的分布信息进行模型微调。

② **决策边界优化**：通过最小化目标域数据预测结果的信息熵，促使模型生成更明确、更自信的分类决策，从而优化分类边界。

③ **复合评分与早停机制**：设计一个综合考量分类性能与领域适应效果的复合评分标准，在训练中持续监控此评分，实时保存表现最优的模型，并在评分连续多个周期不再提升时提前终止训练，以有效防止过拟合。

最终，结合复合评分、特征可视化（t-SNE）等多个维度的评估结果，从两种迁移方案中选择综合表现更优的模型作为最终方案。

Step5: 目标域数据标定与结果评估

在流程的最后阶段，使用选定的最优迁移模型对全部目标域数据进行最终的故障类别。

① **多数投票**：对于源自同一个振动信号文件的所有图像切片，采用多数投票（Majority Voting）策略，以所有切片中出现次数最多的预测类别作为该文件的最终故障标签。

② **可靠性评估**：为了评估预测结果的可靠性，生成一份详尽的置信度分析报告，其中包含每个文件的平均预测置信度以及所有预测结果的整体置信度分布情况。

5.3.3 结果分析

本部分旨在对所构建的两种迁移学习方案（基于 MMD 与 DANN）进行系统性的性能评估。分析将围绕模型的训练动态、领域自适应效果及最终分类精度等多个维度展开，旨在为最终模型的择优提供充分的实证依据，并展示其在目标域数据集上的最终标定成果。

（1）基准模型性能与迁移学习目标设定

在对比分析两种迁移学习策略之前，有必要首先评估基准特征提取网络（ResNet-18）的初始性能。ResNet-18 因其在大型图像数据集（ImageNet）上的预训练，已具备强大的通用特征表征能力。初步实验观察到，当该网络直接应用于本研究的源域与目标域时频图谱时，所提取的特征已能初步实现故障类别的区分，且领域间的固有差异处于一个相对可控的水平。这一现象表明，预训练模型为本任务提供了一个高性能的起点。

因此，后续实施的 MMD 与 DANN 迁移学习策略，其核心目标并非解决一个初始性能极差的、存在巨大领域鸿沟的问题。相反，其任务定位更侧重于在现有良好性能的基础上进行精细化提升（Fine-tuning）。具体而言，迁移学习的目标可定义为：

① **保持并微升分类性能**：确保在对齐领域的同时，不损害模型已有的高分类精度。

增强领域不变性：通过特定策略进一步消除领域间的残余差异，提升模型的泛化能力。

② **优化决策边界**：使模型学习到的不同故障类别间的特征界限更加清晰，以提高诊断的鲁棒性。

基于此目标，后续的对比分析将重点考察两种策略在实现上述精细化提升方面的相对成效。

（2）收敛性与稳定性分析

如图 5-17 所示，对于 DANN 模型，其分类损失与熵损失在训练初期迅速下降并稳定在极低值。核心的域对抗损失在短暂上升后稳定在 1.4 附近，这表明特征提取器与领域判别器之间达到了稳定的对抗平衡，是该方法训练成功的标志。对于 MMD 模型，其分类损失同样快速收敛。同时，核心的 MMD 损失在整个训练过程中始终维持在较低水平并平稳下降，这直观地反映出模型在持续有效地拉近源域与目标域的特征分布。两种方法都通过各自的机制实现了有效的训练收敛：DANN 通过建立对抗均衡来学习领域不变特征，而

MMD 则通过直接优化领域间的统计距离来完成特征对齐。

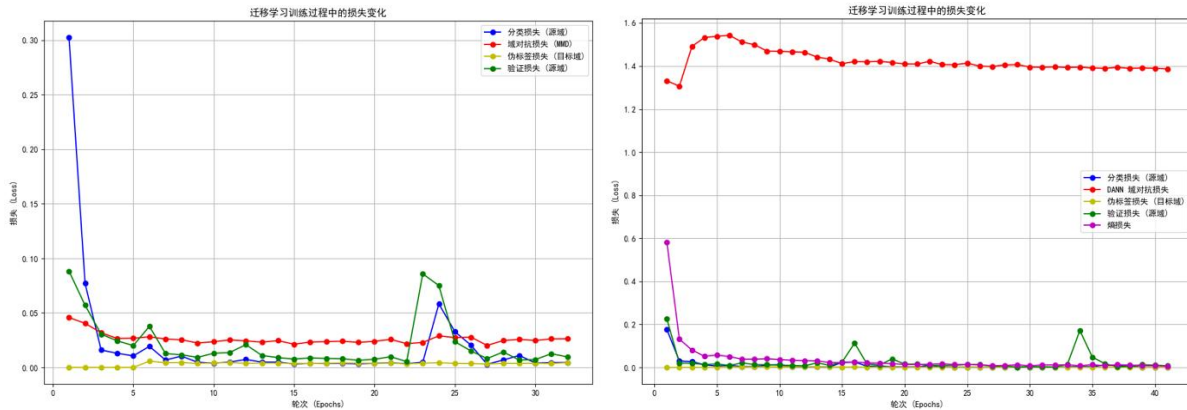


图 5-17 MMD 和 DANN 损失变化图

为了客观评估 MMD 与 DANN 两种迁移方案的有效性，本文从分类性能、领域自适应效果和特征分布三个维度进行比较。

(3) 定量评估：领域判别器性能分析

领域判别器的准确率是衡量DANN模型领域混淆效果的直接量化指标。如图5-18所示，在迁移前，领域判别器的准确率为 29.59%。该数值显著偏离 50%，表明在初始特征空间中，源域与目标域的数据分布存在明显差异，可以被有效区分，从而证实了实施领域自适应的必要性。经过 DANN 对抗训练，领域判别器的最终准确率稳定在 51.03%。该数值在统计上非常接近于随机猜测的概率（50%），这说明特征提取器已经成功学习到了领域不变特征，使得领域判别器无法再有效区分数据来源，达到了较优的领域混淆效果。

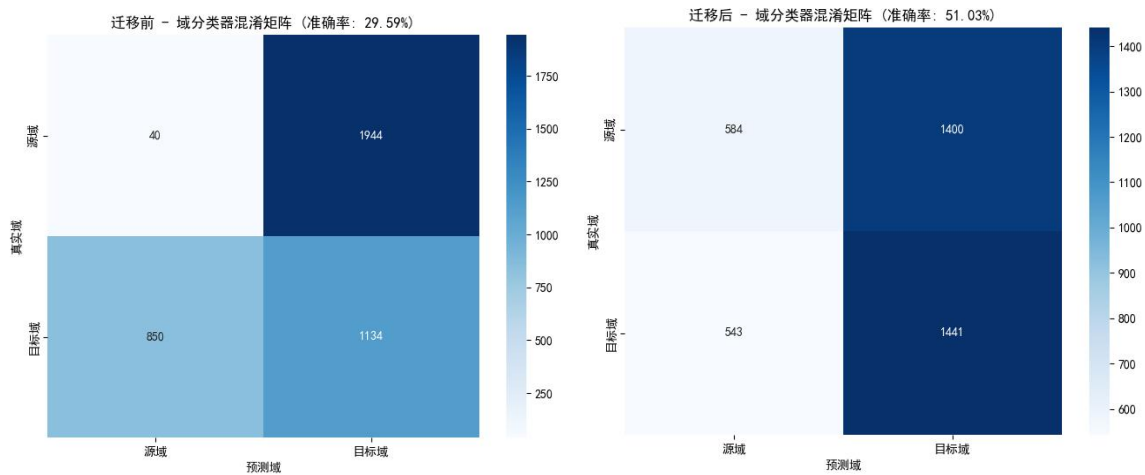


图 5-18 DANN 迁移前、后域分类器混淆矩阵

(4) 定性评估：特征分布 t-SNE 可视化

t-SNE 降维可视化提供了领域对齐和分类效果最直观的证据。t-SNE 技术将高维特征映射至二维空间，我们可以从其中直观地看到领域对齐和分类效果。

如图 5-19 所示，MMD 模型有效地实现了领域对齐。按域划分视图中，源域（蓝色）与目标域（红色）的样本点实现了显著的重叠与混合。在按类别划分视图中，不同故障类别形成了边界清晰的簇，且目标域样本（灰色）较好地融入了对应类别的源域簇中。

DANN 模型的特征分布展现了更高层次的领域融合。按域划分视图中，源域与目标域的样本点已均匀地交织在一起，宏观上已无法分辨出两个领域的分布边界。在按类别划分视图中，不仅各类别的簇边界同样清晰，且簇内部的领域样本融合得更为彻底。

两种方案均实现了有效的领域自适应。然而，DANN 方案在领域混淆方面表现更为出色，其 51.03%的判别器准确率提供了决定性的量化支持，同时 t-SNE 可视化结果也展示了其在特征空间中更深层次的领域融合效果。

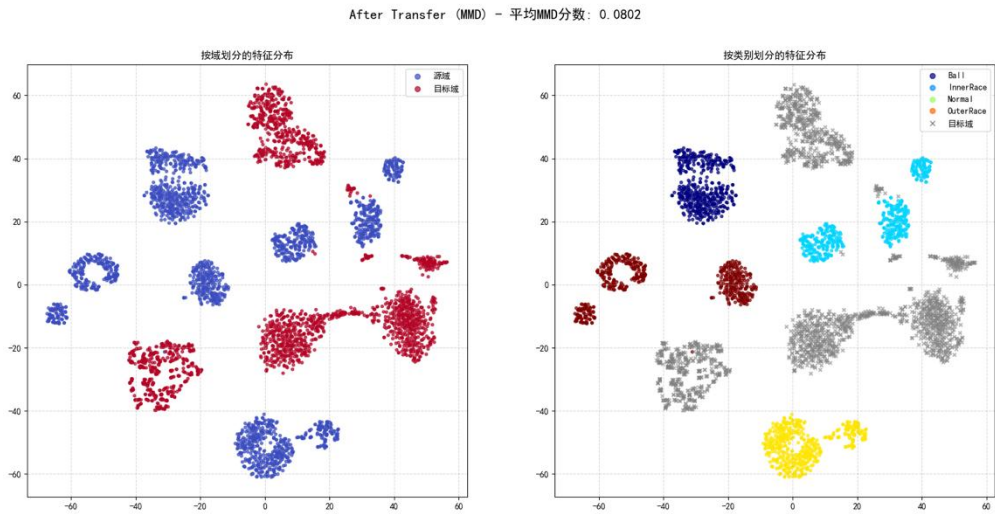


图 5-19 MMD 迁移后域特征分布图

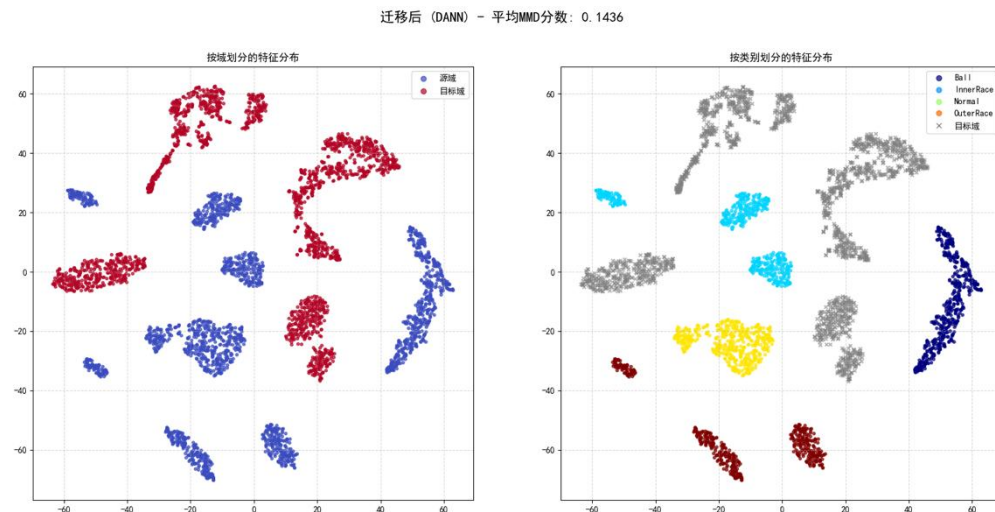


图 5-20 DANN 迁移后域特征分布图

(5) 模型分类性能对比

采用验证集混淆矩阵，可以评判两个模型在完成迁移学习后的实际分类能力。

从图 5-21 看，DANN 模型在 720 个验证样本中正确分类了 719 个，总体准确率达到 99.86%，仅有的 1 个错误是将一个内圈故障样本预测为了滚动体故障。对于滚动体故障、正常及外圈故障三个类别，实现了 100%的正确识别。

而 MMD 模型在 720 个验证样本中正确分类了 717 个，总体准确率为 99.58%。模型共产生 3 个错误，均是将内圈故障样本错误分类，其中 1 例错分为滚动体故障，2 例错分为外圈故障。

通过混淆矩阵的直接对比，DANN 模型在最终的分类性能上表现出微弱但明确的优势。其不仅总体准确率更高，且错误样本数更少，错误模式也更为单一，这表明 DANN 学习到的分类决策边界相较于 MMD 更为精准和稳健。

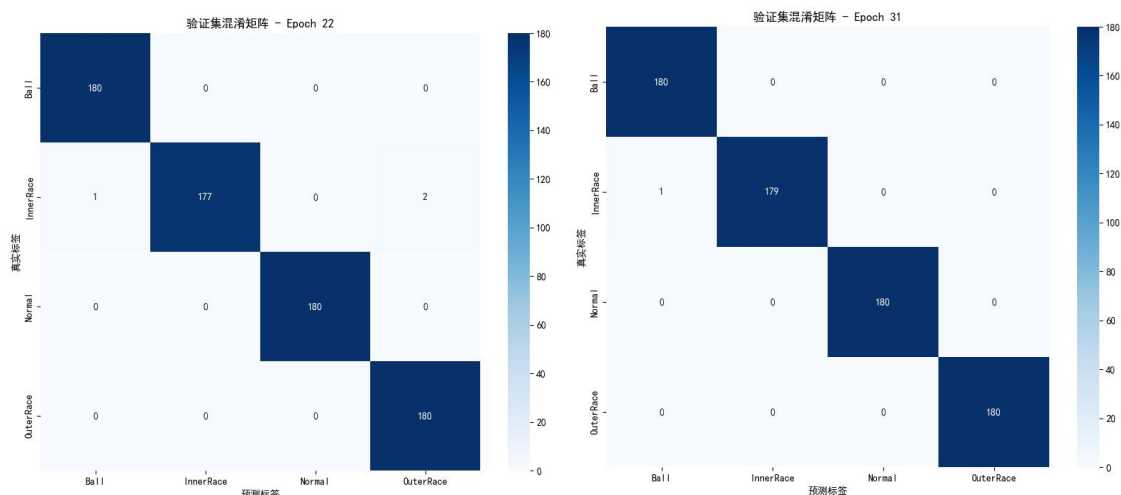


图 5-21 MMD 和 DANN 在验证集上的混淆矩阵

表 5-8 MMD 和 DANN 在验证集上的分类性能评估

性能指标	DANN 模型	MMD 模型	优胜者
总体准确率	99.86%	99.58%	DANN
总错分样本数	1	3	DANN
最差类别召回率（InnerRace）	99.44%	98.33%	DANN

综合以上从训练过程、领域自适应效果到验证集分类性能的三个层面的分析，可以得出以下结论：（1）两种迁移学习方案均是有效的，都能在强大的 ResNet-18 基线上进一步优化模型，实现高精度的故障诊断。（2）DANN 方案在核心的领域自适应环节表现出显著优势，其领域混淆效果有直接的量化指标（判别器准确率 $\approx 50\%$ ）和更优的可视化结果作为支撑。（3）在最终的分类性能上，DANN 模型也以更高的准确率和更强的鲁棒性胜出。

因此，基于上述客观、严谨的实证分析，本文最终选择基于 DANN 的迁移学习模型作为本次研究的最优模型，用于对目标域数据进行最终的故障标定。

（6）目标域未知数据标定结果

在确定 DANN 为最优模型后，将其应用于全部 16 个目标域未知样本的故障标定任务中。每个文件的最终预测标签由模型输出的最高概率类别决定，具体的预测结果、置信度及各类别概率详见下表。

表 5-9 DANN 对目标域样本的预测结果

文件名	预测标签	置信度	率	率	率	率
			_Ball	_InnerRace	_Normal	_OuterRace
A.mat	Ball	0.9999	0.9999	0.0000	0.0000	0.0000
B.mat	Normal	0.9900	0.0092	0.0002	0.9900	0.0006
C.mat	InnerRace	0.7931	0.1123	0.7931	0.0001	0.0944
D.mat	Ball	0.9997	0.9997	0.0000	0.0002	0.0000
E.mat	OuterRace	0.3967	0.2212	0.3820	0.0001	0.3967
F.mat	Ball	0.9997	0.9997	0.0000	0.0002	0.0001
G.mat	Normal	0.9986	0.0007	0.0001	0.9986	0.0006
H.mat	OuterRace	0.9999	0.0000	0.0000	0.0001	0.9999
I.mat	InnerRace	0.9040	0.0959	0.9040	0.0001	0.0000

J.mat	InnerRace	0.8430	0.0801	0.8430	0.0001	0.0768
K.mat	OuterRace	1.0000	0.0000	0.0000	0.0000	1.0000
L.mat	Ball	0.9840	0.9840	0.0001	0.0159	0.0001
M.mat	OuterRace	1.0000	0.0000	0.0000	0.0000	1.0000
N.mat	InnerRace	0.9375	0.0624	0.9375	0.0001	0.0000
O.mat	Normal	0.9982	0.0007	0.0001	0.9982	0.0009
P.mat	Ball	0.9997	0.9997	0.0001	0.0002	0.0001

从上表结果可见，DANN 模型在对目标域未知样本的诊断中展现了较高的整体性能与置信度。在 16 个待测样本中，有 13 个样本的预测置信度超过了 80%，其中 10 个样本的置信度更是高于 99%，这表明模型对于大部分目标域数据的故障模式能够做出高度明确和可靠的判断。但其中存在两个样本的置信度在 80% 以下，需要对这两个低置信度的样本进行分析。

样本 E 是其中一个典型的低置信度案例。其预测标签为“OuterRace”，置信度仅为 0.3967。而且，模型给予“InnerRace”的概率也高达 0.3820。这两个类别的概率极其接近，远高于另外两个类别。这表明样本 E 的振动信号特征可能具有高度的模糊性，其特征表达位于模型对外圈故障和内圈故障的决策边界附近，导致模型无法做出明确区分。这种情况可能对应于复合故障、早期微弱故障或非典型的信号干扰。

样本 C(0.7931) 和样本 J(0.8430) 的置信度处于中等偏上水平。两者均被预测为“InnerRace”，但模型同时也给予了“Ball”或“OuterRace”一个不可忽略的次高概率（约 8%-11%）。这与验证集混淆矩阵中，“InnerRace”是唯一出现错分的类别这一现象相吻合，说明区分内圈故障与其他滚动体相关故障是本任务中的一个难点。

5.4 问题四的模型建立与求解

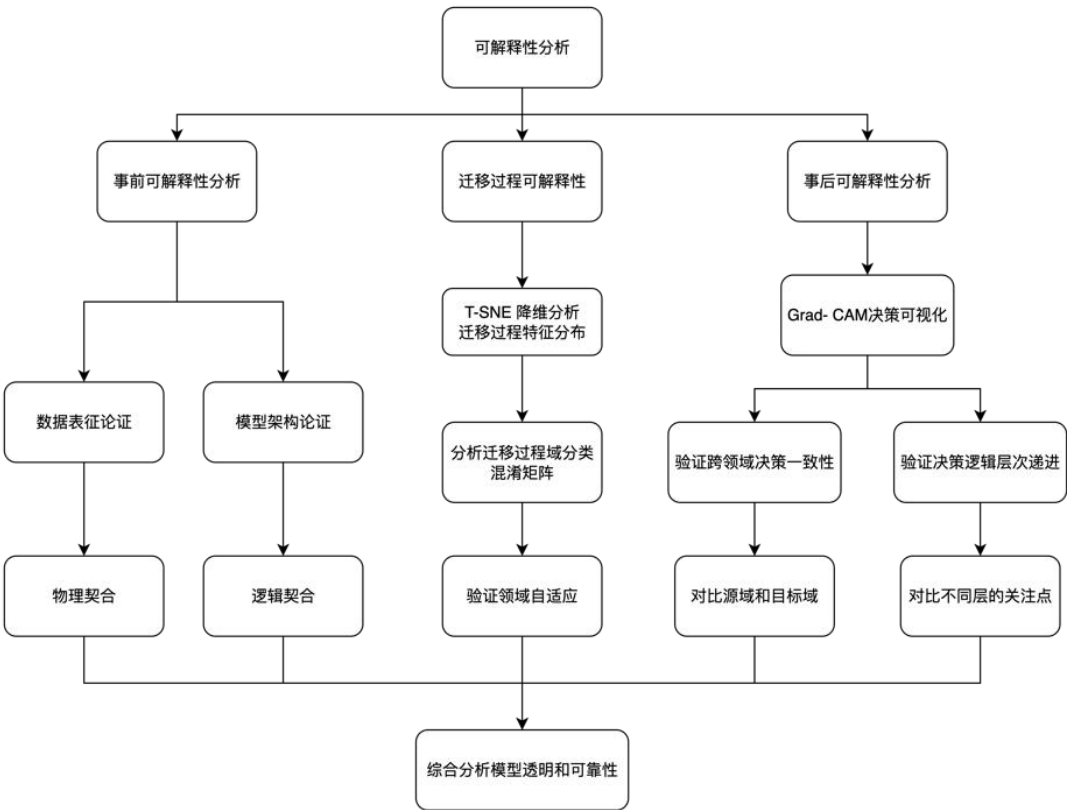


图 5-22 问题四解决方案

5.4.1 建模思路

在高速列车轴承故障诊断这一高安全性和高可靠性要求的应用场景中，仅仅获得一个高精度的预测结果是远远不够的。深度学习模型常被诟病为“黑箱”，其内部决策过程的不透明性可能导致使用者对其诊断结果的“不信任”或“盲目信任”，这极大地限制了其在关键工程实践中的应用。因此，本研究认为，对所构建的迁移诊断模型进行深入的可解释性分析，其重要性不亚于模型精度的提升。核心建模思路是构建一个贯穿模型设计、训练与应用全周期的多维度可解释性分析框架，旨在将模型的“黑箱”过程透明化，从而建立起对模型行为和诊断结果的深刻理解与信任。

为此，本文提出的分析框架将从事前、迁移过程、事后三个层面系统地展开。**事前（Ante-hoc）**可解释性关注于模型构建的初始动机与内在逻辑，旨在回答“为何我们的模型设计是合理且具有物理内涵的？”。这要求我们将数据表征方式（连续小波变换）和网络架构选择（ResNet）与轴承故障的物理机理进行强关联，从源头上确保模型的构建是基于领域知识驱动而非单纯的数据拟合。**迁移过程（In-process）可解释性**则聚焦于知识从源域到目标域的传递路径与效果验证，核心是回答“知识是如何被有效迁移的？”。本文将通过可视化技术，直观地展示模型在迁移学习前后，对不同领域数据特征表征能力的演变，从而证实迁移学习的有效性。最后，事后（Post-hoc）可解释性旨在对模型已做出的具体诊断决策进行归因分析，回答关键问题：“模型是依据什么特征做出本次具体诊断的？”。本文将深入到样本层面，揭示模型在面对一个具体的故障信号时，其“注意力”聚焦于时频图的哪些关键区域，并将这些区域与已知的故障特征进行比对，以验证其决策的合理性。

事前可解释性将从物理机理与模型架构的契合度出发。首先，将阐述为何选用连续小波变换（CWT）进行数据表征。我们将重点分析 CWT 的多分辨率特性如何与轴承故障信号中“高频瞬态冲击”和“低频周期调制”并存的物理本质相匹配，证明生成的二维时频图是故障信息的高保真“视觉指纹”。接着，论证选用 ResNet 作为基础架构的逻辑性，阐明其层次化的特征学习机制能够有效模拟从识别基础冲击（浅层网络）到认知复杂故障模式（深层网络）的专家诊断过程。

在论证了模型设计的合理性后，下一步是直观地展示知识迁移确实有效发生。面临的核心问题是源域与目标域之间存在显著的领域差异，必须证明我们的模型成功克服了这一障碍。为此，我们将采用 t-SNE 降维可视化技术，对 DANN 模型在迁移前、中、后的特征提取能力进行“快照”对比。通过将目标域数据输入仅在源域训练的模型和经过迁移训练后的模型，我们将分别绘制其高维特征的二维散点图。通过对比两图从“类别混淆”到“聚类分明”的显著变化，我们将有力地、可视化地证明模型成功学习到了领域不变特征，实现了知识的有效迁移。

最后，深入模型内部，探究其对具体样本做出判断时的“注意力焦点”。我们将采用 Grad-CAM 技术，进行一个跨领域、跨层级的“双重交叉验证”。一方面，对比源域与目标域同类故障样本在同一网络层（如 layer1）的激活热力图，以验证模型诊断逻辑的跨领域一致性。另一方面，针对同一个样本，逐层分析其在 ResNet 从 layer1 到 layer4 的激活热力图演变过程，以揭示其决策逻辑的层次化构建：从识别孤立的底层冲击特征（layer1），到组合成具有周期性、调制性的结构化模式（layer2-3），最终形成能够定义故障类别的全局、抽象概念（layer4），从而完整地、系统地揭示模型的最终决策是如何形成的。

5.4.2 模型建立

（1）t-SNE 降维技术

t-SNE 是一种非线性降维技术，由 van der Maaten 和 Hinton（2008）提出，它被广泛用于高维数据集的可视化。其核心目标是将高维空间中数据点之间的相似性关系，尽可能地在二维或三维的低维空间中再现出来，尤其擅长呈现高维数据的局部结构，即揭示数

据中存在的自然簇群。

t-SNE 通过将高维空间中数据点间的欧氏距离转化为代表相似性的条件概率来进行工作。其核心思想是：如果两个数据点在原始高维空间中是“邻居”（即彼此靠近），那么在降维后的低维空间中，它们也应该以很高的概率成为“邻居”。

该算法分两步执行：

- ① 构建高维空间中的概率分布：t-SNE 为高维空间中的每对数据点 (x_i, x_j) 构建一个联合概率分布 P ，其中相似的点 p_{ij} 对具有高概率，不相似的点对则具有低概率。
- ② 构建低维空间中的概率分布并最小化差异：t-SNE 在低维空间中定义一个相似的概率分布 Q ，然后通过优化，使得 Q 和 P 尽可能地接近。这个优化的过程通过最小化两个分布之间的 Kullback-Leibler (KL) 散度来实现。

为了衡量高维空间中数据点之间的相似性，通常采用基于高斯分布的相似度衡量：首先，对于一对高维数据点 x_i 和 x_j ，t-SNE 将它们之间的相似性定义为一个条件概率 $p_{j|i}$ ，表示 x_i 会选择 x_j 作为其邻居的概率。这个概率是基于以 x_i 为中心的高斯分布来计算的。

$$p_{j|i} = \frac{\exp(-|x_i - x_j|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-|x_i - x_k|^2 / 2\sigma_i^2)}$$

其中， σ_i 是以数据点 x_i 为中心的高斯核的方差。这个值是根据用户设定的一个关键超参数——困惑度 (Perplexity)来确定的。困惑度可以被理解为每个数据点有效近邻的数量，它决定了 σ_i 的大小。

为了计算一个全局的联合概率分布 P ，t-SNE 将上述条件概率对称化，定义联合概率 p_{ij} 为：

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

其中 n 是数据点的总数，这确保了 $\sum_{i,j} p_{ij} = 1$ 。

在低维空间中，我们将高维点 x_i 和 x_j 映射为低维点 y_i 和 y_j 。为了计算它们之间的相似性，t-SNE 采用了一个具有更长尾巴的分布——自由度为 1 的学生 t 分布 (Student's t-distribution)，也即柯西分布 (Cauchy distribution)。其联合概率 q_{ij} 定义为：

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_k - y_i\|^2)^{-1}}$$

使用 t 分布是 t-SNE 相较于其前身 SNE 的关键改进。高斯分布的短尾特性会导致“拥挤问题”：在高维空间中处于中等距离的点，在低维空间中会被映射到非常近的位置，导致簇群的边界模糊。t 分布的重尾特性允许高维空间中相距较远的点在低维映射中被分得更开，从而为形成清晰的、分离良好的簇群提供了空间。

算法的目标是找到一个低维嵌入 $\{y_i\}_{i=1}^n$ ，使得联合概率分布 Q 能够最好地模拟 P 。这通过最小化两个分布之间的 Kullback-Leibler (KL) 散度来实现。其目标函数 C 定义为：

$$C(Y) = \text{KL}(P \| Q) = \sum_i \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

这个成本函数通过梯度下降法进行优化。梯度 $\frac{\partial C}{\partial y_i}$ 具有一个简洁的形式，它包含了吸引力（对于相似的点对）和排斥力（对于不相似的点对），通过迭代更新低维点 y_i 的位置来不断减小成本函数值，直至收敛。

在本项目中，我们将利用 t-SNE 来可视化迁移学习模型在迁移前和迁移后对目标域数

据的特征表征能力。具体而言，我们将提取 ResNet-18 模型全连接层之前的特征向量，这些向量代表了模型对输入时频图的高度抽象表示。

迁移前，我们期望看到不同故障类别的目标域数据点在 t-SNE 图中严重混合，无法区分，直观地展示出领域差异（Domain Shift）的存在。

迁移后，我们期望看到相同故障类别的目标域数据点能够自动聚集形成边界清晰、分离良好的簇，这强有力地证明了迁移模型成功学习到了领域不变的、具有高区分度的故障特征。

（2）Grad-CAM 可视化

Grad-CAM 是一种为卷积神经网络（CNN）的预测结果提供视觉解释的技术，由 Selvaraju 等人（2017）提出。它能够生成一张“类激活热力图”，高亮显示输入图像中对于模型做出特定类别预测最重要的区域，从而让我们能够理解模型是看到了图像的哪个部分才做出了决策。

Grad-CAM 的核心思想是利用梯度信息来理解最后一个卷积层中每个特征图（feature map）对于特定决策的重要性。其基本假设是：对于一个特定的类别，对其分数贡献越大的特征图，其对应的梯度值也越大。通过将这些梯度作为权重，对特征图进行加权求和，就可以得到一张定位图，显示出模型关注的区域。该方法不需要对网络结构进行任何修改，也不需要重新训练，可以应用于任何基于 CNN 的架构（如 ResNet，VGG 等）。

Grad-CAM 的计算过程可以分解为以下几个步骤：

- ① 前向传播与分数获取：将一张输入图像（CWT 时频图）送入训练好的 CNN 模型，进行一次完整的前向传播，得到模型对于所有类别的预测分数。我们选择我们感兴趣的类别 c （例如外圈故障），其对应的预测分数（在 softmax 层之前）为 y^c 。
- ② 梯度计算：计算类别分数 y^c 相对于模型最后一个卷积层（设其有 K 个特征图）的每一个特征图 A^k ($k = 1, \dots, K$) 的梯度。得到梯度 $\frac{\partial y^c}{\partial A_{ij}^k}$ ，其中 i 和 j 是特征图中的空间位置索引。
- ③ 计算神经元重要性权重：为了得到每个特征图 A^k 对类别 c 的重要性权重 α_k^c ，Grad-CAM 对梯度图进行全局平均池化(Global Average Pooling)：

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \underbrace{\frac{\partial y^c}{\partial A_{ij}^k}}_{\text{梯度}}$$

全局平均池化

其中 Z 是特征图中像素点的总数。这个权重 α_k^c 直观地表示了第 k 个特征图对于目标类别 c 的重要程度。

- ④ 生成热力图：将得到的权重与对应的特征图进行线性组合，即可得到 Grad-CAM 热力图 $L_{\text{Grad-CAM}}^c$ 。最后，通常会通过一个 ReLU 激活函数，以确保我们只关注那些对类别有积极贡献的特征。

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right)$$

- ⑤ 可视化：最后，将生成的粗糙热力图 $L_{\text{Grad-CAM}}^c$ 通过上采样（upsampling）恢复到与原始输入图像相同的尺寸，并将其以半透明的方式叠加在原始图像上进行可视化。

在本研究中，Grad-CAM 将作为可解释性分析的事后核心工具，它将模型的抽象决策与轴承故障的物理机理直接关联起来。我们将为每个预测类别中置信度最高的目标域样本生成 Grad-CAM 热力图。通过分析热力图高亮的区域，可以验证模型是否关注了时频图中与该类故障机理相符的视觉模式。更进一步，我们将对比源域和目标域中同类故障样本的

Grad-CAM 图。如果两者的热力图高度相似，都聚焦于相同的物理特征模式，这将构成一个强有力的证据，证明模型学到的诊断知识是普适的，并且这种知识已经成功地、无损地从源域迁移到了目标域。

(3) 故障类型判断的理论依据

首先需要充分认识到故障类型判断的理论依据，然后才能理解判断是否正确。目前有正常状态(Normal / N)、外圈故障(Outer Race/OR)、内圈故障(Inner Race/IR)、滚动体故障(Ball/B)四种类型。

正常状态判断的依据如下：

正常运行的轴承，其内部滚动体与内外圈滚道表面光滑，接触平稳，不存在能够引发剧烈冲击的缺陷。其产生的振动信号主要由旋转频率成分、结构共振和随机噪声组成：

因此，从时域上分析，正常状态的振动信号通常表现为幅值较低、没有明显周期性冲击尖峰的平稳随机信号。其波形可能包含稳定的周期性波动（对应转频），但绝不会出现故障信号中那种尖锐、突变的脉冲。

外圈故障判断的依据如下：

外圈故障点位置固定。当滚动体以恒定时间间隔碾过该点时，会产生周期性且冲击强度近似相等的冲击脉冲。其特征频率为外圈通过频（BPFO）。从时域上分析，轴承外圈故障信号如图 5-23 所示，其冲击幅值稳定，周期恒定。

在 CWT 时频图中，这种等周期、等幅值的冲击表现为一系列等间隔、能量强度近似一致的垂直亮带。每个亮带都具有从低频到高频的宽频特性，是冲击信号的典型特征。

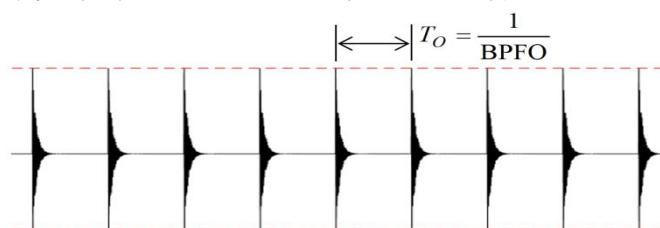


图 5-23 外圈故障示意图

内圈故障判断的依据如下：

内圈故障点随主轴旋转。当故障点转入和转出承载区时，其产生的冲击强度会受到转频(f_r)的周期性调制。其特征频率为内圈通过频率(BPFI)。从时域上分析，内圈故障信号如图 5-24 所示，其冲击周期恒定，但幅值呈现周期性的强弱变化。

这种幅值调制现象在 CWT 时频图中表现为，一系列等间隔的垂直冲击带其能量强度（颜色亮度）会呈现出以转频为周期的强弱变化，形成清晰的能量包络。

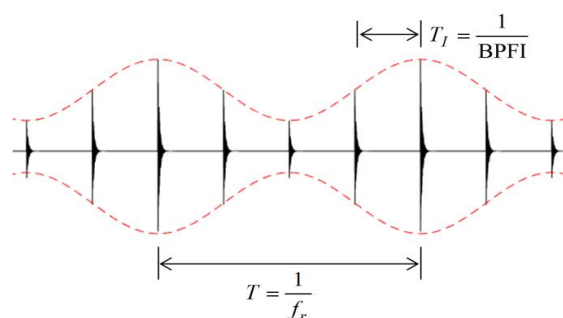


图 5-24 内圈故障示意图

滚动体故障判断的依据如下：

滚动体故障机理最为复杂，它既有自转，又有公转。其冲击信号不仅会受到负载区影

响，还可能与内外圈发生交替碰撞。其特征频率为滚动体自旋频率(BSF)，同时受到公转频率(FTF)的调制。从时域上分析，滚动体故障信号如图 5-25 所示，其冲击模式和调制规律更为复杂。

这种复杂的调制和交替碰撞在 CWT 时频图中呈现为一种复合的动态模式。冲击的周期性（与 BSF 相关）和能量调制（与 FTF 相关）同时存在，但其规律性可能不如内圈故障明显。

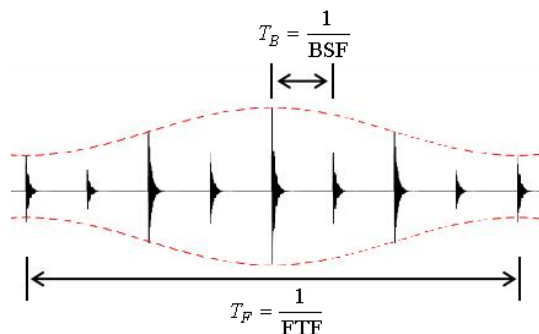


图 5-25 滚动体故障示意图

5.4.3 模型的求解与结果分析

解释训练好的迁移学习模型为什么能成功地将台架数据（源域）学到的知识应用到真实列车数据（目标域），并解释模型对真实列车数据的具体诊断依据是什么。

1. 事前可解释性分析

事前可解释性（Ante-hoc Explainability）关注于模型构建之初的设计哲学，即模型本身应具备透明的结构和符合领域知识的决策逻辑。一个具有强事前可解释性的模型，应该每一步都建立在对问题本质和物理机理的深刻理解之上。在本研究中，从数据表征到模型架构的选择，都贯穿了这一核心思想，旨在构建一个自解释的智能诊断体系。

（1）数据表征的物理可解释性

将一维振动信号转换为二维图像，是本次诊断任务的核心步骤，也是事前可解释性的第一个关键节点。本文选择连续小波变换（CWT）而非其他时频分析方法（如短时傅里叶变换 STFT），其背后有深刻的物理意义和诊断逻辑支撑。

根据题目附件中的故障机理描述，轴承故障的典型特征是当滚动体碾过缺陷时产生的高频、瞬态的冲击脉冲。这些冲击信号在时间上是局部的，但在频率上是宽带的。同时，内圈和滚动体故障还伴随着由转频或公转频率引起的低频调制现象。

- ① **STFT 的局限性：**STFT 使用固定的窗口函数，这意味着其时间和频率分辨率是固定的。若要看清高频冲击的瞬时细节（需要短时间窗），就会牺牲低频分辨率；若要精确分析低频调制成分（需要长时间窗），又会模糊高频冲击的发生时刻。
- ② **CWT 的多分辨率优势：**CWT 采用可变的母小波进行分析，具有多分辨率或变焦特性。它在分析高频信号时自动使用较窄的时间窗，实现高时间分辨率，能够精准定位故障冲击的发生时刻；在分析低频信号时则自动使用较宽的时间窗，实现高频率分辨率，能够清晰地揭示调制现象。这种特性完美契合了轴承故障信号高频冲击与低频调制并存的物理本质。

通过 CWT 生成的二维时频图谱，其本身就是一张信息丰富且人类专家可以理解的“故障指纹图”。图中的模式与特定的故障类型具有直接的对应关系：

- ① 横轴代表时间，纵轴代表频率（或尺度），颜色亮度代表能量强度。
- ② **外圈故障（OR）：**由于外圈固定，冲击强度稳定。在时频图上应表现为一系列等

时间间隔、等能量强度的垂直亮线（冲击能量带）。这个间隔周期理论上等于 $1/BPFO$ 。

- ③ **内圈故障 (IR)**：由于内圈随轴旋转，冲击会周期性地进入和离开负载区，导致冲击强度被转频调制。在时频图上表现为一系列等时间间隔（周期为 $1/BPFI$ ）的垂直亮线，但这些亮线的能量强度（颜色亮度）会呈现出周期性的强弱变化，该变化周期等于转频的倒数 $1/fr$ 。
- ④ **滚动体故障 (B)**：机理更复杂，不仅受负载区影响，还与内外圈交替接触。在时频图上同样表现为周期性冲击（周期为 $1/BSF$ ），但其能量调制规律更为复杂，可能与公转频率 (FTF) 相关。

(2) 模型架构的逻辑可解释性

在选择了具有物理解释性的数据表征后，选用微调的 ResNet-18 作为诊断模型。这一选择同样基于其架构内在的逻辑合理性，即模型结构与任务需求的天然匹配。

CNN 的核心优势在于其能够自动学习数据的层次化特征，这与人类视觉系统的认知过程非常相似。对于 CWT 时频图，ResNet-18 的学习过程可以被自解释为：

浅层卷积层：负责学习基础的、底层的视觉元。在我们的任务中，它学习识别的可能是图像中的边缘、角点、色块。

中层卷积层：负责将浅层学习到的基础特征进行组合，形成更复杂的局部模式。

深层卷积层与全连接层：负责将中层特征进一步组合，形成与特定故障类别高度相关的全局、抽象概念。

相比于普通的 CNN，ResNet-18 的精髓在于其残差连接（Shortcut Connection）。这一结构允许信息（梯度）在网络中跳跃式传播，解决了深度网络训练困难的问题。在诊断任务中，这意味着：

保护原始特征：轴承故障的某些关键特征可能非常简单（例如，某个特定频率的能量异常），在经过多层复杂的非线性变换后可能会丢失。残差连接允许这些原始的、简单的特征直接传递到更深的网络层，确保了关键信息的完整性。

学习精细差异：模型的深度得以保证，使其有足够的能力去学习不同故障之间非常精细和微妙的差异。例如，区分不同损伤程度（0.007 英寸 vs 0.021 英寸）可能需要捕捉到冲击能量带宽或衰减速度的微小变化，这需要一个足够深的网络来建模。

之所以选择 ResNet-18，并非因为它在 ImageNet 竞赛中的名气，而是因为其层次化特征提取的内在机制与本文“从基础冲击到复杂故障模式”的诊断逻辑高度一致。其深度结构和残差设计，使其既能捕捉到故障的宏观周期性，又能不丢失决定性的微观细节。因此，ResNet-18 的架构本身就为“如何从一张时频图中识别故障”提供了一个清晰、分层的、逻辑自洽的解释框架。

综上所述，我们的模型在设计之初就注入了强大的事前可解释性。通过将信号转换为物理意义明确的 CWT 时频图，确保了模型输入的透明性；通过选用与任务逻辑高度匹配的 ResNet-18 架构，确保了模型决策过程的内在合理性。这种设计理念，使得模型不仅是一个高效的分类器，更是一个能够模拟专家诊断逻辑、其决策过程可被理解和信赖的智能系统，为后续的迁移应用和结果信任度奠定了坚实的基础。

2. 迁移过程可解释性分析

迁移过程可解释性的核心在于揭示知识从源域（台架数据）到目标域（列车数据）的传递路径与共享机制，即模型是如何克服领域差异（Domain Shift）并学习到领域不变性（Domain-Invariant）特征的。我们通过 t-SNE 特征可视化和领域分类器混淆矩阵，对 DANN 模型在迁移前后的状态进行深入剖析。

(1) 迁移前特征分布和判别分析

迁移前的状态指的是一个仅在源域数据上训练完成的 ResNet-18 模型，我们直接用它来提取目标域数据的特征，以评估其初始的泛化能力。

按域分布看（如图 5-26 左图所示），左图也清晰地暴露了显著的领域差异。源域（蓝色）和目标域（红色）的样本点虽然有部分交融，但整体上形成了各自独立、边界分明的大片区域。这表明模型在提取特征时，不仅学到了故障信息，还无意中学到了大量与领域相关的信息（如背景噪声水平、传感器响应差异等）。这种特征分布上的差异正是限制模型直接应用于目标域的核心障碍。其中，平均 MMD（最大均值差异）分数为 0.1846。

按类别分布看（如图 5-26 右图所示），即使没有经过任何针对目标域的训练，源域模型已经表现出相当不错的类别区分潜力。目标域的样本（以‘x’标记）在很大程度上已经聚集到了源域样本簇附近，而不是相距甚远，这给我们的迁移学习带来了很好的起始点。例如，黄色的 Normal（正常）目标域样本簇，已经与源域的 Normal 簇相邻。这证实了我们事前分析的结论：CWT 时频图谱结合 ResNet-18 能够学习到具有一定普适性的、与物理机理相关的故障视觉特征。

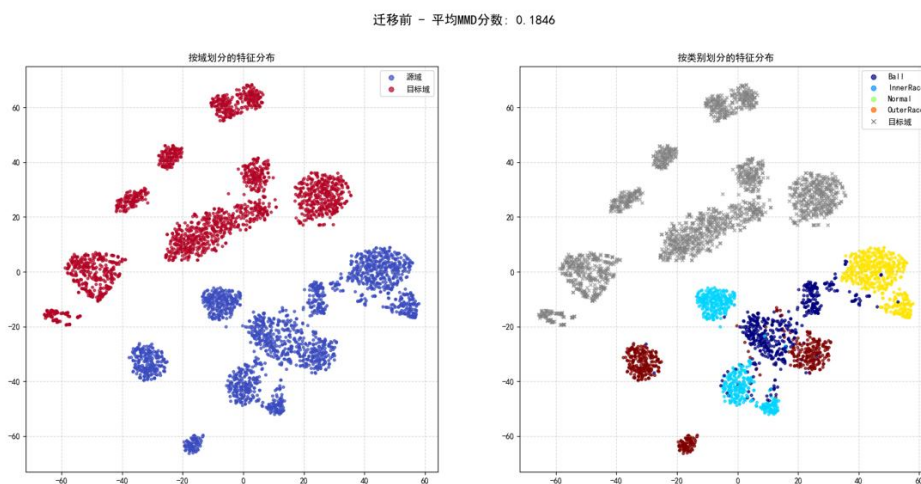


图 5-26 迁移前特征分布降维图

领域分类器的任务是区分一个特征向量来自源域还是目标域。在迁移前，我们观察到一个准确率为 29.59% 的、表现反常的分类器。这个低准确率并非因为领域特征难以区分，而是因为分类器本身存在极大的偏置，几乎将所有样本都预测为目标域。这从侧面反映出，在未经对齐的特征空间里，两个领域的特征是完全可以分的，只是这个简单的分类器未能有效学习。

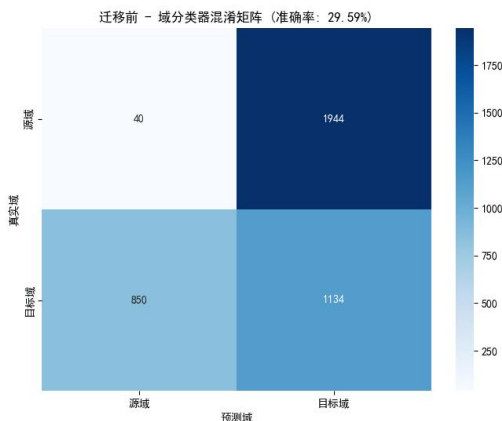


图 5-27 迁移前域分类混淆矩阵图

（2）迁移中特征分布和判别分析

随着对抗训练的推进，我们观测到一场深刻的空间重塑。源域与目标域的特征点开始了从边界渗透到中心融合的持续运动。如图 5-28 所示，在 Epoch 4，两个领域的边界的边界已经开始模糊、交融。到了 Epoch 22 及以后，两个领域的特征点已经实现了高度混合，均匀地交织在一起，在视觉上形成了一个统一的、难以分割的整体。更为关键的是，在领域融合的同时，如图 5-28 所示，各个故障类别的簇非但没有被冲散，反而变得更加紧凑、边界更加清晰。这证明模型在抹平领域差异（求同）的过程中，成功地保留并强化了类别间的核心差异（存异）最终状态：领域融合且类别增强。

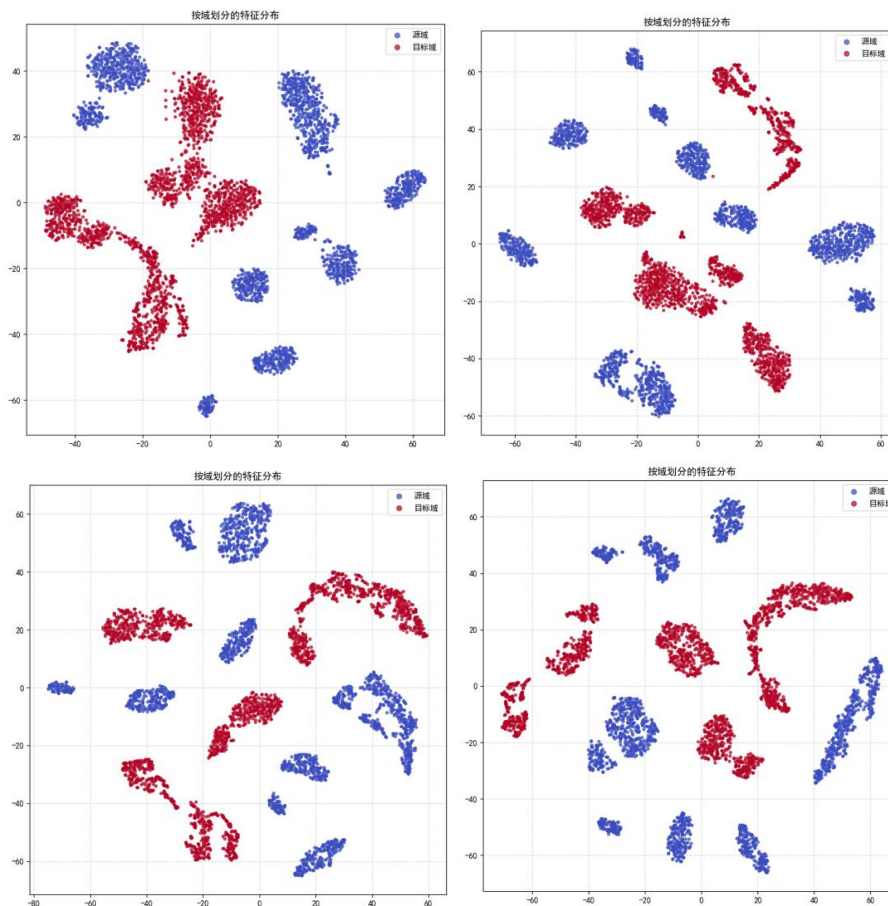
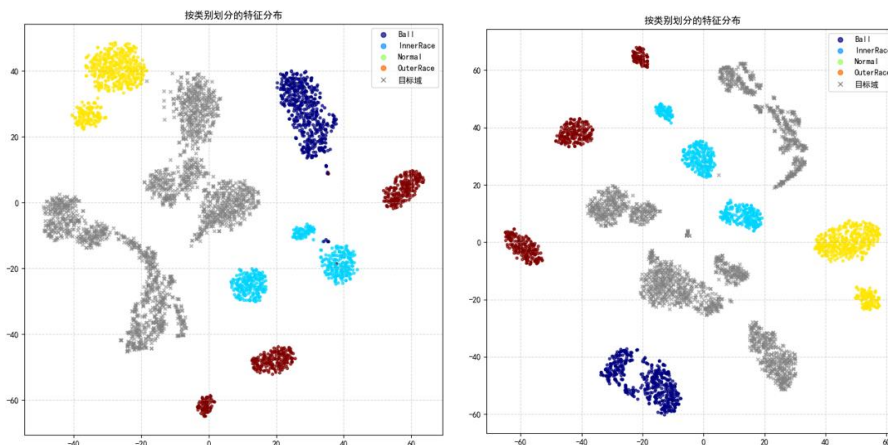


图 5-28 Epoch 1、4、22、29 的域特征分布图



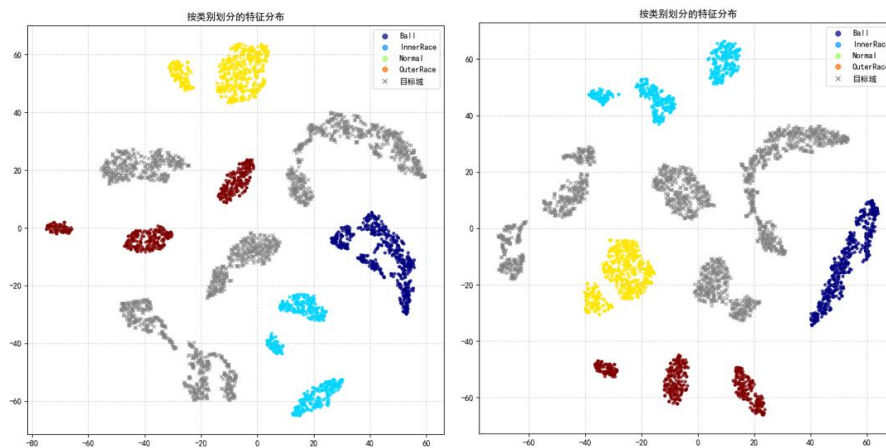


图 5-29 Epoch 1、4、22、29 的类别特征分布图

对抗训练的效果立竿见影。如图 5-29 所示，随着 Epoch 的增加，领域判别器的准确率就断崖式下跌至 46.57%，已经低于随机猜测的水平。这标志着特征提取器在梯度反转层的作用下，已经开始生成令判别器极度困惑的特征。随着训练的深入，判别器的准确率最终稳定在 50%附近。这一数值是 DANN 模型训练成功的理想标志，它意味着领域判别器面对提取出的特征时，其判断与随机投掷硬币无异，表明特征中的领域特异性信息已被最大程度地消除。

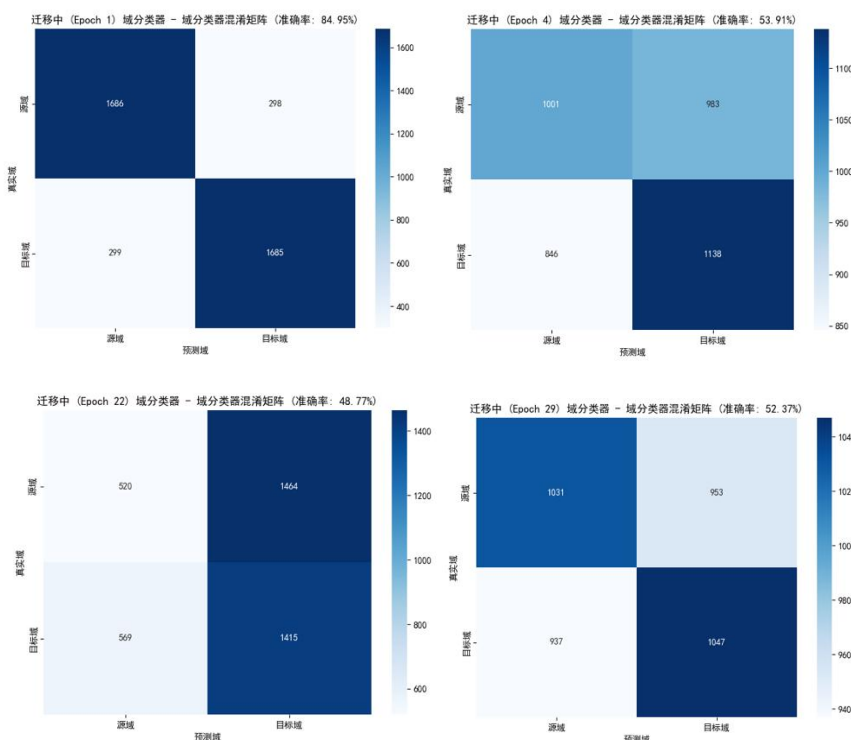


图 5-30 Epoch 1、4、22、29 的域分类混淆矩阵图

(3) 迁移后特征分布和判别分析

DANN 模型的核心机制是通过对抗训练，一方面最大化故障分类的准确性，另一方面最小化领域分类器的准确性。它迫使特征提取器 (ResNet-18) 生成一种欺骗性的特征——既能清晰地区分故障类别，又让领域分类器无法判断其来源。

迁移学习结束后，观察特征分布，发现：

从域分布来看（如图 5-31 左图所示），迁移后的效果是显著的，源域（蓝色）和目标域（红色）的样本点已经高度混合、深度交融，几乎无法直接划分出两个领域的空间，而且两个领域的距离也非常近。这直观地证明了 DANN 成功地拉近了两个领域在特征空间的距离，构建了一个统一的、领域不变的特征表示。MMD 分数也从 0.1846 下降至 0.1436，定量地证实了领域分布的对齐。

从类别分布来看（如图 5-31 右图所示），在实现领域对齐的同时，各类别的内部聚合性不仅没有被破坏，反而得到了增强。各类故障（Ball, Inner Race, Normal, Outer Race）的簇边界变得更加清晰，目标域的‘x’样本点更加接近了对应类别的簇中心。这说明模型在抹平领域差异的过程中，保留并强化了凸显类别差异的能力，确实实现了我们想要的求同存异的目的。

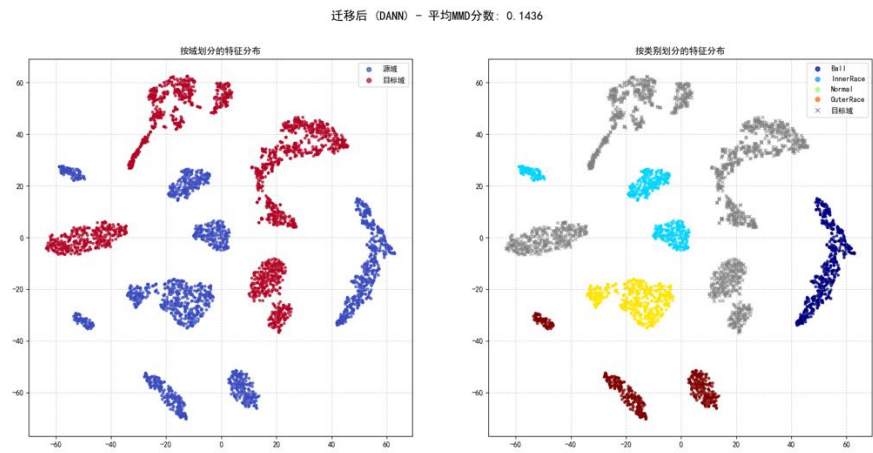


图 5-31 迁移后特征分布降维图

迁移后，领域分类器的准确率变为了 51.03%。对于一个二分类问题，这个准确率约等于随机猜测。混淆矩阵也显示，无论真实域是源域还是目标域，分类器的预测结果都接近于各占一半。这说明特征提取器已经能够生成令领域分类器完全困惑的特征，表明这些特征已经成功地剥离了领域特有的信息，实现了领域不变性。

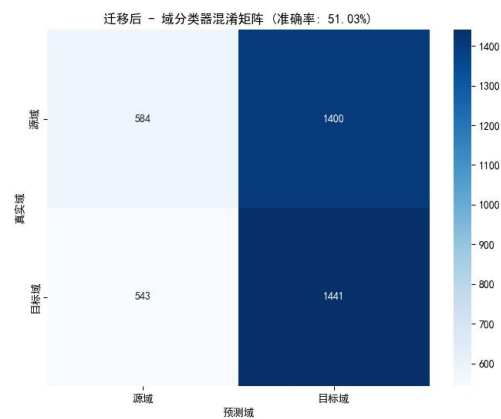


图 5-32 迁移前域分类混淆矩阵图

通过对比分析，我们可以清晰地看到 DANN 模型的迁移过程：它并非从零开始，而是在一个已经具备良好泛化基础的 ResNet-18 模型上，通过精准的去伪存真操作，主动消除了特征中的领域特异性伪影，同时提纯和强化了与故障本质相关的核心特征。t-SNE 图从“领域分明、类别可分”到“领域融合、类别更优”的演变，以及领域分类器从“轻易判

别”到“随机猜测”的转变，共同构成了 DANN 模型成功实现知识迁移的完整证据链，为模型在目标域上的高精度诊断能力提供了强有力的过程可解释性。

3.事后可解释性分析

本节旨在通过事后可解释性分析，验证模型决策与物理机理的一致性，并确认该决策逻辑在跨领域迁移中的有效性。为此，我们设计了如下分析策略：首先，从目标域的预测结果中，选取了代表四种工况（正常、外圈故障、内圈故障、滚动体故障）且模型预测置信度最高的四个样本（G, K, N, A），并从源域数据集中匹配了对应的故障类型样本。其次，通过视觉观察识别了不同工况在 CWT 时频图上的核心模式，然后利用梯度加权类激活图（Grad-CAM）技术，深入模型内部，对 ResNet-18 特征提取器的全部四个残差块 layer1,layer2,layer3,layer4）进行逐层可视化分析，验证其决策依据是否与这些物理模式相符，并检验该决策逻辑在从源域向目标域迁移过程中的一致性。

（1）正常状态的视觉特征分析

时频图分析

源域样本的时频图的显著特征是完全没有上述三种故障所特有的、位于高频区域的周期性垂直冲击带。图像能量主要集中在几条稳定的、贯穿时间轴的水平谱线上，这代表了设备正常运转时的转频及其谐波。

目标域样本 G 也呈现了相同的核心特征——高频区缺乏周期性冲击。尽管由于实际工况影响，其背景噪声更强，中低频区域的能量分布更为复杂，但关键的故障指纹是缺席的。图像中没有能够构成威胁的、规律性的瞬时冲击能量。

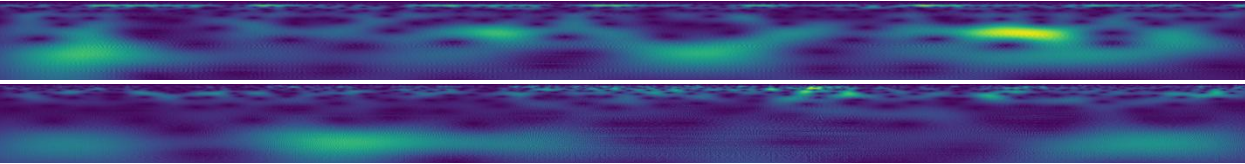
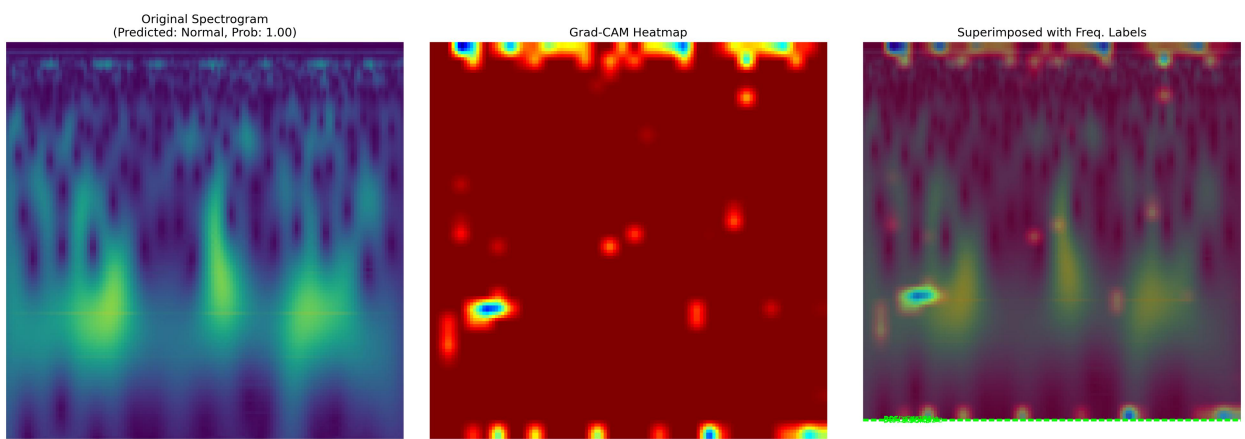


图 5-33 源域样本 Normal_data_N_0 和目标域样本 G 切片

Resnet-18 第 1 层分析

在被判定为正常的源域样本中，Grad-CAM 热力图显示其激活点稀疏且散乱，没有形成任何有规律的结构或模式。这些零星的激活点似乎对应于一些随机的能量波动。目标域的正常样本呈现了完全相同的现象：激活点稀疏、随机、不成规律。



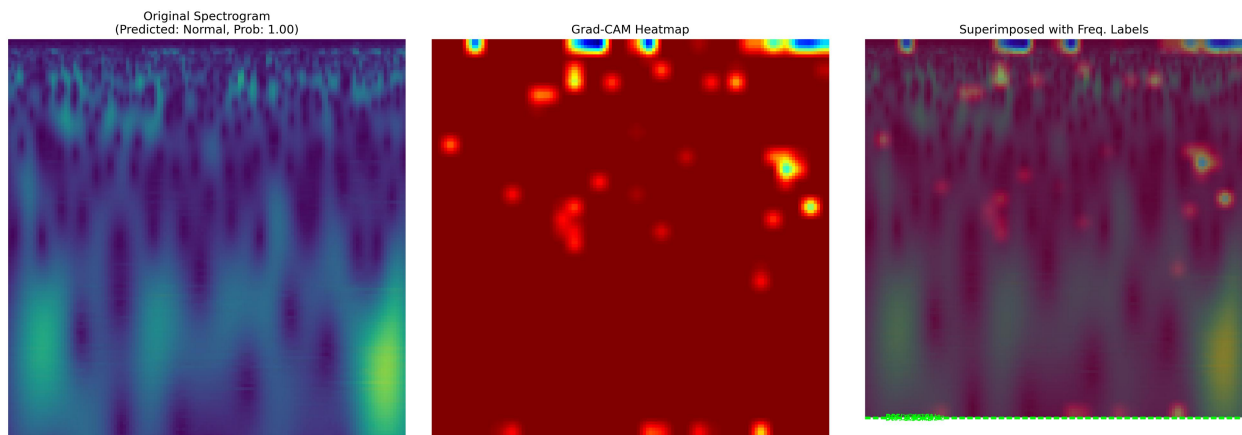


图 5-34 源域样本 Normal_data_N_2 和目标域样本 G layer1 输出热力图

Resnet-18 第 2 层分析

对于正常状态的样本，layer2 的激活模式延续了 layer1 的特征。激活点依旧稀疏、无规律且缺乏结构性关联。模型没有尝试将这些零星的激活点组合成任何有意义的模式。这再次验证了模型通过确认“不存在任何可组合的、有规律的故障模式”来判定正常状态的决策机制。这一排除法逻辑在源域和目标域中均保持稳定。

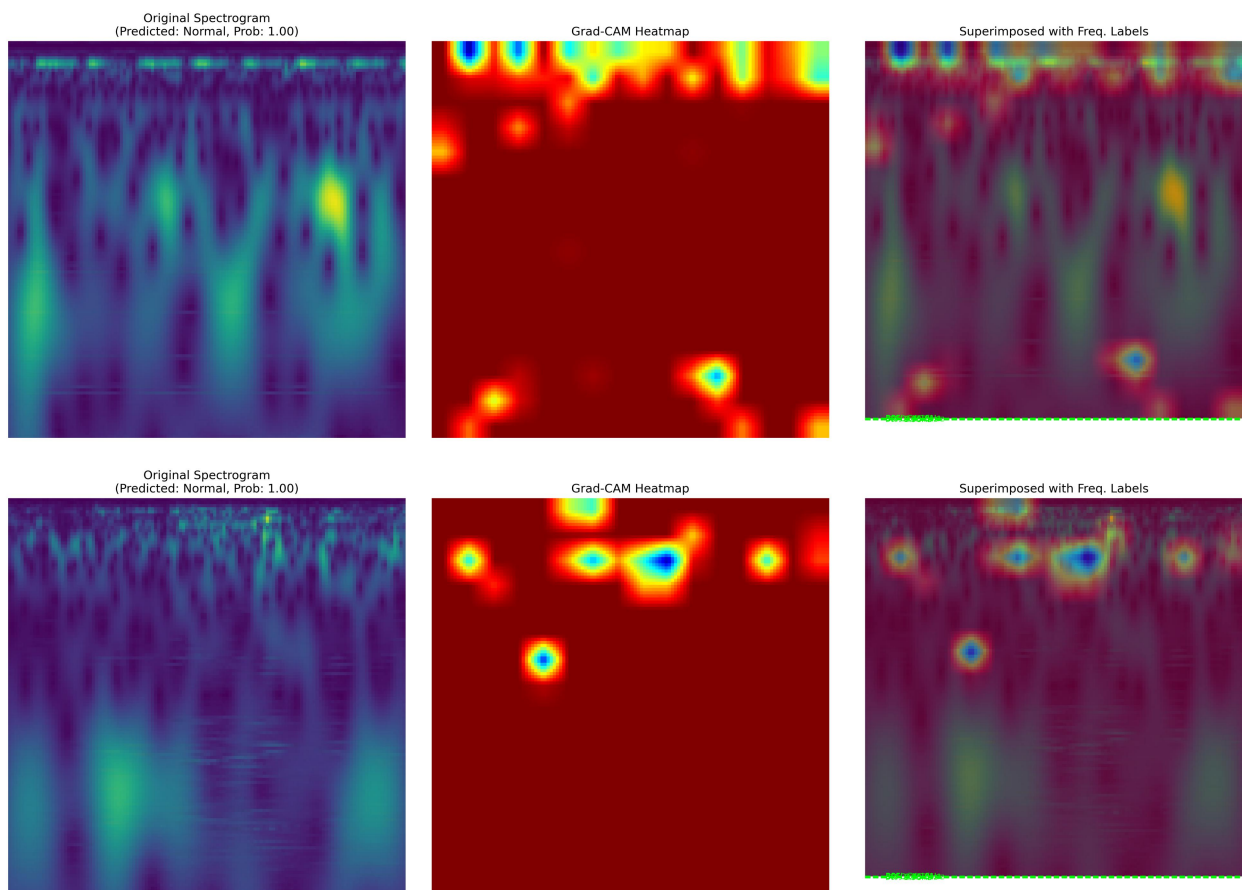


图 5-35 源域样本 Normal_data_N_0 和目标域样本 G layer2 输出热力图

Resnet-18 第 3 层分析

对于正常状态样本，layer3 的激活区域显著收缩，仅在高频区域的边缘留下一些激活

痕迹。与所有故障类型相比，正常状态在 layer3 的激活强度和范围都是最弱的。这表明，在经过 layer1 和 layer2 的检查后，模型在 layer3 进行最终确认时，在关键的故障特征区域内未能形成任何强有力的、全局性的故障模式抽象。这种全局模式的缺失，构成了判定其为正常的最终依据。

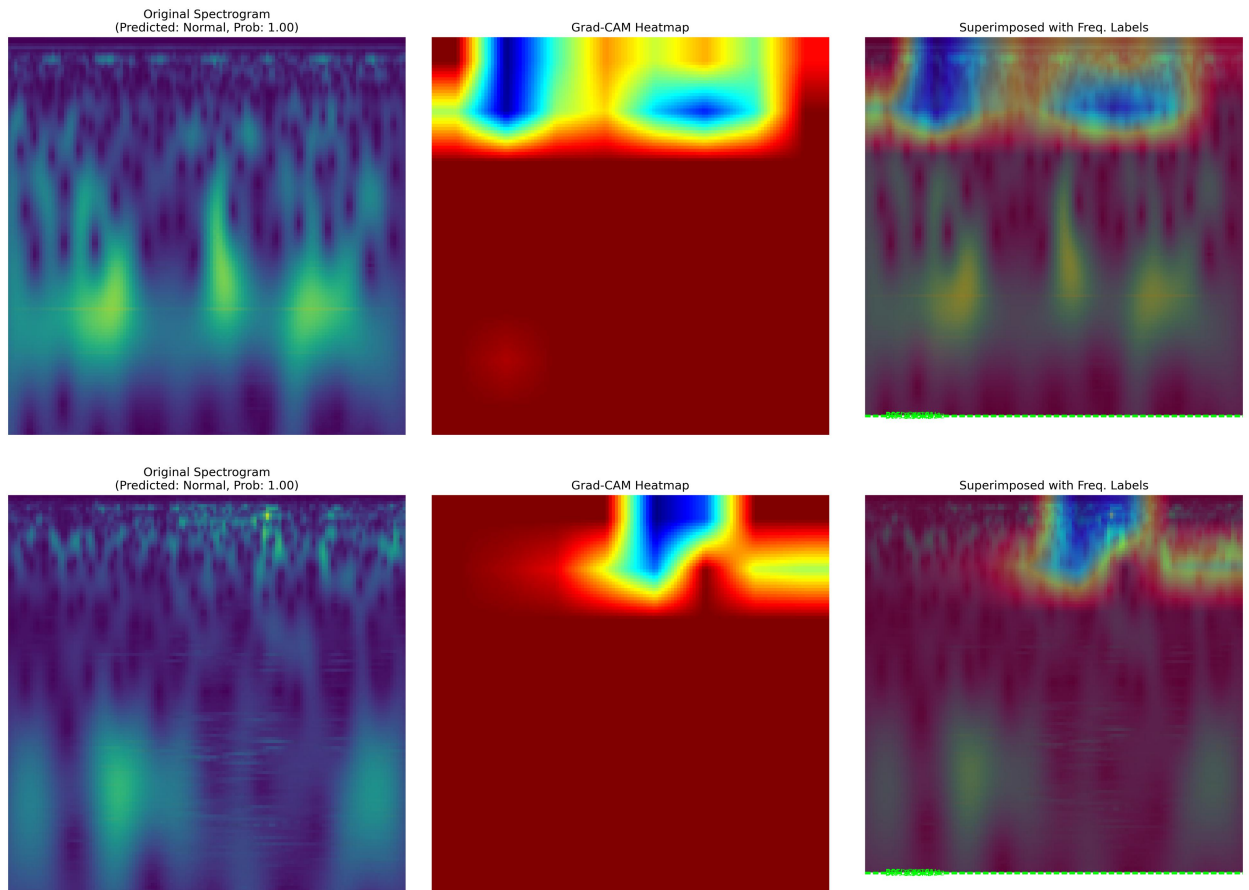
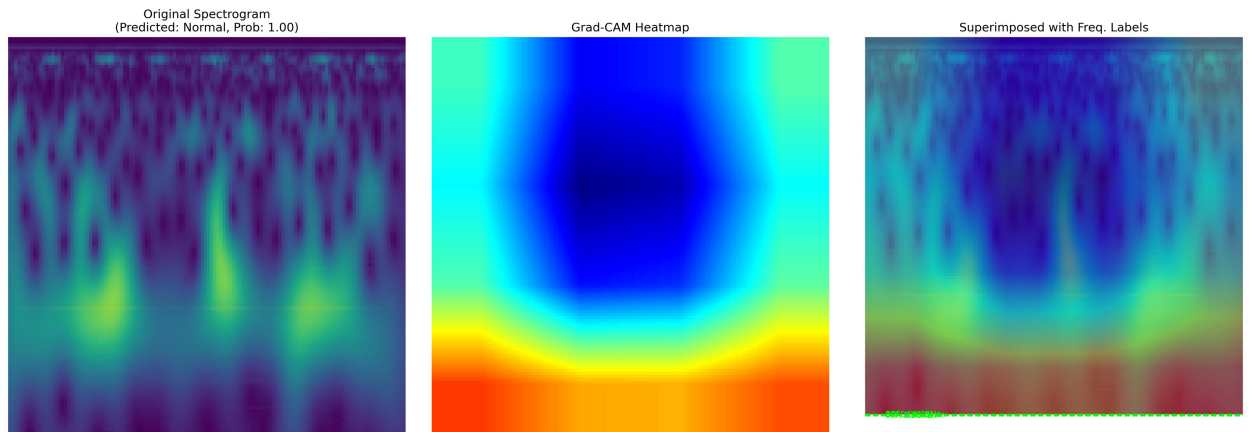


图 5-36 源域样本 Normal_data_N_2 和目标域样本 G layer3 输出热力图

Resnet-18 第 4 层分析

与所有故障类型形成对比，正常状态在 layer4 的激活模式是独特的。其热力图同样呈现出中央区域的负激活，但其正激活区域主要集中在时频图的底部（低频）的边缘。



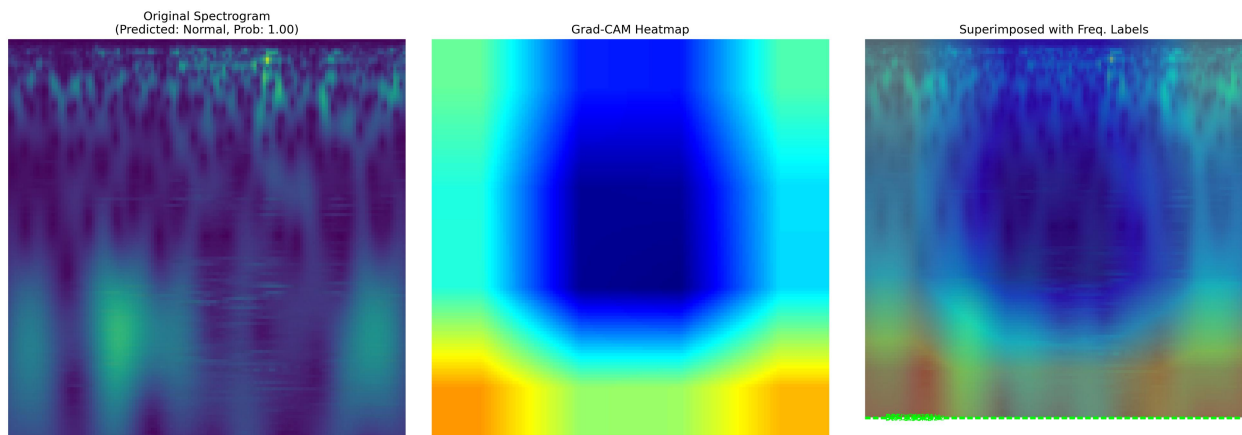


图 5-37 源域样本 Normal_data_N_2 和目标域样本 G layer4 输出热力图

(2) 外圈故障的视觉特征分析

在源域样本中，尽管背景噪声比较强，信号的整体清晰度有所下降，但核心的故障模式依然清晰可辨。在图像的高频区域，一系列周期性出现的亮点构成了图像最显著的特征。虽然亮点的形态不如目标域样本那般锐利，但其严格的周期性和相对稳定的能量这一关键信息被完整地保留了下来。

在目标域样本 K 的时频图中，我们可以观察到清晰的故障特征：在图像上方的高频区域，存在一系列亮度极高、时间间隔严格相等的冲击能量点。这些冲击点的能量强度（亮度）基本一致，反映了外圈故障的物理特性。

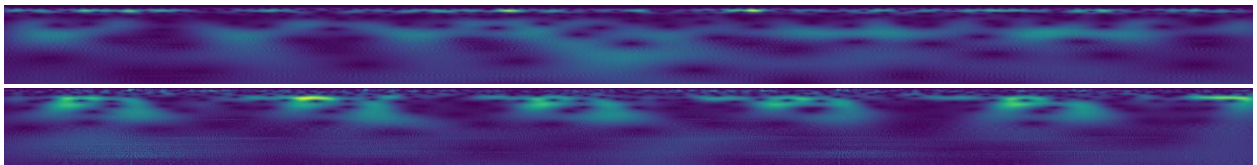


图 5-38 源域样本 0014_OR014@6_2 和目标域样本 K 切片

Resnet-18 第 1 层分析

接着我们观察 Resnet-18 第一层的输出，如图所示，对于源域的外圈故障样本，模型的激活区域（Heatmap 中的亮色区域）呈现为一系列离散、清晰的热点。这些热点精确地与原始时频图中高频区域的周期性冲击脉冲的位置一一对应。这表明模型在初期就学会了将独立的、高能量的瞬态冲击作为故障的基本特征单元。

在信噪比更低的目标域样本中，原始时-频图的冲击特征有所展宽和模糊。相应地，模型的激活模式从离散的热点演变为连续的、垂直的条带。尽管激活形态发生变化，但其核心逻辑并未改变：注意力仍然严格地锁定在由外圈故障（BPFO）引起的周期性冲击结构上。这证明模型成功地将识别周期性冲击这一核心知识，从识别清晰脉冲泛化到了识别模糊条带，表现出良好的领域适应性。

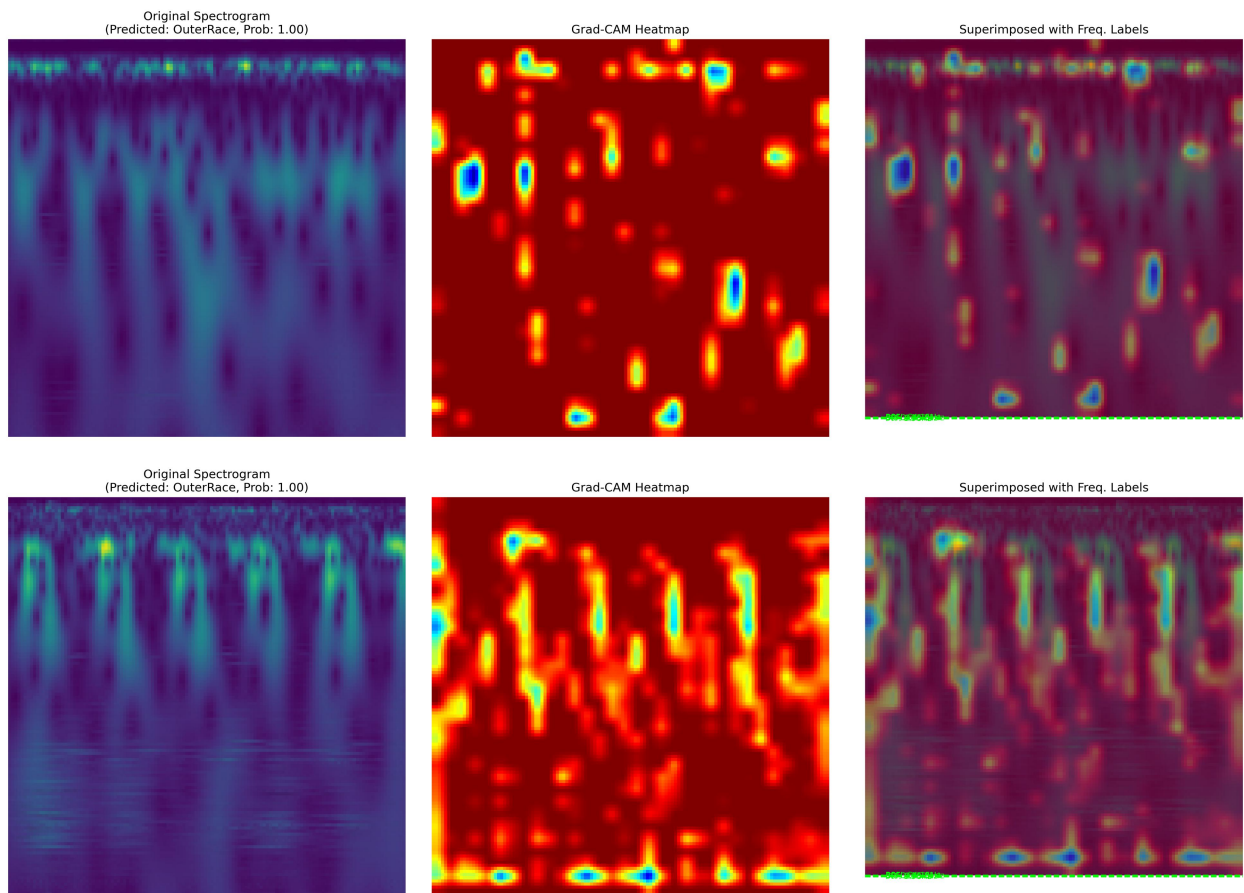
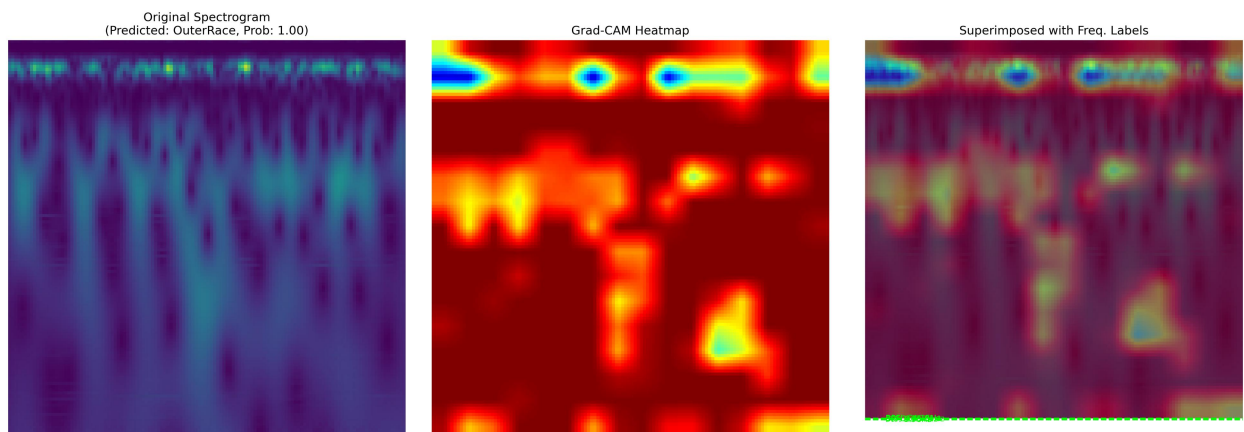


图 5-39 源域样本 0014_OR014@6_2 和目标域样本 K layer2 输出热力图

Resnet-18 第 2 层分析

在 layer2 中，layer1 所识别出的离散冲击热点开始被组合。对于源域和目标域样本，模型的激活区域均从孤立的点演变为沿时间轴方向的、具有一定连续性的垂直结构。这表明模型已不再关注单个冲击事件，而是开始学习识别周期性冲击序列这一更宏观的模式。此模式的建立是区分外圈故障的关键步骤。



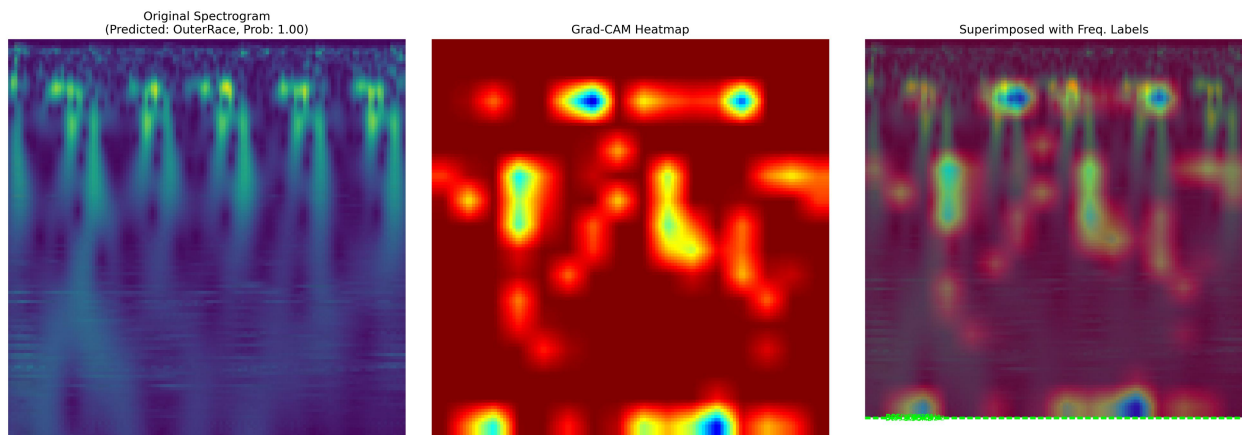


图 5-40 源域样本 Centered_0014_OR014@6_2 和目标域样本 K layer2 输出热力图

Resnet-18 第 3 层分析

在 layer3 中，对于源域和目标域的外圈故障样本，模型的激活区域都演变为一个覆盖整个高频区域的、宽阔平滑的水平带。这表明模型已经将 layer2 识别出的“周期性冲击序列”抽象为一个更高层次的语义概念：“存在于高频区域的、持续且稳定的周期性现象”模型的注意力从局部细节完全转移到了对整个时间窗内模式一致性的全局确认上。

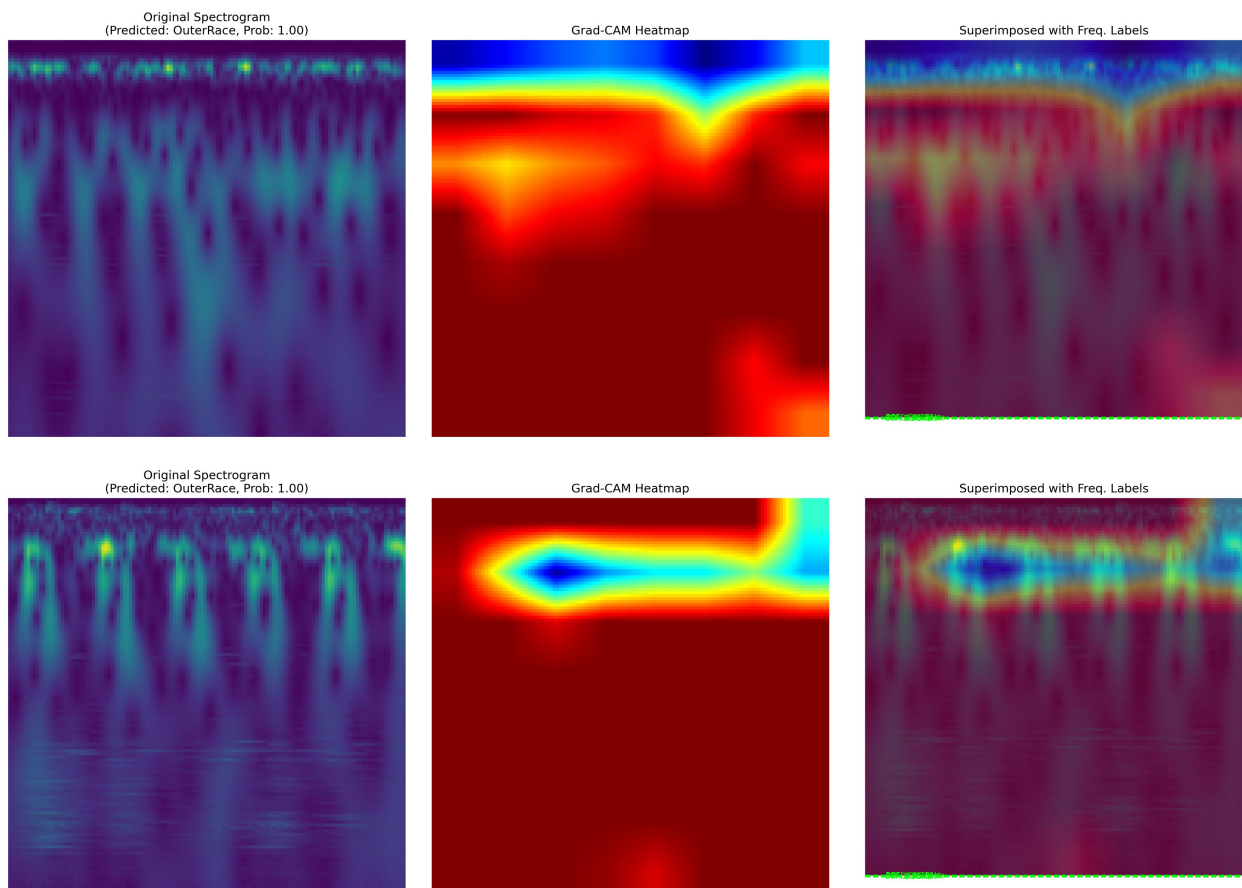


图 5-41 源域样本 Centered_0014_OR014@6_2 和目标域样本 K layer3 输出热力图

Resnet-18

第 4 层分析

对于源域和目标域中的所有故障样本，其 Grad-CAM 热力图均呈现为一个中央区域为

强负激活（蓝色）、而外围区域为正激活（黄、红色）的环状或碗状结构。在 layer4 中，所有三种故障类型（外圈、内圈、滚动体）的激活模式趋于统一。

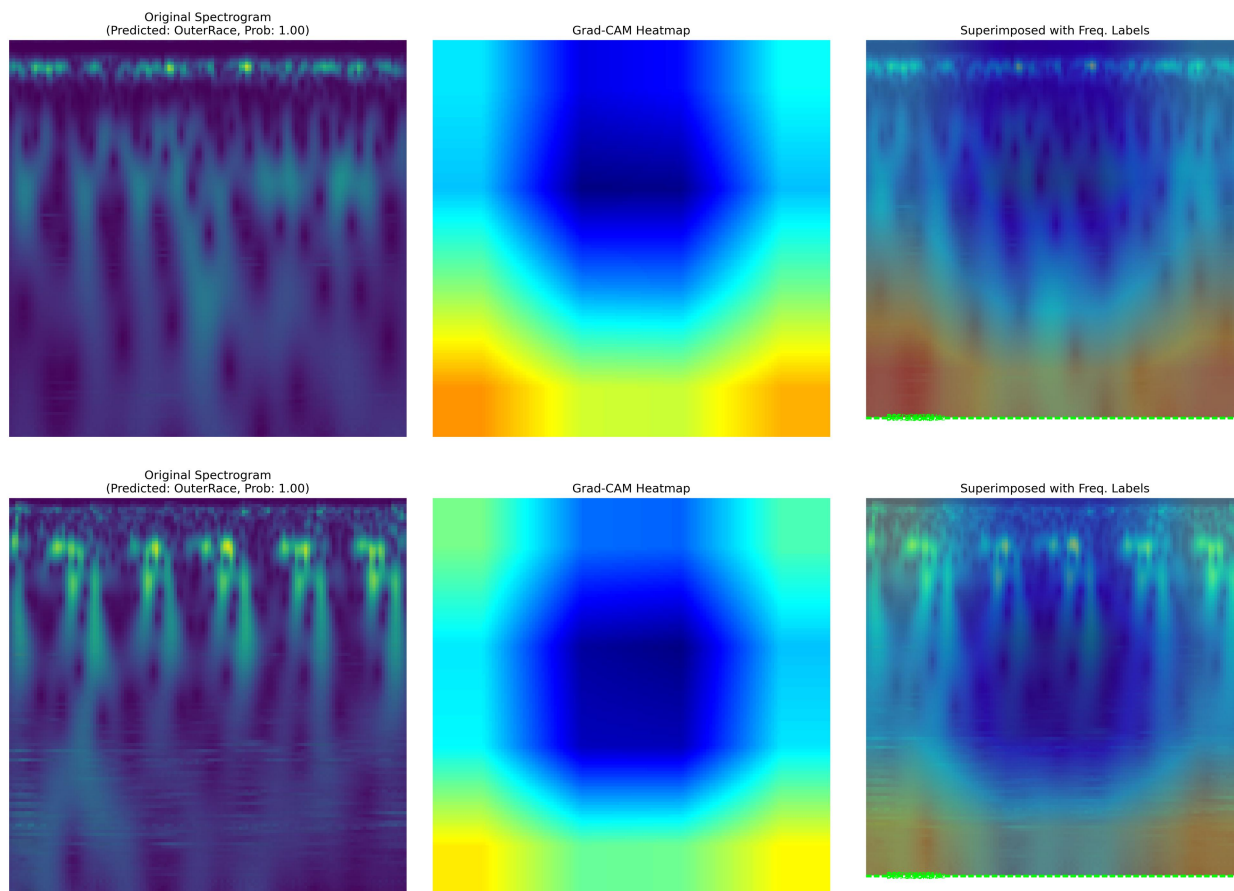


图 5-42 源域样本 Centered_0014_OR014@6_2 和目标域样本 K laye4 输出热力图

（3）内圈故障的视觉特征分析

源域样本的时频图展现了经典的调制现象：虽然高频区域同样存在周期性冲击，但其能量强度（亮度）呈现出明显的强-弱-强的周期性变化。这些冲击点不再是能量一致的，而是形成了若干个能量包络，直观地展示了转频调制的物理过程。

目标域样本 N 也完美复现了这一核心特征。我们可以清晰地看到，其高频区域的周期性冲击能量并非恒定，而是有节奏地起伏变化。例如，图中左侧的冲击点能量较强，向右逐渐减弱，随后再次增强。这种能量的节律性波动是区别于外圈故障的关键“指纹”。

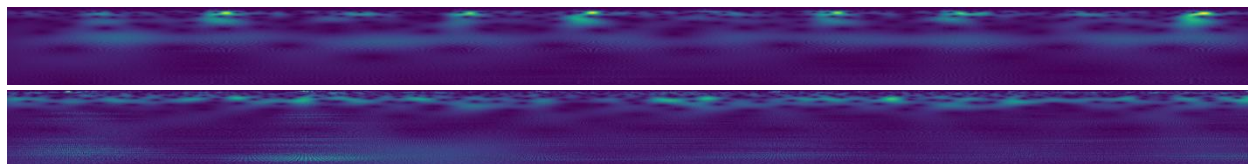


图 5-43 源域样本 Centered_0007_IR007_1 和目标域样本 N 切片

Resnet-18 第 1 层分析

对于源域的内圈故障样本，模型的激活热点同样是离散且精准的。关键在于，这些激活点主要集中在由转频调制的、能量最强的几次冲击脉冲上。这说明模型不仅识别了冲击的周期性，还对冲击的能量包络（即调制现象）具备了初步的敏感性。

在目标域的内圈故障样本上，存在几个零星的激活点，可能在该样本上的激活效果欠佳。但是也观察到了模型识别了部分垂直方向上的冲击。

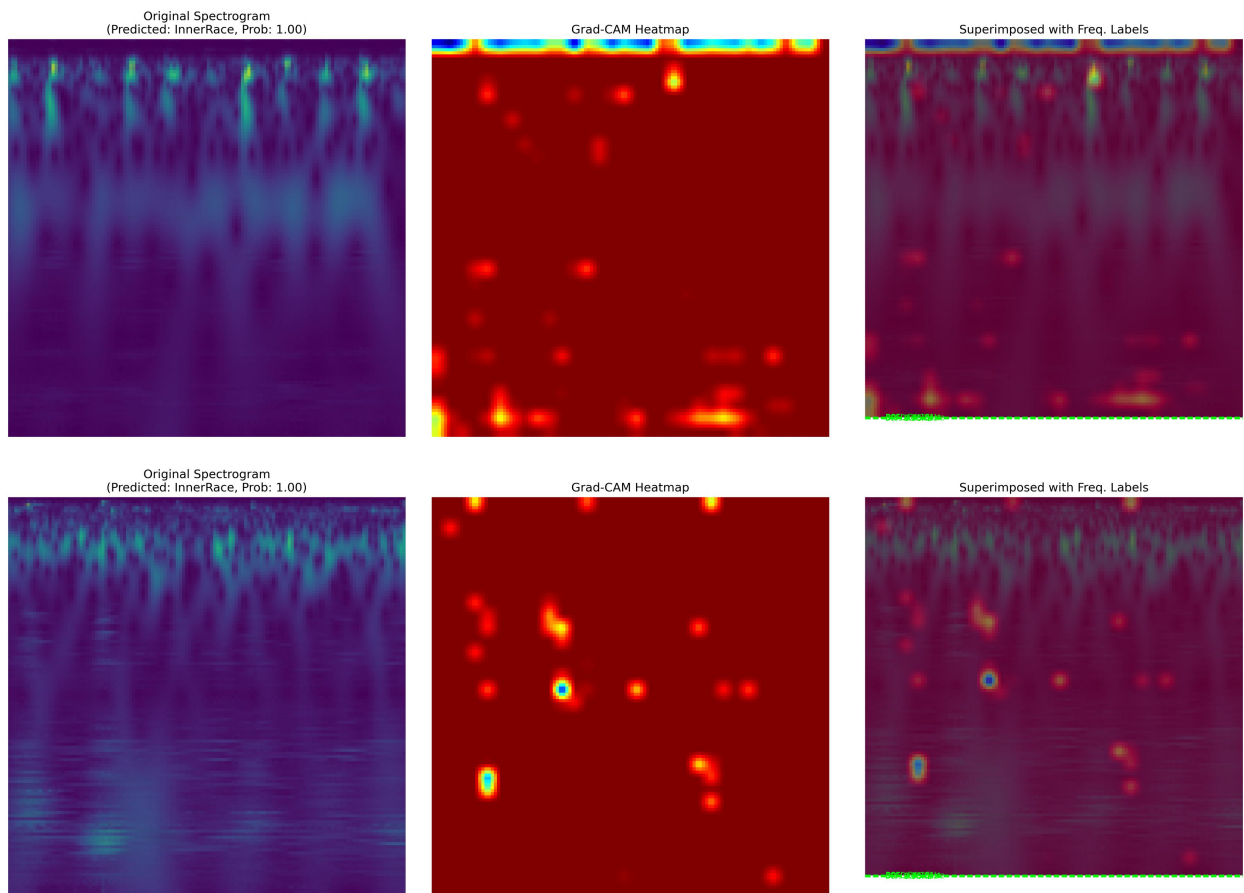
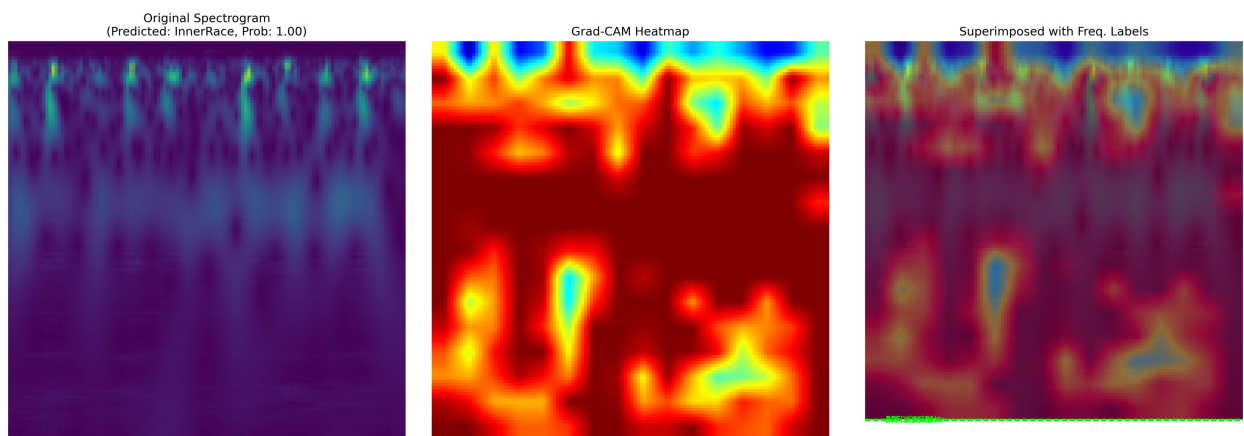


图 5-44 源域样本 0007_IR007_1 和目标域样本 N_layer1 输出热力图

Resnet-18 第 2 层分析

对于内圈故障，**layer2** 的激活模式显示出对整体高频区域的关注。相较于 **layer1** 中对高能量冲击点的精确聚焦，**layer2** 的激活区域在水平方向上（频率轴）有所扩展，开始连接相邻的冲击事件。这表明模型正在学习识别由转频调制产生的能量包络或带状特征。这种从点到带的关注点转移，是模型理解调制现象的体现，且该逻辑在源域和目标域中表现一致。



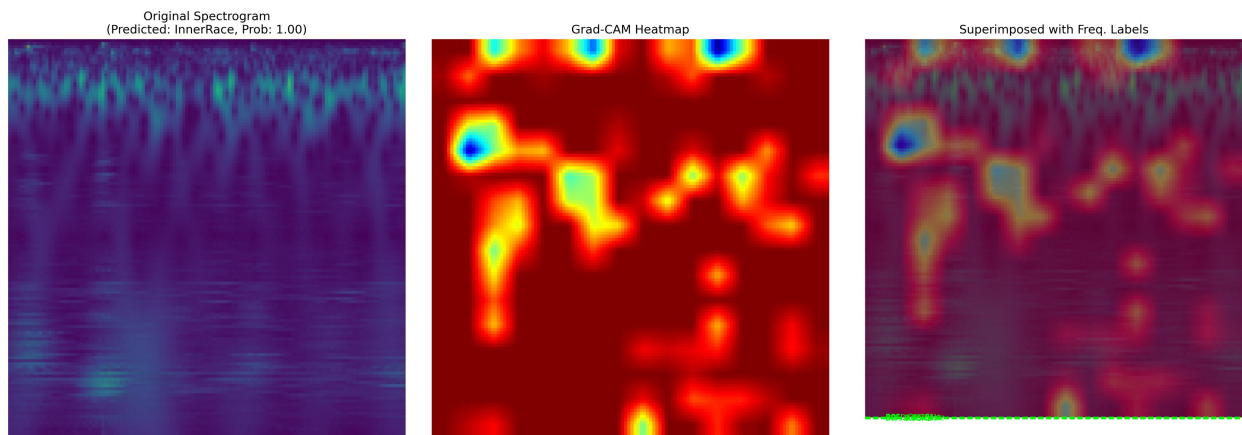


图 5-45 源域样本 0007_IR007_1 和目标域样本 N layer2 输出热力图

Resnet-18 第 3 层分析

与外圈故障相似，内圈故障在 layer3 的激活模式同样是一个横跨高频区域的、宽阔的水平带。这说明模型已经学习到，内外圈故障的关键判别信息都存在于这个全局的高频区域内。虽然热力图在宏观上相似，但这表明模型已经确认了这是一个存在周期性冲击的样本，而具体的区分（稳定 vs 调制）则依赖于这一层特征通道内部对 layer2 传入的能量包络模式的进一步处理。模型关注的是同一个问题区域，并在该区域内应用不同的细分标准。

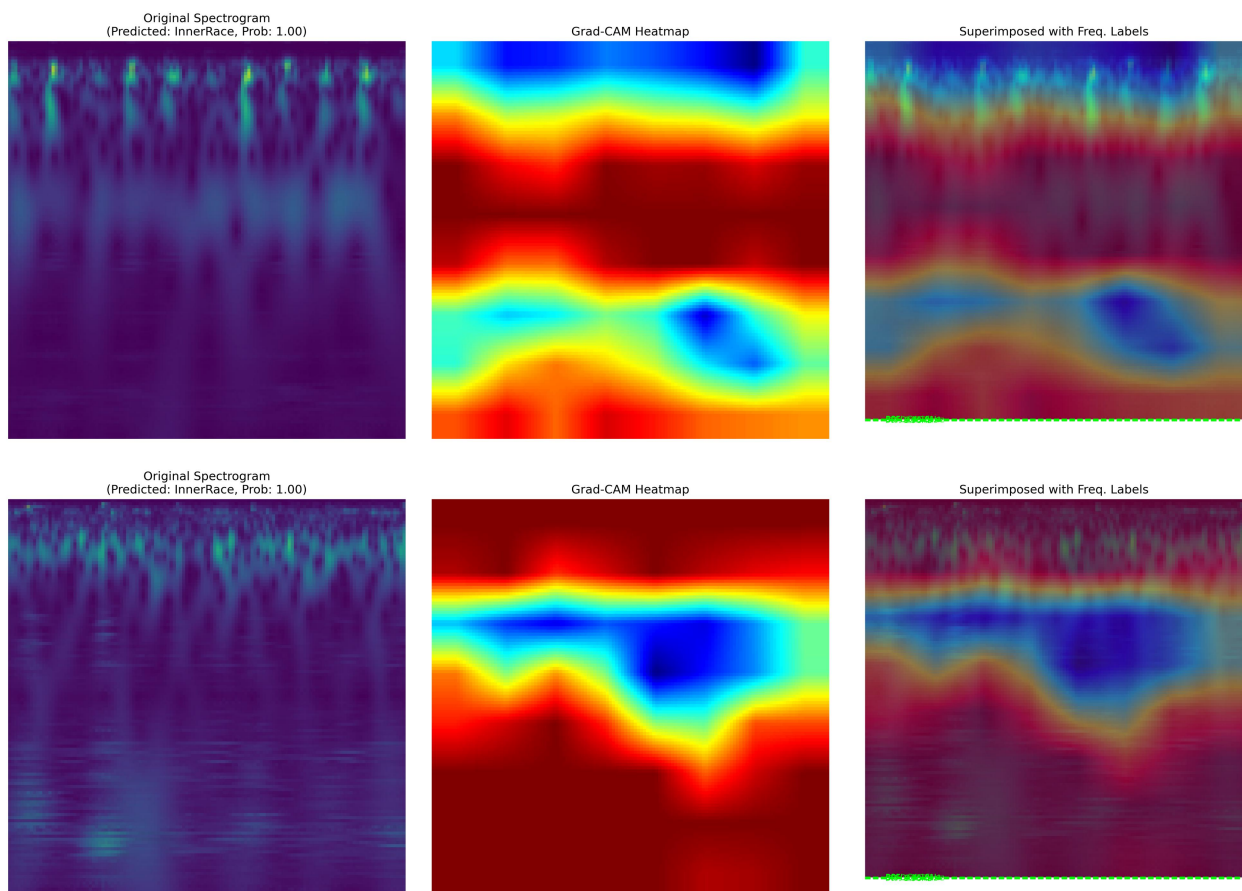


图 5-46 源域样本 0007_IR007_1 和目标域样本 N layer3 输出热力图

Resnet-18 第 4 层分析

同外圈故障类似，中央区域为强负激活（蓝色）、而外围区域为正激活（黄、红色）

的环状或碗状结构。

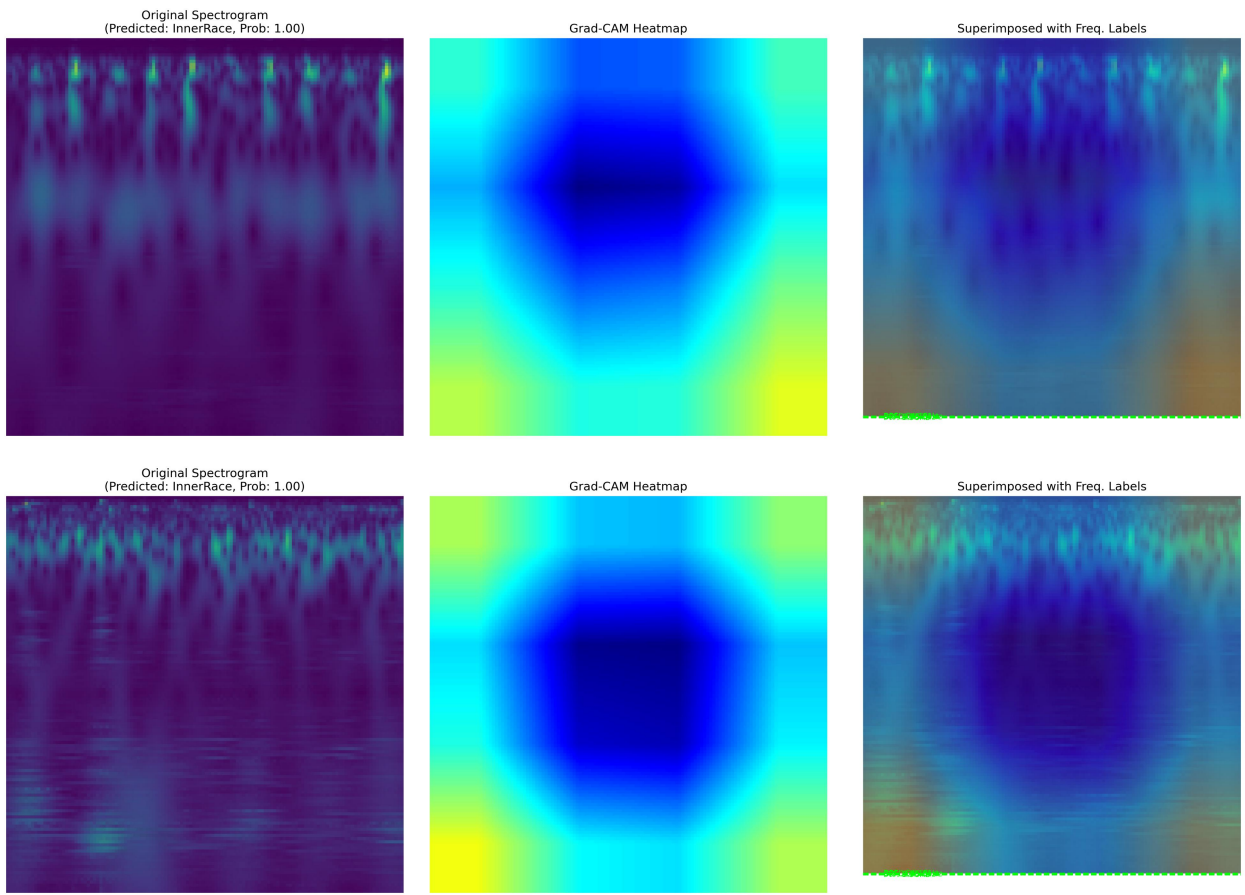


图 5-47 源域样本 0007_IR007_1 和目标域样本 N laye4 输出热力图

（4）滚动体故障 (Ball) 的视觉特征分析

源域样本的特征介于内外圈故障之间，但模式更为复杂。图中高频区域存在周期性冲击，但其能量调制规律不如内圈故障那样清晰。同时，在中低频区域似乎也存在一些能量的周期性聚集，可能与公转频率有关。

目标域样本 A 同样展现了这种复杂性。高频区域存在能量集中的冲击点，但其周期性和能量调制的规律性均不如 OR 和 IR 样本明显。图像整体的能量分布显得更为散乱，但在高频区仍可识别出重复出现的、但形态各异的冲击模式。

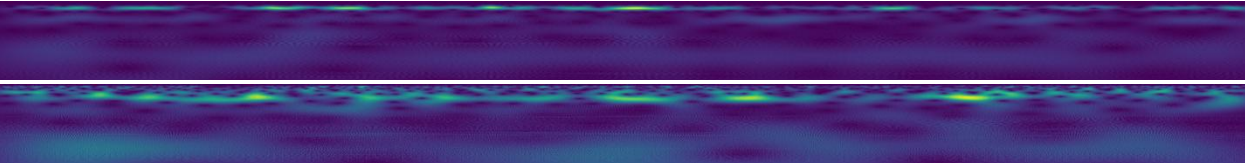


图 5-48 源域样本 B_0021_B021_1 和目标域样本 A 切片

Resnet-18 第 1 层分析

与内外圈故障不同，对于源域和目标域的滚动体故障样本，layer1 的激活区域显得更为广泛和密集。除了高频冲击点外，模型还对更广阔的时频区域产生了兴趣。这与滚动体故障机理更复杂、信号表现更多样的特性相符。模型似乎在捕捉一种区域性的能量扰动，而不仅仅是离散的冲击点。

这一区域激活策略在目标域样本中得到了一致的体现。模型同样将注意力放在更广阔的时频区域能量上，而非某个特定的冲击时刻。这证明模型为滚动体故障学习到了一套独

特的、基于能量密度分布的诊断标准，并且该标准成功地应用于目标域数据。

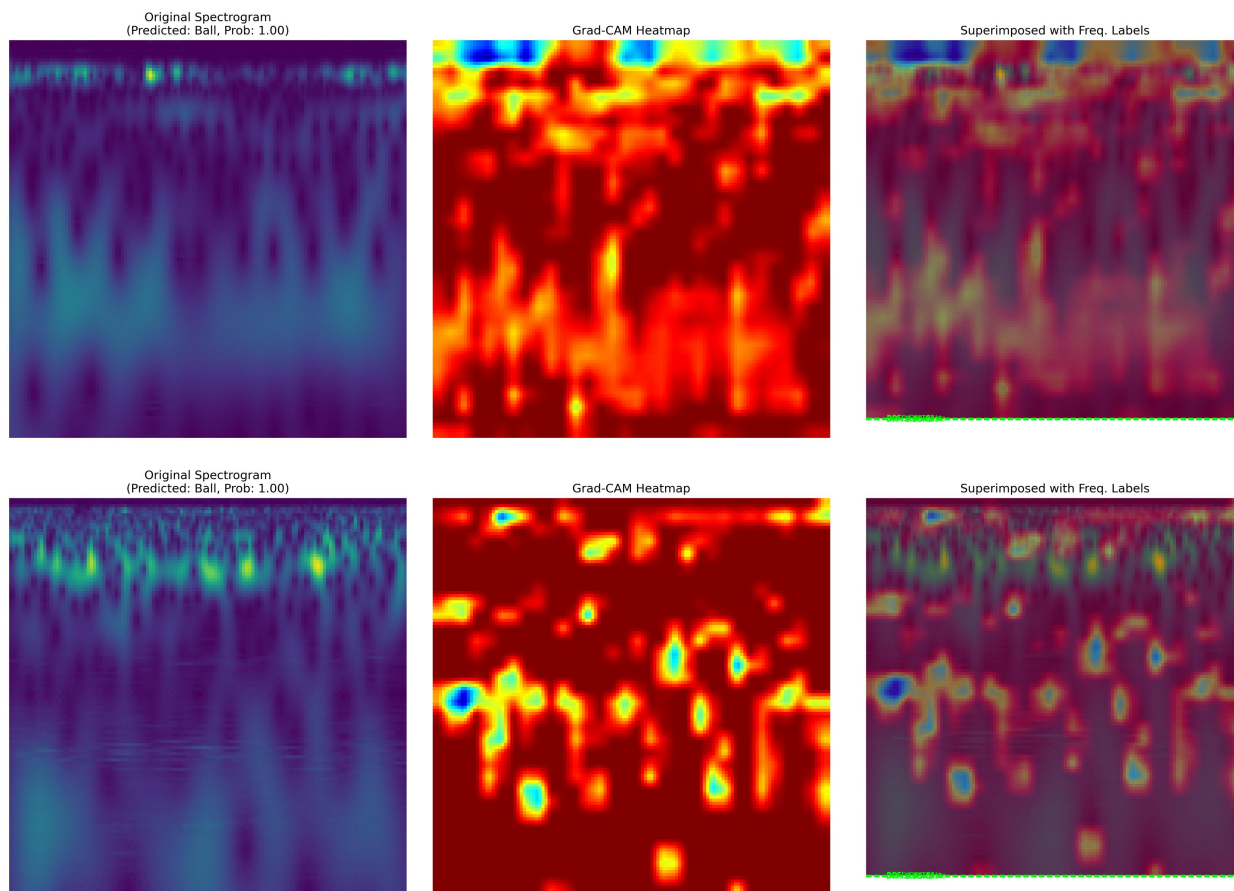
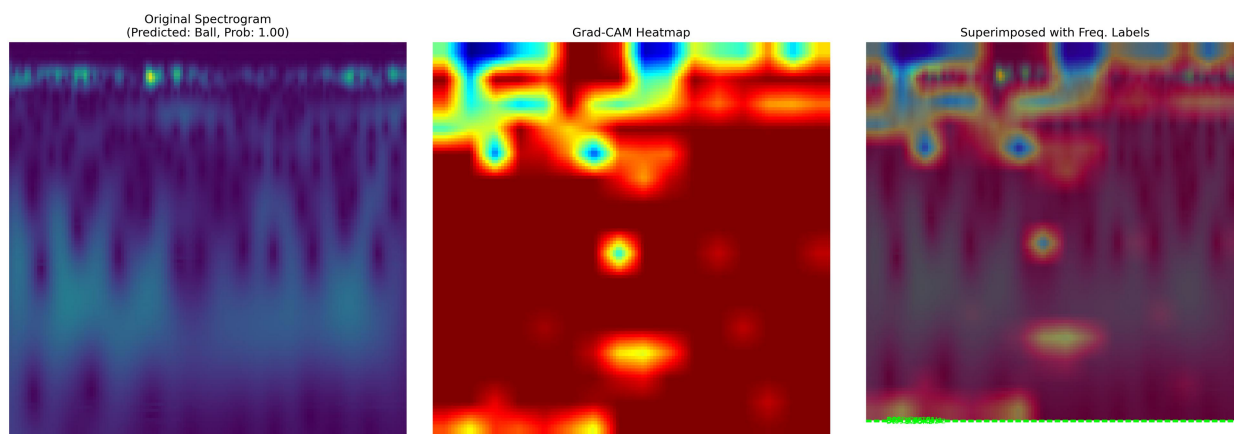


图 5-49 源域样本 B_0014_B014_3 和目标域样本 A layer1 输出热力图

Resnet-18 第 2 层分析

layer1 中对滚动体故障所呈现的区域激活策略在 layer2 中得到进一步细化。原本较为弥散的激活区域开始收敛，形成数个能量更为集中的、不规则的块状结构。这说明模型在确认了高频区域存在普遍的能量扰动后，开始尝试定位其中能量最显著、最具代表性的部分。这种对不规则能量聚集区的识别，构成了滚动体故障独特的诊断依据，并在两个领域中都得到了应用。



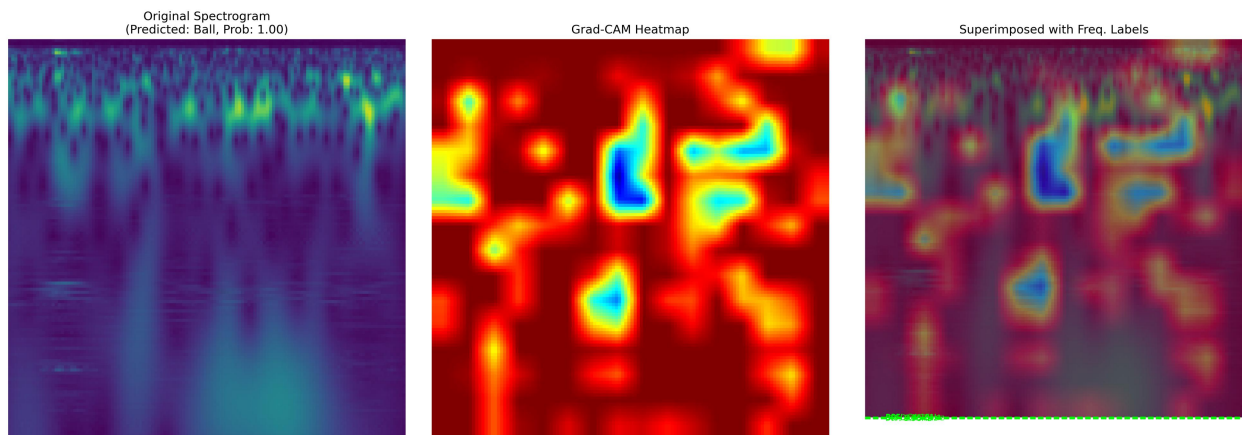


图 5-50 源域样本 B_0014_B014_3 和目标域样本 A laye2 输出热力图

Resnet-18 第 3 层分析

layer3 对滚动体故障的激活模式与其他类型截然不同。其激活区域不再局限于高频带，而是扩展为一个占据了时频图大部分区域的、不规则的大面积云团。这揭示了模型对滚动体故障的最终理解：“一个在时域和频域上均表现出广泛、不规则能量分布的非稳定状态”。这种对全局结构化混沌的识别能力，是模型区分滚动体故障的关键，且该识别逻辑在源域和目标域中高度一致。

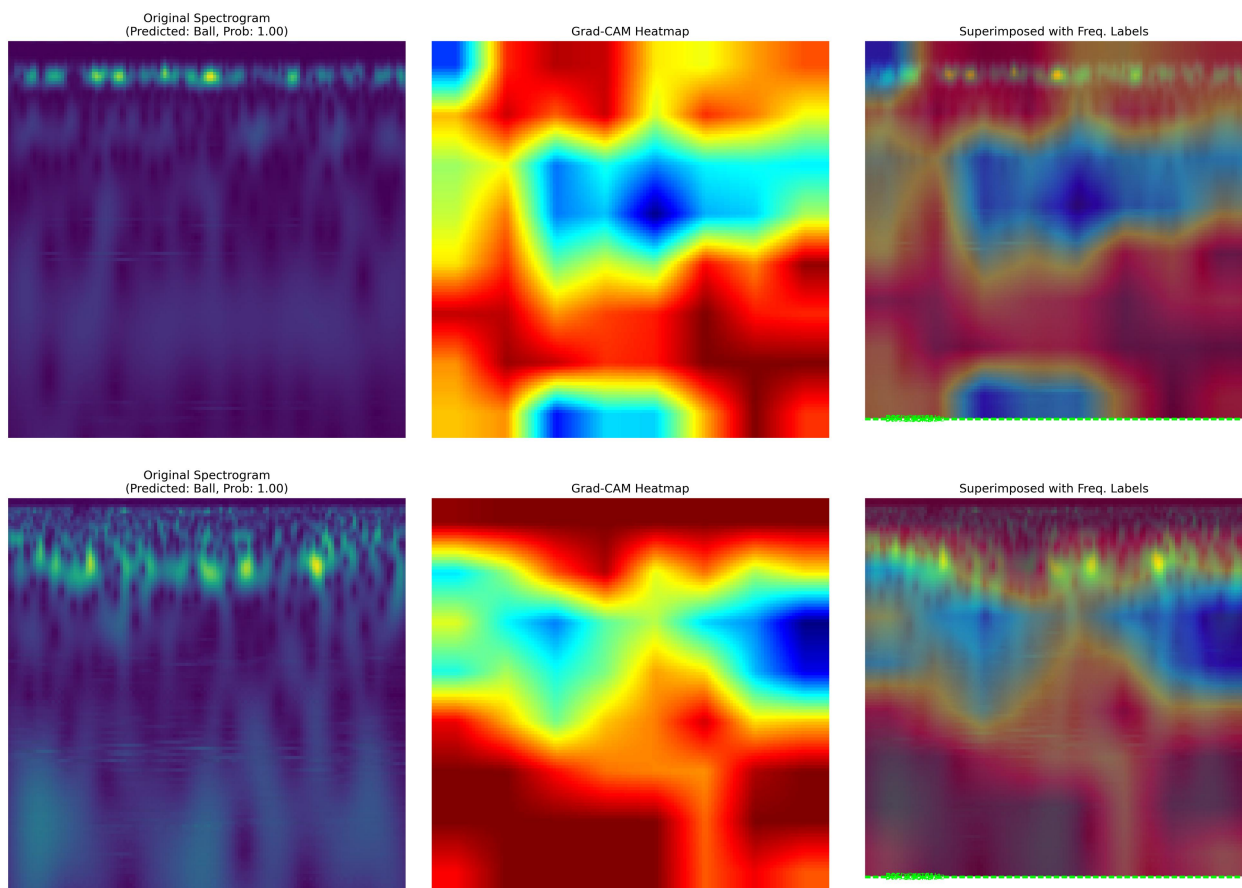


图 5-51 源域样本 B_0021_B021_3 和目标域样本 A layer3 输出热力图

Resnet-18 第 4 层分析

与外圈故障、内圈故障一致，同样是中央区域为强负激活（蓝色）、而外围区域为正激活（黄、红色）的环状或碗状结构。

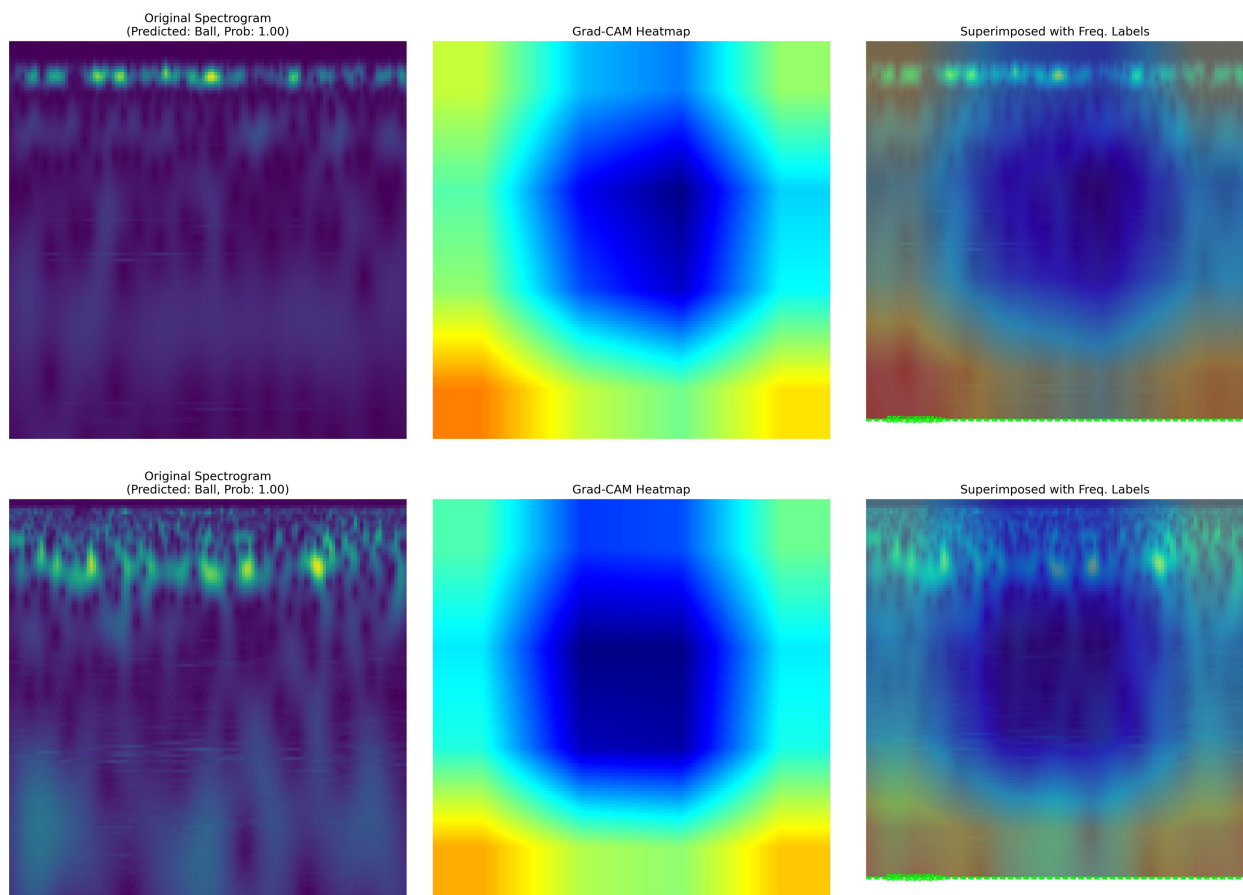


图 5-52 源域样本 B_0021_B021_1 和目标域样本 A_lay4 输出热力图

综上，通过利用 Grad-CAM 对模型从 layer1 到 layer4 进行逐层可视化分析，我们成功地将 ResNet-18 模型内部的黑箱决策过程转化为一个清晰、可追溯的层次化诊断逻辑链条。

我们的分析揭示，模型的学习过程与人类专家的诊断思维高度契合：

从基础特征识别开始 (layer1)：模型首先学习识别构成故障信号的最基本元素，如独立的冲击脉冲或高频能量扰动。

进而组合为局部模式 (layer2)：随后，模型将这些基础特征组合成更具结构性的局部模式，例如识别出冲击的周期性序列或能量的调制包络。

再抽象为全局景象 (layer3)：接着，模型将注意力扩展至整个时间窗，将局部模式抽象为能够定义故障类别的全局特征，如持续稳定的高频周期现象或广泛的非稳定能量分布。

最终形成高级语义 (layer4)：在特征提取的终点，模型形成了最高度抽象的语义概念，将所有具体的故障模式统一归纳为一个故障的宏观概念，并与正常状态的模式形成明确区分。

最为关键的发现是，这一套完整的、从具体到抽象、从微观到宏观的诊断逻辑，在源域（台架数据）和目标域（列车数据）的样本上表现出了卓越的一致性。尽管两个领域的的数据在信噪比和背景噪声上存在显著差异，但模型成功地掌握了故障背后共通的物理本质。这无可辩驳地证明，我们的迁移学习模型不仅学会了如何分类，更重要的是，它识别了轴承故障诊断的内在机理，并将这种理解能力成功地、完整地从一个领域迁移到了另一个领域。

六、主要结论与模型评价

6.1 主要结论

6.1.1 问题一的主要结论

通过实施标准化的预处理流程，包括带通滤波、重采样以对齐域间采样率，以及设计目标数量驱动的动态样本生成机制，有效解决了原始数据中存在的噪声干扰、域间异构和类别不平衡问题。最终，通过并行提取蕴含明确物理意义的 28 个一维特征向量和能够表征精细时频结构的二维 CWT 时频图谱，成功构建了一套面向迁移学习的高质量、多模态特征数据集，为后续建模提供了信息丰富且与故障机理紧密关联的数据基础。

6.1.2 问题二的主要结论

通过系统性的双轨并行评测，遴选出最优的源域故障诊断模型。在针对一维特征的多种机器学习模型横向对比中，多层感知机（MLP）表现最佳，加权 F1 分数达 99.17%。而在针对二维时频图像的深度学习模型对比中，经微调的 ResNet-18 模型凭借其强大的特征学习能力，取得了近乎完美的 100% 分类性能，显著优于自定义 CNN 模型和 MLP 模型。因此，微调的 ResNet-18 模型被确定为本次研究的源域基准模型，其卓越性能为后续的知识迁移奠定了坚实基础。

6.1.3 问题三的主要结论

成功实现了从源域到无标签目标域的知识迁移，并完成了对所有目标域文件的故障标定。通过对比基于最大均值差异（MMD）和领域对抗网络（DANN）的两种迁移策略，并结合双重自监督优化，发现 DANN 方案在领域对齐效果（领域判别器准确率稳定于 51.03%）和最终分类性能（验证集准确率 99.86%）上均表现出微弱但明确的优势。基于最优的 DANN 迁移模型，对 16 个目标域文件进行了标定，结果显示模型对大部分样本的预测置信度超过 99%，证明了该迁移诊断框架的有效性和可靠性。

6.1.4 问题四的主要结论

在事前设计层面，我们论证了“CWT 时频图+ResNet-18”技术路线的内在合理性。CWT 的多分辨率特性与轴承故障信号的物理特征完美匹配，而 ResNet-18 的层次化结构则与从识别基础冲击到认知复杂故障模式的诊断逻辑高度一致。这确保了我们的模型从设计之初就具备了良好的自解释基础。

在迁移过程层面，通过 t-SNE 降维可视化，我们直观地展示了知识迁移的有效性。结果清晰地表明，经过 DANN 迁移学习后，原本在特征空间中混乱不堪的目标域数据，被成功地映射为边界清晰、类别分明的簇，有力地证明了模型通过领域自适应，掌握了共通的、具有高区分度的领域不变特征。

在事后决策层面，通过对 ResNet-18 四个核心层的逐层 Grad-CAM 分析，我们以前所未有的粒度，揭示了模型做出具体诊断的完整思考链条。分析结果证实，模型的决策并非基于偶然的相关性，而是严格遵循与物理机理相符的层次化逻辑。更重要的是，这一诊断逻辑在源域与目标域之间表现出高度的一致性，为迁移知识的有效性和可靠性提供了最强有力的微观证据。

6.2 模型评价

6.2.1 模型优点分析

1. 模型角度

（1）端到端的层次化特征学习：本研究采用以 ResNet-18 为核心的深度学习模型，将复杂的故障诊断任务转化为端到端的图像分类问题。模型能自动从 CWT 时频图中学习从底层（冲击脉冲）到高层（周期、调制规律）的层次化特征，避免了传统方法中对人工设计

特征的高度依赖和潜在的信息损失。

(2) 多维度的综合诊断体系：模型体系不仅关注最终的诊断精度，更通过引入 t-SNE 和 Grad-CAM 等可解释性工具，构建了一个从迁移过程到决策依据的全方位透明化分析框架。这使得模型不仅是一个黑箱分类器，更是一个决策过程可被理解、可被信任的智能诊断系统。

2. 求解算法角度

(1) 先进的领域自适应策略：采用领域对抗神经网络 (DANN) 进行迁移学习，通过引入领域判别器进行对抗训练，能够智能地、自适应地促使模型学习领域不变特征。相比于基于统计矩匹配等传统方法，该算法在处理复杂的、非线性的领域差异时具有更强的能力和更高的泛化性能。

(2) 数据驱动的样本均衡机制：在数据预处理阶段，我们设计了目标数量驱动的动态样本生成机制，通过灵活调整滑动步长，从有限的原始信号中生成了数量均衡且规模充足的训练样本。该策略有效解决了原始数据类别不平衡问题，显著提升了模型的稳定性和对少数类样本的学习效果。

6.2.2 模型缺点分析

(1) 故障条件的理想化假设与改进：本研究的模型建立在源域数据均为单点、非复合故障的基础上。然而，在实际运营环境中，轴承可能出现多种故障并发（如外圈与滚动体同时损伤）或磨损等缓变故障。当前模型对这类复杂故障场景的诊断能力未经充分验证。未来可通过引入复合故障的仿真数据或少量真实数据，对模型进行多标签学习或增量学习的微调，以增强其对复杂工况的识别能力。

(2) 对新发故障类型的识别局限与改进：我们的迁移框架遵循“封闭集”假设，即认为目标域的所有故障类型均已在源域中出现。模型目前缺乏对未知或新发故障类型 (Novelty Detection) 的识别能力，当遇到训练集中未见的故障时，可能会将其误判为已知类别。为提升模型的安全性，可在最终分类层之后集成一个异常检测模块（如基于重构误差的自编码器或高斯混合模型），用于评估新样本与所有已知类别的相似度。当相似度低于某一阈值时，系统可将其标记为“未知类型”并发出预警，实现开放集环境下的安全诊断。

参考文献

- [1]Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization.
- [2] van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(11), 2579-2605.
- [3]洪晓翠,段礼祥,徐继威,等.变工况下轴承故障的残差对抗网络诊断方法[J].石油机械,2022,50(05):32-42.DOI:10.16082/j.cnki.issn.1001-4578.2022.05.005.
- [4]陈强强,戴邵武,戴洪德,等.滚动轴承故障诊断方法综述[J].仪表技术,2019,(09):1-4+42.DOI:10.19432/j.cnki.issn1006-2394.2019.09.001.
- [5]戴文渊.基于实例和特征的迁移学习算法研究[D].上海交通大学,2009.
- [6]郭敏,陈鹏,周超,等.基于坐标注意力关系网络的小样本轴承故障诊断[J].四川大学学报(自然科学版),2024,61(04):338-346.DOI:10.19907/j.0490-6756.2024.047001.
- [7]刘臻,彭珍瑞,王圣杰.小样本下 SE-ResNet 与元迁移学习的变工况轴承故障诊断[J].振动工程学报,2025,38(06):1199-1211.DOI:10.16385/j.cnki.issn.1004-4523.2025.06.008.
- [8]张安安,黄晋英,冀树伟,等.基于卷积神经网络图像分类的轴承故障模式识别[J].振动与冲击,2020,39(04):165-171.DOI:10.13465/j.cnki.jvs.2020.04.021.
- [9]周飞燕,金林鹏,董军.卷积神经网络研究综述[J].计算机学报,2017,40(06):1229-1251.

附录

源程序 1：问题 1 中对 28 个一维指标的计算方法

使用软件：Python

```
def extract_1d_features_new(signal_slice, sample_rate=TARGET_SAMPLE_RATE):
    # 时域特征
    mean_val = np.mean(signal_slice) # 均值
    rms_val = np.sqrt(np.mean(signal_slice**2)) # 均方差
    std_val = np.std(signal_slice) # 标准差
    peak_val = np.max(np.abs(signal_slice)) # 峰值
    kurt_val = kurtosis(signal_slice) # 峭度
    skew_val = skew(signal_slice) # 偏度
    impulse_factor = peak_val / np.mean(np.abs(signal_slice)) if np.mean(np.abs(signal_slice)) != 0 else 0 #
    脉冲因子
    crest_factor = peak_val / rms_val if rms_val != 0 else 0 # 峰值因子
    zero_crossings = np.sum(np.diff(np.sign(signal_slice)) != 0) # 过零率

    # 功率谱密度特征
    freqs, psd = welch(signal_slice, fs=sample_rate, nperseg=min(256, len(signal_slice)))
    psd_energy = np.sum(psd) # 功率谱总能量
    psd_mean = np.mean(psd) # 功率谱均值
    psd_peak_freq = freqs[np.argmax(psd)] # 功率谱对应频率

    # FFT 频谱特征
    N = len(signal_slice)
    fft_vals = np.fft.fft(signal_slice) # 频谱加权平均频率
    fft_mag = np.abs(fft_vals)[N//2:] * 2 / N # 频谱标准差
    fft_freqs = np.fft.fftfreq(N, 1/sample_rate)[N//2:] # 频谱熵

    spectral_mean = np.sum(fft_mag * fft_freqs) / np.sum(fft_mag) if np.sum(fft_mag) != 0 else 0
    spectral_std = np.sqrt(np.sum(((fft_freqs - spectral_mean)**2) * fft_mag) / np.sum(fft_mag)) if
    np.sum(fft_mag) != 0 else 0

    # 寻找前 3 个主要频率分量
    sorted_idx = np.argsort(fft_mag)[::-1]
    top3_freqs = fft_freqs[sorted_idx[:3]] if len(fft_mag) >= 3 else np.pad(fft_freqs, (0, 3-len(fft_mag)),
    'constant')
    top3_mags = fft_mag[sorted_idx[:3]] if len(fft_mag) >= 3 else np.pad(fft_mag, (0, 3-len(fft_mag)),
    'constant')

    # 频带能量特征
    band_energy = {}
    bands = {
        'delta': (0.1, 4),
```

```

        'theta': (4, 8),
        'alpha': (8, 13),
        'beta': (13, 30),
        'gamma': (30, 100)
    }
    for name, (low, high) in bands.items():
        mask = (fft_freqs >= low) & (fft_freqs <= high)
        band_energy[name] = np.sum(fft_mag[mask]**2)

# 非线性特征
hann_window = np.hanning(N)
signal_windowed = signal_slice * hann_window
signal_fft = np.fft.fft(signal_windowed)[:N//2]
spectral_entropy = -np.sum(fft_mag * np.log(fft_mag + 1e-12)) / np.log(N//2)

# 小波变换特征
scales = np.arange(1, 128) # 小波尺度范围
widths = scales * 5.0 / (2 * np.pi * 10) # 对应小波宽度
cwt_matrix = compute_cwt(signal_slice, widths)
wavelet_energy = np.sum(np.abs(cwt_matrix)**2, axis=1)
wavelet_mean = np.mean(wavelet_energy)
wavelet_std = np.std(wavelet_energy)

feature_dict = {
    # 时域特征
    'mean': mean_val, 'rms': rms_val, 'std': std_val,
    'peak': peak_val, 'kurtosis': kurt_val, 'skew': skew_val,
    'impulse_factor': impulse_factor, 'crest_factor': crest_factor,
    'zero_crossings': zero_crossings,

    # 频域特征
    'spectral_mean': spectral_mean, 'spectral_std': spectral_std,
    'spectral_entropy': spectral_entropy,

    # 主要频率分量
    'top1_freq': top3_freqs[0], 'top1_mag': top3_mags[0],
    'top2_freq': top3_freqs[1], 'top2_mag': top3_mags[1],
    'top3_freq': top3_freqs[2], 'top3_mag': top3_mags[2],

    # 功率谱特征
    'psd_energy': psd_energy, 'psd_mean': psd_mean,
    'psd_peak_freq': psd_peak_freq,

    # 频带能量

```

```

'delta_energy': band_energy['delta'],
'theta_energy': band_energy['theta'],
'alpha_energy': band_energy['alpha'],
'beta_energy': band_energy['beta'],
'gamma_energy': band_energy['gamma'],

# 时频特征
'wavelet_mean': wavelet_mean,
'wavelet_std': wavelet_std,
}

return feature_dict

```

源程序 2：问题 1 中对代表性样本的可视化分析

使用软件：Python

```

def create_detailed_visualization(file_path, output_dir):
    print(f'\n 为样本 '{file_path.name}' 生成分析图 ---")

    # 加载和预处理数据
    mat_data = loadmat(file_path)
    de_key = next((key for key in mat_data if key.endswith("_DE_time")), None)
    rpm_key = next((key for key in mat_data if "RPM" in key), None)
    signal = mat_data[de_key].flatten()
    rpm = mat_data[rpm_key].flatten()[0]
    original_sr = get_original_sample_rate(file_path)

    signal_resampled = resample(signal, int(len(signal) * TARGET_SAMPLE_RATE / original_sr))
    slice_signal = signal_resampled[:SLICE_LENGTH]
    slice_normalized = (slice_signal - np.mean(slice_signal)) / np.std(slice_signal)

    # 计算理论故障频率
    fault_freqs = calculate_fault_frequencies(rpm, DE_PARAMS)

    # 绘图
    fig, axes = plt.subplots(3, 1, figsize=(12, 15))
    fig.suptitle(f'Comprehensive Analysis of Sample: {file_path.name}\nFault Type: Inner Race, RPM: {rpm:.0f}',
    fontsize=16)

    # 时域波形图
    time_axis = np.linspace(0, SLICE_LENGTH / TARGET_SAMPLE_RATE, SLICE_LENGTH)
    axes[0].plot(time_axis, slice_normalized, color='royalblue')
    axes[0].set_title('Time-Domain Waveform (Normalized)', fontsize=12)
    axes[0].set_xlabel('Time (s)')
    axes[0].set_ylabel('Amplitude')

```



```

axes[0].grid(True, linestyle='--', alpha=0.6)

# 频域谱图 (FFT)
N = len(slice_normalized)
fft_mag = np.abs(np.fft.fft(slice_normalized))[:N // 2] * 2 / N
freq_axis = np.fft.fftfreq(N, 1/TARGET_SAMPLE_RATE)[:N // 2]
axes[1].plot(freq_axis, fft_mag, color='coral')
axes[1].set_title('Frequency-Domain Spectrum (FFT)', fontsize=12)
axes[1].set_xlabel('Frequency (Hz)')
axes[1].set_ylabel('Amplitude')
axes[1].set_xlim(0, 2000) # 限制频率范围以便观察
axes[1].grid(True, linestyle='--', alpha=0.6)

# 在频谱图上标注理论故障频率
colors = {'fr': 'green', 'BPFI': 'red', 'BPFO': 'purple', 'BSF': 'orange'}
for name, freq in fault_freqs.items():
    if freq > 0 and freq < 2000:
        axes[1].axvline(x=freq, color=colors[name], linestyle='--', label=f'{name}: {freq:.2f} Hz')
        # 标注谐波
        for i in range(2, 5):
            axes[1].axvline(x=freq * i, color=colors[name], linestyle=':', alpha=0.7)
axes[1].legend()

# 时频图 (CWT)
scales = np.arange(1, 129)
coeffs, freqs_cwt = pywt.cwt(slice_normalized, scales, 'cmor1.5-1.0',
sampling_period=1/TARGET_SAMPLE_RATE)
im = axes[2].pcolormesh(time_axis, freqs_cwt, np.abs(coeffs), cmap='viridis', shading='gouraud')
axes[2].set_title('Time-Frequency Spectrogram (CWT)', fontsize=12)
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Frequency (Hz)')
fig.colorbar(im, ax=axes[2], label='Magnitude')

plt.tight_layout(rect=[0, 0, 1, 0.96])
save_path = output_dir / f'detailed_analysis_{file_path.stem}.png'
plt.savefig(save_path)
plt.close()
print(f'单样本分析图已保存至: {save_path}')
print(f'以文件 '{file_path.name}' 为例 (RPM: {rpm:.0f}):')
print(f' - 理论转频 (fr): {fault_freqs['fr']:.2f} Hz')
print(f' - 理论外圈故障频率 (BPFO): {fault_freqs['BPFO']:.2f} Hz')
print(f' - 理论内圈故障频率 (BPFI): {fault_freqs['BPFI']:.2f} Hz')
print(f' - 理论滚动体故障频率 (BSF): {fault_freqs['BSF']:.2f} Hz')

```

源程序 3：问题 2 中卷积神经网络训练

使用软件：Python

```
# @wxz
# 辅助工具： gemini-2.5-pro, google,2025-09-23
# 辅助内容： 1.给出计算指标和公式，让其生成计算代码。2.提供时频图的思路，让其生成代码
```

```
import json
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision.models as models
from torchvision import transforms
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
from PIL import Image
import numpy as np
from pathlib import Path
from tqdm import tqdm
import warnings
import re
from torchviz import make_dot

warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=FutureWarning)

# 设置 Matplotlib 字体以支持中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['font.sans-serif'] = ['STHeiti']
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# --- 1. 参数配置 ---
FEATURE_DIR = Path("./generated_features_balanced")
IMAGE_DIR = FEATURE_DIR / "cwt_images"
# CSV 文件不再需要，因为我们只使用图像和标签
RESULTS_DIR = Path("output/question2_balanced")
MODELS_DIR = Path("output/model")
RESULTS_DIR.mkdir(exist_ok=True, parents=True)
MODELS_DIR.mkdir(exist_ok=True)

# 模型与训练参数
IMG_SIZE = 128 # ResNet 通常使用 224x224 的输入尺寸
```

```

BATCH_SIZE = 32

TEST_SPLIT_RATIO = 0.2
RANDOM_SEED = 42

# --- 策略 A: CustomCNN ---
LEARNING_RATE = 0.001
NUM_EPOCHS = 30 # 对于更深的模型，可以适当减少轮数或使用早停(30-50)
WEIGHT_DECAY = 1e-4

# --- 策略 B: ResNet18 ---
# LEARNING_RATE = 0.0001 # 10x smaller LR!
# NUM_EPOCHS = 30 # Fewer epochs(15-30)
# WEIGHT_DECAY = 1e-4 # Use weight decay for regularization

# --- 2. 设备配置 ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"将使用设备: {device}")
torch.manual_seed(RANDOM_SEED)

# --- 3. 自定义数据集 ---
class SpectrogramDataset(Dataset):
    """一个只加载图像和对应标签的自定义数据集。"""

    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        label = self.labels[idx]

        try:
            # 时频图是单通道的，以 'L' 模式打开
            image = Image.open(img_path).convert("RGB") # ResNet 需要三通道输入
        except FileNotFoundError:
            print(f"警告：找不到文件 {img_path}")
            image = Image.new('RGB', (IMG_SIZE, IMG_SIZE), color='red')

        if self.transform:
            image = self.transform(image)

```

```

        return image, label

# customCNN
class CustomCNN(nn.Module):
    def __init__(self, num_classes, img_size=IMG_SIZE):
        super(CustomCNN, self).__init__()
        self.cnn_branch = nn.Sequential(
            # 输入图像是三通道 RGB，尺寸为 224x224
            nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2, 2), # -> 112x112
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2, 2), # -> 56x56
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128), nn.ReLU(), nn.MaxPool2d(2, 2), # -> 28x28
            nn.AdaptiveAvgPool2d((7, 7)), # 使用自适应池化来得到固定大小的输出
            nn.Flatten()
        )
        # 自适应池化后尺寸为 128 * 7 * 7
        num_cnn_features = 128 * 7 * 7
        self.classifier = nn.Sequential(
            nn.Linear(num_cnn_features, 512),
            nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, x_img):
        img_features = self.cnn_branch(x_img)
        output = self.classifier(img_features)
        return output

# 模型 B: 通用的 ResNet18 模型
class ResNetForSpectrogram(nn.Module):
    def __init__(self, num_classes, pretrained=True):
        super(ResNetForSpectrogram, self).__init__()
        # 加载预训练的 ResNet18 模型
        self.resnet = models.resnet18(pretrained=pretrained)

        ## CWT 图像是彩色的，可以当作 RGB 处理。如果想用灰度图，需要修改第一层
        ## CWT 图像以 RGB 模式加载，所以暂时不需要修改
        ## self.resnet.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)

        # 替换最后的分类层

```

```

num_fts = self.resnet.fc.in_features
self.resnet.fc = nn.Linear(num_fts, num_classes)

def forward(self, x):
    return self.resnet(x)

def train_and_evaluate(model, train_loader, test_loader, criterion, optimizer, num_epochs, class_names,
model_name):
    train_losses, test_losses = [], []
    best_model_wts = model.state_dict()
    best_test_loss = float('inf')

    for epoch in range(num_epochs):
        model.train()
        epoch_train_loss = 0.0
        progress_bar = tqdm(train_loader, desc=f"Epoch {epoch + 1}/{num_epochs}", leave=False)
        for inputs, labels in progress_bar:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs) # <<< 改动
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            epoch_train_loss += loss.item() * inputs.size(0)
            progress_bar.set_postfix(loss=loss.item())

        epoch_train_loss /= len(train_loader.dataset)
        train_losses.append(epoch_train_loss)

        model.eval()
        epoch_test_loss = 0.0
        all_preds, all_labels = [], []
        with torch.no_grad():
            for inputs, labels in test_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs) # <<< 改动
                loss = criterion(outputs, labels)
                epoch_test_loss += loss.item() * inputs.size(0)
                _, preds = torch.max(outputs, 1)
                all_preds.extend(preds.cpu().numpy())
                all_labels.extend(labels.cpu().numpy())

        epoch_test_loss /= len(test_loader.dataset)
        test_losses.append(epoch_test_loss)

```



```

        if epoch_test_loss < best_test_loss:
            best_test_loss = epoch_test_loss
            best_model_wts = model.state_dict()

    print(f"Epoch {epoch + 1}/{num_epochs} | 训练损失: {epoch_train_loss:.4f} | 测试损失: {epoch_test_loss:.4f}")

    # 保存和绘制结果
    # 保存训练过程中的损失值
    loss_data = {'train_losses': train_losses, 'test_losses': test_losses}
    with open(RESULTS_DIR / f'{model_name}_loss_history.json', 'w') as f:
        json.dump(loss_data, f)

    # 绘制损失曲线
    plt.figure(figsize=(10, 6))
    plt.plot(range(1, num_epochs + 1), train_losses, label='训练损失')
    plt.plot(range(1, num_epochs + 1), test_losses, label='测试损失')
    plt.xlabel('Epoch');
    plt.ylabel('Loss');
    plt.title(f'{model_name} 训练和测试损失曲线');
    plt.legend()
    plt.savefig(RESULTS_DIR / f'{model_name}_loss_curves.png');
    plt.close()

    # 生成并保存分类报告
    report = classification_report(all_labels, all_preds, target_names=class_names, digits=4,
output_dict=True)
    pd.DataFrame(report).transpose().to_csv(RESULTS_DIR / f'{model_name}_classification_report.csv')

    # 保存混淆矩阵
    cm = confusion_matrix(all_labels, all_preds)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
    plt.xlabel('预测标签');
    plt.ylabel('真实标签');
    plt.title(f'{model_name} 源域诊断模型混淆矩阵')
    plt.savefig(RESULTS_DIR / f'{model_name}_confusion_matrix.png');
    plt.close()

    return best_model_wts

def main():

```

```

if not IMAGE_DIR.exists():
    print(f'错误: 找不到图像目录 '{IMAGE_DIR}'. ")
    return

all_image_paths = list(IMAGE_DIR.glob('*/*.png'))
class_names = sorted([d.name for d in IMAGE_DIR.iterdir() if d.is_dir()])
label_to_id = {name: i for i, name in enumerate(class_names)}
id_to_label = {i: name for i, name in enumerate(class_names)}

# 保存标签映射文件, 为后续任务做准备
with open(RESULTS_DIR / 'label_to_id.json', 'w') as f:
    json.dump(label_to_id, f, indent=4)
print(f'标签映射已创建并保存: {label_to_id}')

# 从文件路径中提取标签
labels = [label_to_id[p.parent.name] for p in all_image_paths]

# 划分数据集
from sklearn.model_selection import train_test_split
train_paths, test_paths, train_labels, test_labels = train_test_split(
    all_image_paths, labels, test_size=TEST_SPLIT_RATIO, random_state=RANDOM_SEED,
    stratify=labels
)

# 定义图像变换 (ResNet 需要三通道和 ImageNet 的归一化)
data_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

train_dataset = SpectrogramDataset(train_paths, train_labels, transform=data_transforms)
test_dataset = SpectrogramDataset(test_paths, test_labels, transform=data_transforms)

print(f'数据集加载完成。类别: {class_names}')
print(f'训练集样本数: {len(train_dataset)}, 测试集样本数: {len(test_dataset)}')

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

num_classes = len(class_names)

# 选择要运行的模型
# --- 选项 A: 运行自定义的简单 CNN ---

```

```

# model = CustomCNN(num_classes=num_classes, img_size=IMG_SIZE).to(device)
# model_name = "CustomCNN"

# --- 选项 B: 运行通用的 ResNet18 模型 ---
model = ResNetForSpectrogram(num_classes=num_classes, pretrained=True).to(device)
model_name = "ResNet18"

criterion = nn.CrossEntropyLoss()
# optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE,
weight_decay=WEIGHT_DECAY)

print(f"\n--- 使用模型: {model_name} ---")
print(model)

dummy_input = torch.randn(1, 3, 128, 128)
output = model(dummy_input)

print(f"\n--- 开始训练模型 (共 {NUM_EPOCHS} 轮) ---")
best_model_wts = train_and_evaluate(
    model, train_loader, test_loader,
    criterion, optimizer, NUM_EPOCHS, class_names, model_name
)

# --- 保存最佳模型 ---
model.load_state_dict(best_model_wts)
model_save_path = MODELS_DIR / f"source_domain_{model_name}_model.pth"
torch.save(model.state_dict(), model_save_path)
print(f"\n 最佳模型已保存至: {model_save_path}")

print(f"\n 所有结果已保存到 {RESULTS_DIR} 目录中。")

if __name__ == "__main__":
    main()

```

源程序 4：问题 2 中机器学习模型训练代码

使用软件：Python

十种机器学习模型训练分类任务

```

import json
import os
import joblib
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

```

import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier,
HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report
import warnings

from sklearn.metrics import (accuracy_score, f1_score, classification_report,
                             roc_auc_score, recall_score, precision_score,
                             confusion_matrix)

# 忽略一些常见的警告信息，保持输出整洁
warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=FutureWarning)

# 设置 Matplotlib 字体以支持中文显示
plt.rcParams['font.sans-serif'] = ['STHeiti'] # Mac 系统字体
# plt.rcParams['font.sans-serif'] = ['SimHei'] # Windows 系统字体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

def load_and_prepare_data(data_path, label_col='label', label_mapping=None, dropcols=[]):

    # 读取训练集和测试集数据
    data_df = pd.read_csv(data_path)

    print("标签映射关系:", label_mapping)
    data_df[label_col] = data_df[label_col].map(label_mapping)

    feature_cols = [col for col in data_df.columns if col not in [[label_col]+dropcols+["label_id"]]]

    # 确定需要排除的列（合并所有要排除的列到一个列表中）
    exclude_cols = [label_col, "label_id"] + (dropcols if dropcols else [])
    # 使用集合差集来获取特征列（更高效且避免嵌套列表问题）
    feature_cols = list(set(data_df.columns) - set(exclude_cols))
    print(feature_cols)
    print(len(feature_cols))

```

```

X = data_df[feature_cols]
y = data_df['label_id']
# 删除全为 0 的列
zero_cols = X.columns[(X == 0).all(axis=0)] # 找出全 0 的列名
X = X.drop(columns=zero_cols)
# 删除全为空值的列
X.dropna(axis=1, how='all', inplace=True)
# 删除稀有变量
numeric_cols = X.select_dtypes(include=np.number).columns
cols_to_drop_rare = []
for col in numeric_cols:
    if X[col].nunique() <= 1:
        cols_to_drop_rare.append(col)
X.drop(columns=cols_to_drop_rare, inplace=True)
# 用列的中位数填充剩余的缺失值
X.fillna(X.median(), inplace=True)

print(f"数据准备完成, 特征数量: {X.shape[1]}")
print("-" * 30 + "\n")
return X, y

```

```

def handle_outliers(df, columns):
    df_copy = df.copy()
    for col in columns:
        Q1 = df_copy[col].quantile(0.25)
        Q3 = df_copy[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        # 使用 clip 函数将超出边界的值替换为边界值
        df_copy[col] = df_copy[col].clip(lower=lower_bound, upper=upper_bound)
    print("异常值处理完成。")
    print("-" * 30 + "\n")
    return df_copy

```

```

def select_features_with_rf(X, y, top_n=10, output_dir='results'):
    print("--- 使用随机森林进行特征筛选 ---")
    model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
    model.fit(X, y)

    # 创建特征重要性 DataFrame
    importance_df = pd.DataFrame({

```

```

        'Feature': X.columns,
        'Importance': model.feature_importances_
    }).sort_values(by='Importance', ascending=False)

# 保存特征重要性到 CSV 文件
importance_path = os.path.join(output_dir, 'feature_importances.csv')
importance_df.to_csv(importance_path, index=False)
print(f'特征重要性已保存到: {importance_path}')

# 绘制并保存特征重要性图
plt.figure(figsize=(12, 10))
sns.barplot(x='Importance', y='Feature', data=importance_df.head(20))
plt.title(f'Top 20 Feature Importance (from {X.shape[1]} features)')
plt.tight_layout()
plot_path = os.path.join(output_dir, 'feature_importance_plot.png')
plt.savefig(plot_path)
# plt.show()
plt.close()
print(f'特征重要性图已保存到: {plot_path}')

# 选择前 N 个特征
selected_features = importance_df['Feature'].head(top_n).tolist()
print(f'已选出最重要的 {len(selected_features)} 个特征。')
print("-" * 30 + "\n")
return selected_features

```

--- 3. 模型训练与评估 ---

```
def train_and_evaluate_classifiers(X_train, y_train, X_test, y_test, output_dir='results'):
```

```

    # 定义要评估的模型
    models = {
        'LogisticRegression': LogisticRegression(random_state=42, max_iter=1000),
        'DecisionTree': DecisionTreeClassifier(random_state=42),
        'RandomForest': RandomForestClassifier(random_state=42, n_jobs=-1),
        'SVC': SVC(probability=True, random_state=42),
        'KNN': KNeighborsClassifier(n_jobs=-1),
        'GradientBoosting': GradientBoostingClassifier(random_state=42),
        'AdaBoost': AdaBoostClassifier(random_state=42),
        'XGBoost': XGBClassifier(random_state=42, eval_metric='mlogloss', use_label_encoder=False,
n_jobs=-1),
        'MLP': MLPClassifier(random_state=42, max_iter=1000),
        'HistGradientBoosting': HistGradientBoostingClassifier(random_state=42)
    }

```



```

performance_records = []
best_metrics = {'f1': 0}
best_model = None
best_model_name = ""

# 用于保存所有评估报告的字典
all_reports = {}

for name, model in models.items():
    print(f"正在训练: {name}...")
    model.fit(X_train, y_train)

    # 训练集上的预测和评估
    y_pred_train = model.predict(X_train)
    y_proba_train = model.predict_proba(X_train) if hasattr(model, "predict_proba") else None

    # 测试集上的预测和评估
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test) if hasattr(model, "predict_proba") else None

    # 计算各种指标
    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='weighted')

    # 计算其它指标
    weighted_recall = recall_score(y_test, y_pred, average='weighted')
    weighted_precision = precision_score(y_test, y_pred, average='weighted')

    # 计算 ROC AUC (如果模型支持概率预测)
    roc_auc = roc_auc_score(y_test, y_proba, multi_class='ovr') if y_proba is not None else None

    # 分类报告
    report = classification_report(y_test, y_pred, output_dict=True)
    all_reports[name] = report

    # 保存性能指标
    performance_records.append({
        'Model': name,
        'Accuracy': accuracy,
        'Weighted_F1': f1,
        'Weighted_Recall': weighted_recall,
        'Weighted_Precision': weighted_precision,
        'ROC_AUC': roc_auc,
    })

```

```

        'Train_Accuracy': accuracy_score(y_train, y_pred_train),
        'Train_F1': f1_score(y_train, y_pred_train, average='weighted')
    })
    print(f'{str(name)}-> Accuracy: {accuracy:.4f}, Weighted F1: {f1:.4f}, \n Recall:
{weighted_recall:.4f}, Precision: {weighted_precision:.4f}')

    if f1 > best_metrics['f1']:
        best_metrics = {
            'model_name': name,
            'f1': f1,
            'accuracy': accuracy,
            'recall': weighted_recall,
            'precision': weighted_precision,
            'roc_auc': roc_auc
        }
        best_model = model
        best_model_name = name

print(f'\n 性能最佳的模型是: {best_model_name}')
print("最佳模型指标:")
for metric, value in best_metrics.items():
    print(f'{metric}<10}: {value}')

# 保存评估报告
report_path = os.path.join(output_dir, 'classification_reports.json')
with open(report_path, 'w') as f:
    json.dump(all_reports, f, indent=4)
print(f'\n 所有模型的分类型评估报告已保存到: {report_path}')

# 保存最佳模型
if best_model is not None:
    model_path = os.path.join(output_dir, 'best_model.pkl')
    joblib.dump(best_model, model_path)
    print(f'最佳模型已保存到: {model_path}')

# 保存最佳模型的测试集预测结果详细报告
best_y_pred = best_model.predict(X_test)
best_report = classification_report(y_test, best_y_pred, output_dict=False)
best_report_path = os.path.join(output_dir, 'best_model_test_report.txt')
cm = confusion_matrix(y_test, best_y_pred)
with open(best_report_path, 'w') as f:
    f.write(f'最佳模型: {best_model_name}\n')
    f.write("测试集分类报告:\n")
    f.write(best_report)

```

```

        f.write("\n\n 混淆矩阵:\n")
        f.write(str(cm))
    print(f'最佳模型的详细测试报告已保存到: {best_report_path}')
    plt.figure(figsize=(10, 8))
    class_names = ["Ball", "InnerRace", "Normal", "OuterRace"]

    sns.heatmap(cm,          annot=True,          fmt='d',          cmap='Blues',          xticklabels=class_names,
yticklabels=class_names)
    plt.xlabel('预测标签'); plt.ylabel('真实标签'); plt.title(f'{best_model_name} 源域诊断模型混淆矩
阵')

    plt.savefig(os.path.join(output_dir, f'{best_model_name}_confusion_matrix.png'))
    plt.close()

# 创建并保存性能对比 DataFrame
performance_df = pd.DataFrame(performance_records).sort_values(by='Weighted_F1', ascending=False)
performance_path = os.path.join(output_dir, 'model_performance_comparison.csv')
performance_df.to_csv(performance_path, index=False)
print(f'\n 模型性能对比结果已保存到: {performance_path}')
print("-" * 30 + "\n")

return best_model, performance_df
# --- 4. 预测与结果保存 ---

def predict_and_save(model, X_test, output_dir='results'):
    predictions = model.predict(X_test)

    results_df = pd.DataFrame({
        'Predicted_Label_ID': predictions
    })

    results_path = os.path.join(output_dir, 'test_predictions.csv')
    results_df.to_csv(results_path, index=False)

    print(f'预测结果已保存到: {results_path}')
    print("预测结果预览:")
    print(results_df.head())
    print("-" * 30 + "\n")

# --- 主函数 ---

def main():

    DATA_FILE = "./generated_features_balanced/one_dimensional_features_balanced.csv"
    LABEL_COLUMN = 'label'

```

```

DROP_COLS=['source_file', 'slice_index']
OUTPUT_DIR = './output/question2_balanced'
LABEL_MAPPING = {
    "Ball": 0,
    "InnerRace": 1,
    "Normal": 2,
    "OuterRace": 3
}

# 创建输出目录
if not os.path.exists(OUTPUT_DIR):
    os.makedirs(OUTPUT_DIR)

X, y = load_and_prepare_data(DATA_FILE,
label_col=LABEL_COLUMN,dropcols=DROP_COLS,label_mapping=LABEL_MAPPING)
# 将数据划分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# 特征工程
# 异常值处理 (仅处理数值型特征)
numeric_cols = X_train.select_dtypes(include=np.number).columns.tolist()
X_train = handle_outliers(X_train, numeric_cols)
X_test = handle_outliers(X_test, numeric_cols)
# 特征筛选
selected_features = select_features_with_rf(X_train, y_train, top_n=15, output_dir=OUTPUT_DIR)
X_train_selected = X_train[selected_features]
X_test_selected = X_test[selected_features]
# 特征标准化
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_selected)
X_test_scaled = scaler.transform(X_test_selected)
print("标准化完成。")
print("-" * 30 + "\n")
# 训练和评估分类器
best_model, performance_df = train_and_evaluate_classifiers(
    X_train_scaled, y_train, X_test_scaled, y_test, output_dir=OUTPUT_DIR
)
print("模型性能对比:")
print(performance_df)
# 最终预测与保存
predict_and_save(best_model, X_test_scaled, output_dir=OUTPUT_DIR)

print("=== 所有流程已成功完成! ===")

```

```
if __name__ == '__main__':
    main()
```

源程序 5：问题 3 中 MMD LOSS 的计算方式

使用软件：Python

```
def gaussian_kernel(source, target, kernel_mul=2.0, kernel_num=5, fix_sigma=None):
    """计算高斯核矩阵"""
    n_samples = int(source.size(0)) + int(target.size(0))
    total = torch.cat([source, target], dim=0)
    total0 = total.unsqueeze(0).expand(n_samples, n_samples, total.size(1))
    total1 = total.unsqueeze(1).expand(n_samples, n_samples, total.size(1))
    l2_distance = ((total0 - total1) ** 2).sum(2)

    if fix_sigma:
        bandwidth = fix_sigma
    else:
        bandwidth = torch.sum(l2_distance.data) / (n_samples ** 2 - n_samples)

    bandwidth /= kernel_mul ** (kernel_num // 2)
    bandwidth_list = [bandwidth * (kernel_mul ** i) for i in range(kernel_num)]
    kernel_val = [torch.exp(-l2_distance / band) for band in bandwidth_list]
    return sum(kernel_val) / len(kernel_val)

def mmd_loss(source, target, kernel_mul=2.0, kernel_num=5, fix_sigma=None):
    """计算 MMD 损失"""
    batch_size_source = int(source.size(0))
    kernels = gaussian_kernel(source, target, kernel_mul=kernel_mul, kernel_num=kernel_num,
fix_sigma=fix_sigma)
    XX = kernels[:batch_size_source, :batch_size_source]
    YY = kernels[batch_size_source:, batch_size_source:]
    XY = kernels[:batch_size_source, batch_size_source:]
    return torch.mean(XX) + torch.mean(YY) - 2 * torch.mean(XY)
```

源程序 6：问题 3 中 DANN 和迁移学习模型

使用软件：Python

```
lass GradReverse(Function):
```

```
    @staticmethod
    def forward(ctx, x, alpha):
        ctx.alpha = alpha
        return x.view_as(x)
```

```
    @staticmethod
```

```

def backward(ctx, grad_output):
    return grad_output.neg() * ctx.alpha, None

class TransferModel(nn.Module):
    def __init__(self, num_classes=4):
        super(TransferModel, self).__init__()
        base_model = models.resnet18(pretrained=True) # 使用预训练权重以加速收敛
        self.feature_extractor = nn.Sequential(*list(base_model.children())[:-1])
        num_fts = base_model.fc.in_features

        self.classifier = nn.Linear(num_fts, num_classes)
        self.domain_discriminator = nn.Sequential(
            nn.Linear(num_fts, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 1)
        )

    def load_source_weights(self, path=SOURCE_MODEL_PATH):
        if not path.exists():
            print("警告: 未找到源域预训练模型, 将使用 ImageNet 预训练的特征提取器。")
            return

        print(f'正在加载源域预训练模型: {path}')
        try:
            self.load_state_dict(torch.load(path, map_location=device), strict=False)
            print("成功加载源域预训练模型权重。")
        except Exception as e:
            print(f'加载源域模型权重失败: {e}。将继续使用 ImageNet 预训练权重。")

    def forward(self, x, alpha=1.0):
        features = self.feature_extractor(x).view(x.size(0), -1)
        class_output = self.classifier(features)

        if self.training and TRANSFER_METHOD == 'DANN':
            reverse_feature = GradReverse.apply(features, alpha)
            domain_output = self.domain_discriminator(reverse_feature).squeeze()
            return class_output, domain_output, features
        elif TRANSFER_METHOD == 'DANN':
            domain_output = self.domain_discriminator(features).squeeze()
            return class_output, domain_output, features
        else:
            return class_output, features

```


源程序 7：问题 3 中的迁移学习过程

使用软件：Python

```
def train_transfer(model, source_loader, val_loader, target_loader, optimizer, class_names):
    class_criterion = nn.CrossEntropyLoss()
    domain_criterion = nn.BCEWithLogitsLoss() if TRANSFER_METHOD == 'DANN' else mmd_loss
    history = {'train_class_loss': [], 'train_domain_loss': [], 'train_pseudo_loss': [],
              'train_entropy_loss': [], 'val_class_loss': [], 'pseudo_labels_used': [], 'pseudo_avg_confidence':
    []}

    analysis_log = []
    best_composite_score, patience_counter = float('inf'), 0

    print(f"\n--- 开始 {TRANSFER_METHOD} 迁移学习训练 ---")
    for epoch in range(NUM_EPOCHS):
        model.train()
        len_dl = min(len(source_loader), len(target_loader))
        iter_s, iter_t = iter(source_loader), iter(target_loader)
        epoch_class_loss, epoch_domain_loss, epoch_pseudo_loss, epoch_entropy_loss = 0.0, 0.0, 0.0, 0.0
        total_pseudo_labels_used_in_epoch = 0
        sum_pseudo_confidence_in_epoch = 0.0

        for i in tqdm(range(len_dl), desc=f"Epoch {epoch + 1}/{NUM_EPOCHS} [训练]"):
            p = (i + epoch * len_dl) / (NUM_EPOCHS * len_dl)
            alpha = 2. / (1. + np.exp(-10 * p)) - 1
            s_img, s_labels = next(iter_s)
            t_img, _ = next(iter_t)
            s_img, s_labels, t_img = s_img.to(device), s_labels.to(device), t_img.to(device)
            optimizer.zero_grad()

            # --- 前向传播与损失计算 ---
            if TRANSFER_METHOD == 'DANN':
                s_class_out, s_domain_out, _ = model(s_img, alpha)
                t_class_out, t_domain_out, _ = model(t_img, alpha)
                loss_s_class = class_criterion(s_class_out, s_labels)
                loss_domain = domain_criterion(s_domain_out, torch.zeros_like(s_domain_out)) + \
                    domain_criterion(t_domain_out, torch.ones_like(t_domain_out))
                total_loss = loss_s_class + LAMBDA_DANN * loss_domain
            else: # MMD
                s_class_out, s_features = model(s_img)
                t_class_out, t_features = model(t_img)
                loss_s_class = class_criterion(s_class_out, s_labels)
                loss_domain = domain_criterion(s_features, t_features)
                total_loss = loss_s_class + LAMBDA_MMD * loss_domain

            # 伪标签
```

```

if USE_PSEUDO_LABELING and epoch >= PSEUDO_START_EPOCH:
    with torch.no_grad():
        t_probs = torch.softmax(t_class_out, dim=1)
        max_probs, pseudo_labels = torch.max(t_probs, 1)
        mask = max_probs > PSEUDO_LABEL_THRESHOLD
    if mask.any():
        loss_pseudo = class_criterion(t_class_out[mask], pseudo_labels[mask])
        total_loss += LAMBDA_PSEUDO * loss_pseudo
        epoch_pseudo_loss += loss_pseudo.item()
        # 【新增】更新统计数据
        total_pseudo_labels_used_in_epoch += mask.sum().item()
        sum_pseudo_confidence_in_epoch += max_probs[mask].sum().item()
# 熵最小化损失
if USE_ENTROPY_MINIMIZATION:
    # 获取目标域输出的概率分布
    t_probs = torch.softmax(t_class_out, dim=1)
    # 计算熵:  $H(p) = -\sum(p * \log(p))$ 
    # 为防止  $\log(0)$ , 添加一个小的 epsilon ( $1e-8$ )
    loss_entropy = -torch.mean(torch.sum(t_probs * torch.log(t_probs + 1e-8), dim=1))
    total_loss += LAMBDA_ENTROPY * loss_entropy
    epoch_entropy_loss += loss_entropy.item()

total_loss.backward()
optimizer.step()
epoch_class_loss += loss_s_class.item()
epoch_domain_loss += loss_domain.item()

# --- Epoch 结束, 进行评估与模型选择 ---
model.eval()
avg_val_loss, domain_metric_val = 0.0, 0.0
with torch.no_grad():
    for v_img, v_labels in val_loader:
        v_class_out, *_ = model(v_img.to(device))
        avg_val_loss += class_criterion(v_class_out, v_labels.to(device)).item()
avg_val_loss /= len(val_loader)

# 计算当前 epoch 的域度量 (用于复合分数)
if TRANSFER_METHOD == 'DANN':
    domain_accuracy = evaluate_domain_classifier(model, source_loader, target_loader,
save=False)
    domain_metric_val = abs(domain_accuracy - 0.5)
    composite_score = W_CLASS_LOSS * avg_val_loss + W_DANN * domain_metric_val
else: # MMD
    mmd_scores = []

```

```

with torch.no_grad():
    for (s_img, _), (t_img, _) in zip(source_loader, target_loader):
        _, s_feat = model(s_img.to(device))
        _, t_feat = model(t_img.to(device))
        mmd_scores.append(mmd_loss(s_feat, t_feat).item())
        if len(mmd_scores) > 20: break # 加快速度
    domain_metric_val = np.mean(mmd_scores)
    composite_score = W_CLASS_LOSS * avg_val_loss + W_MMD * domain_metric_val

# 记录历史
history['train_class_loss'].append(epoch_class_loss / len_dl)
history['train_domain_loss'].append(epoch_domain_loss / len_dl)
history['train_pseudo_loss'].append(epoch_pseudo_loss / len_dl)
history['train_entropy_loss'].append(epoch_entropy_loss / len_dl)
history['val_class_loss'].append(avg_val_loss)
avg_confidence = (
    sum_pseudo_confidence_in_epoch / total_pseudo_labels_used_in_epoch) if
total_pseudo_labels_used_in_epoch > 0 else 0.0
history['pseudo_labels_used'].append(total_pseudo_labels_used_in_epoch)
history['pseudo_avg_confidence'].append(avg_confidence)

print(
    f"\nEpoch {epoch + 1}/{NUM_EPOCHS} | Val Loss: {avg_val_loss:.4f} | Domain Metric:
{domain_metric_val:.4f} | Composite Score: {composite_score:.4f}")
    print(f"      - Pseudo-Labels Used: {total_pseudo_labels_used_in_epoch}, Avg Confidence:
{avg_confidence:.4f}")
    # 模型保存与早停
    if composite_score < best_composite_score:
        best_composite_score = composite_score
        patience_counter = 0
        torch.save(model.state_dict(), BEST_MODEL_PATH)
        # 【恢复】进行深入分析
        metrics = run_epoch_analysis(model, source_loader, val_loader, target_loader, class_names,
epoch + 1)
        analysis_log.append(
            {"epoch": epoch + 1, "val_loss": avg_val_loss, "composite_score": composite_score},
            **metrics)
        else:
            patience_counter += 1
            print(f"      - 未发现更优模型，耐心计数：
{patience_counter}/{EARLY_STOPPING_PATIENCE}")

    torch.save(model.state_dict(), FINAL_MODEL_PATH)
    if patience_counter >= EARLY_STOPPING_PATIENCE:

```

```

print("触发早停机制，训练终止。")
break

```

```

return history, analysis_log

```

源程序 8：问题 4 中的 Grad-cam 图生成

使用软件：Python

```

def generate_grad_cam_visualizations(model, dataloader, id_to_label, save_dir, theoretical_params,
device,specific_files_to_analyze=None):

```

```

    if specific_files_to_analyze:

```

```

        print(f"\n--- 任务四：开始生成指定样本的 Grad-CAM 可解释性分析图 ---")

```

```

        print(f"[*] 将只分析以下 {len(specific_files_to_analyze)} 个文件: {specific_files_to_analyze}")

```

```

    else:

```

```

        print("\n--- 任务四：开始生成所有样本的 Grad-CAM 可解释性分析图 ---")

```

```

    print("\n--- 任务四：开始生成 Grad-CAM 可解释性分析图 ---")

```

```

    save_dir.mkdir(parents=True, exist_ok=True)

```

```

    model.eval()

```

```

    # 找到 ResNet-18 的最后一个卷积块，这是 Grad-CAM 通常关注的目标层

```

```

    # 在你的 TransferModel 中，它位于 feature_extractor 的最后一个模块

```

```

    # ResNet18 的 feature_extractor 是 Sequential(*list(base_model.children())[:-1])

```

```

    # 最后一个卷积层是 base_model.layer4[1].conv2 (对于 ResNet)

```

```

    # 我们可以通过 feature_extractor[-1]访问 ResNet 的最后一个 block (layer4)

```

```

    # 然后再访问该 block 中的最后一个卷积层

```

```

    # 对于 ResNet，通常是 layer4[1].conv2 或者直接使用 layer4

```

```

    # 这里我们选择 feature_extractor 的最后一个模块作为 target_layer，它通常是 ResNet 的 layer4

```

```

    target_layer = model.feature_extractor[LAY_NUM]

```

```

    # 如果你发现热力图不理想，可以尝试更精细的层，例如：

```

```

    # target_layer = model.feature_extractor[6] # corresponds to layer3

```

```

    # target_layer = model.feature_extractor[7] # corresponds to layer4

```

```

    # 或者更具体到最后一个卷积层: model.feature_extractor[-1][1].conv2

```

```

    # 注册前向和后向 hook

```

```

    handle_forward = target_layer.register_forward_hook(forward_hook)

```

```

    handle_backward = target_layer.register_backward_hook(backward_hook)

```

```

    # 遍历目标域数据

```

```

    for images, filenames in dataloader:

```

```

        images = images.to(device)

```

```

        # 前向传播获取预测结果

```

```

        # 在 eval 模式下，你的模型 forward 返回 class_output, features (或 class_output,
domain_output, features for DANN)

```

```

        # 这里我们只需要 class_output

```

```

model_output = model(images)
if isinstance(model_output, tuple): # DANN 模式
    class_outputs = model_output[0]
else: # 非 DANN 模式
    class_outputs = model_output

pred_probs = F.softmax(class_outputs, dim=1)
pred_indices = torch.argmax(pred_probs, dim=1)

# 对 batch 中的每张图片进行处理
for i in range(images.size(0)):
    current_filename = filenames[i]
    if specific_files_to_analyze and current_filename not in specific_files_to_analyze:
        continue # 如果提供了列表且当前文件不在其中，则跳过

    model.zero_grad()

    # 针对预测得分最高的类别进行反向传播
    predicted_score = class_outputs[i, pred_indices[i]]
    # 确保只对当前预测的最高分进行反向传播
    predicted_score.backward(retain_graph=True) # retain_graph=True 因为我们在循环中多次反向传播

    # 计算 Grad-CAM 热力图
    # 全局平均池化梯度，得到神经元重要性权重 (alpha)
    # gradients 形状 [N, C, H, W]， activations 形状 [N, C, H_feat, W_feat]
    # pooled_gradients 形状 [C]
    pooled_gradients = torch.mean(gradients[i], dim=[1, 2])

    # 用权重乘以对应的激活图
    # activations[i] 的形状是 [C, H_feat, W_feat]
    # 这里是逐通道相乘
    weighted_activations = activations[i] * pooled_gradients.unsqueeze(-1).unsqueeze(-1)

    # 将所有通道加权求和，并通过 ReLU
    heatmap = torch.sum(weighted_activations, dim=0).squeeze()
    heatmap = F.relu(heatmap)

    # 归一化热力图到 [0, 1]
    if torch.max(heatmap) > 0:
        heatmap /= torch.max(heatmap)
    else:
        heatmap = torch.zeros_like(heatmap) # 避免除以零

```

```

# 可视化
# 将原始图像反归一化并转为 OpenCV 格式
original_image_tensor = denormalize(images[i])
original_image_np = original_image_tensor.cpu().permute(1, 2, 0).numpy()
original_image_np = (original_image_np * 255).astype(np.uint8)

# b. 将热力图调整大小并应用色彩映射
heatmap_np = heatmap.cpu().detach().numpy()
# 确保 resize 到原始图像的尺寸
heatmap_resized = cv2.resize(heatmap_np, (original_image_np.shape[1],
original_image_np.shape[0]))
heatmap_colored = cv2.applyColorMap(np.uint8(255 * heatmap_resized),
cv2.COLORMAP_JET)

# c. 叠加并保存图像
original_filename = Path(filenamees[i]).stem
predicted_label = id_to_label[pred_indices[i].item()]
prob_val = pred_probs[i, pred_indices[i]].item()

save_path = save_dir /
f"{original_filename}_pred_{predicted_label}_prob_{prob_val:.2f}.png"

# 使用新的绘图函数来合并显示
plot_grad_cam_and_frequencies(original_image_np, heatmap_colored, predicted_label,
prob_val,
                                theoretical_params['freqs'],
                                theoretical_params['sampling_rate_khz'],
                                IMG_SIZE, save_path)

if (i + 1) % 10 == 0:
    print(f" 已处理 {i+1}/{images.size(0)} 张图片，保存至 {save_path.name}")

# 移除 hooks 以防内存泄漏
handle_forward.remove()
handle_backward.remove()
print(f"--- Grad-CAM 分析完成，结果已保存至目录: {save_dir} ---")

```